Московский политехнический университет

Отчёт

по дисциплине «Программирование криптографических алгоритмов»

Выполнил студент
группы 241-372:
Сергеев Н.А.
Преподаватель:
Бутакова Н.Г.

Москва, 2026 г.

Содержание

Содержание 3
Аннотация 5
Введение 6
Общий интерфейс 6
Блок A: Шифры однозначной замены 15
1. Шифр Атбаш 15
Блок B: Шифры многозначной замены 21
5. Шифр Белазо 21
Блок C: Шифры блочной замены 28
8. Матричный шифр 28
Блок D: Шифры перестановки 38
10. Вертикальная перестановка 38
Блок E: Шифры гаммирования 56
13. Одноразовый блокнот Шеннона 56
Блок F: Поточные шифры 66
15. Поточный шифр A5/1 66
Блок G: Комбинационные шифры 82
19. Шифр AES 82
Блок H: Ассиметричные шифры 99
21. Шифр RSA 99

Блок I: алгоритмы цифровых подписей 110
24. Цифровая подпись RSA 110
Блок J: Стандарты цифровых подписей 119
27. Цифровая подпись ГОСТ Р 34.10-2012 119
Блок K: обмен ключами 137
28. Обмен ключами по алгоритму Диффи-Хеллмана 137
Заключение 150
Приложение А. Дополнительные реализации алгоритмов 151
17. Шифр МАГМА 152
20. Шифр Кузнечик 157
Приложение Б. Полная таблица расчёта A5/1 для слова ПЕРЕСДАЧА 162

Аннотация

Среда программирования: PyCharm

Язык программирования: python

Процедуры для запуска программ: скачать папку с программами, открыть файл main.py, запустить его, при необходимости выбрать интерпретатор, откроется меню со всеми шифрами, дальше действовать по требованиям программы.

Пословица — текст по варианту 20: Леопард не может изменить своих пятен.

Контрольный текст длиной 1000 символов (вариант 20):

Леопард не может изменить своих пятен. Проверка по варианту №20!

Введение

Цель работы — изучить реализацию криптографических алгоритмов и проверить их работу на практических примерах. В отчёте рассматриваются шифрование и расшифрование, формирование и проверка цифровой подписи, а также согласование общего ключа.

Для этого реализации объединены общим консольным интерфейсом. Для алгоритмов приведены описание, блок-схемы, код и результаты выполнения; параметры и сообщения вводятся согласно логике соответствующей программы.

Общий интерфейс

Данный интерфейс объединяет все программы с шифрами в одну программу.

Для создания приложения выбран консольный интерфейс. Пользователь выбирает шифр, после чего пользователю предоставляется выбор: зашифровать, расшифровать, выход из программы, а также в индивидуальном порядке ввести какие-то дополнительные данные.

Код программы:

```
import runpy
from pathlib import Path


BASE_DIR = Path(__file__).parent


def run_script(relative_path):
    path = BASE_DIR / relative_path
```

```
    try:
        print("\n" + "=" * 60)
        print(f"Запуск: {path.name}")
        print("=" * 60 + "\n")
        runpy.run_path(str(path), run_name="__main__")
    except SystemExit:
        # Возврат в меню, если вложенная лабораторная программа вызвала exit.
        pass
    except FileNotFoundError:
        print(f"\nФайл не найден:\n{path}")
    except Exception as exc:
        print(f"\nОшибка:\n{exc}")
    input("\nНажмите Enter для продолжения...")


def classic_menu():
    options = {
        "1": ("Атбаш", "Лабы Тема 1/Лаб 1_1/1.Шифр Атбаш.py"),
        "2": ("Белазо", "Лабы Тема 1/Лаб 1_2/5.Шифр Белазо.py"),
        "3": ("Матричный шифр", "Лабы Тема 1/Лаб 1_3/8.Матричный шифр.py"),
        "4": ("Вертикальная перестановка", "Лабы Тема 1/Лаб 1_4/10.Вертикальная перестановка.py"),
    }
```

```
    while True:
        print("\n" + "=" * 60)
        print("КЛАССИЧЕСКИЕ ШИФРЫ")
        print("=" * 60)
        for number, (name, _) in options.items():
            print(f"{number} - {name}")
        print("0 - Назад")
        choice = input("\nВаш выбор: ").strip()
        if choice == "0":
            return
        if choice in options:
            run_script(options[choice][1])
        else:
            print("Неверный выбор")



def symmetric_menu():
    options = {
        "1": ("Одноразовый блокнот Шеннона", "Лабы Тема 1/Лаб 1_5/13.Одноразовый блокнот К.Шеннона.py"),
        "2": ("A5/1", "Лабы Тема 1/Лаб 1_6/15.A5_1.py"),
        "3": ("AES", "Лабы Тема 1/Лаб 1_7/19.AES.py"),
    }
```

```
    while True:
        print("\n" + "=" * 60)
        print("СИММЕТРИЧНЫЕ АЛГОРИТМЫ")
        print("=" * 60)
        for number, (name, _) in options.items():
            print(f"{number} - {name}")
        print("0 - Назад")
        choice = input("\nВаш выбор: ").strip()
        if choice == "0":
            return
        if choice in options:
            run_script(options[choice][1])
        else:
            print("Неверный выбор")



def asymmetric_menu():
    options = {
        "1": ("RSA (шифрование)", "Лабы Тема 2/Лаб 2_1/21-23.RSA_Elgamal_ECC.py"),
        "2": ("Диффи—Хеллман", "Лабы Тема 4/Лаб 4_1/28.Диффи-Хеллман.py"),
    }
    while True:
```

```
        print("\n" + "=" * 60)
        print("АСИММЕТРИЧНЫЕ АЛГОРИТМЫ И ОБМЕН КЛЮЧАМИ")
        print("=" * 60)
        for number, (name, _) in options.items():
            print(f"{number} - {name}")
        print("0 - Назад")
        choice = input("\nВаш выбор: ").strip()
        if choice == "0":
            return
        if choice in options:
            run_script(options[choice][1])
        else:
            print("Неверный выбор")


def signature_menu():
    options = {
        "1": ("Цифровая подпись RSA", "Лабы Тема 3/Лаб 3_1/24.RSA.py"),
        "2": ("ГОСТ Р 34.10-2012", "Лабы Тема 3/Лаб 3_2/27.ГОСТ_Р_34.10-2012.py"),
    }
    while True:
        print("\n" + "=" * 60)
```

```
        print("ЭЛЕКТРОННАЯ ПОДПИСЬ")
        print("=" * 60)
        for number, (name, _) in options.items():
            print(f"{number} - {name}")
        print("0 - Назад")
        choice = input("\nВаш выбор: ").strip()
        if choice == "0":
            return
        if choice in options:
            run_script(options[choice][1])
        else:
            print("Неверный выбор")


def main():
    menus = {
        "1": ("Классические шифры", classic_menu),
        "2": ("Симметричные алгоритмы", symmetric_menu),
        "3": ("Асимметричные алгоритмы и обмен ключами", asymmetric_menu),
        "4": ("Электронная подпись", signature_menu),
    }
    while True:
```

```
        print("\n" + "=" * 60)
        print("ГЛАВНОЕ МЕНЮ КРИПТОГРАФИЧЕСКИХ АЛГОРИТМОВ")
        print("=" * 60)
        for number, (name, _) in menus.items():
            print(f"{number} - {name}")
        print("0 - Выход")
        choice = input("\nВаш выбор: ").strip()
        if choice == "0":
            print("\nДо свидания!")
            break
        if choice in menus:
            menus[choice][1]()
        else:
            print("\nНеверный выбор")


if __name__ == "__main__":
    main()
```

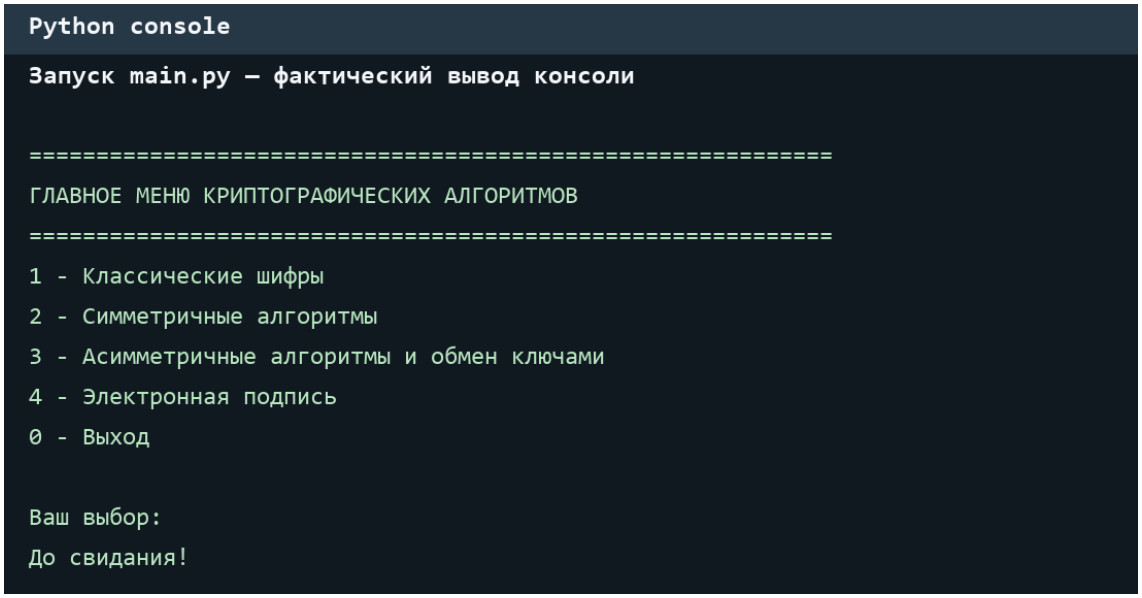


Рисунок 1. Общий интерфейс.

Блок-схема:

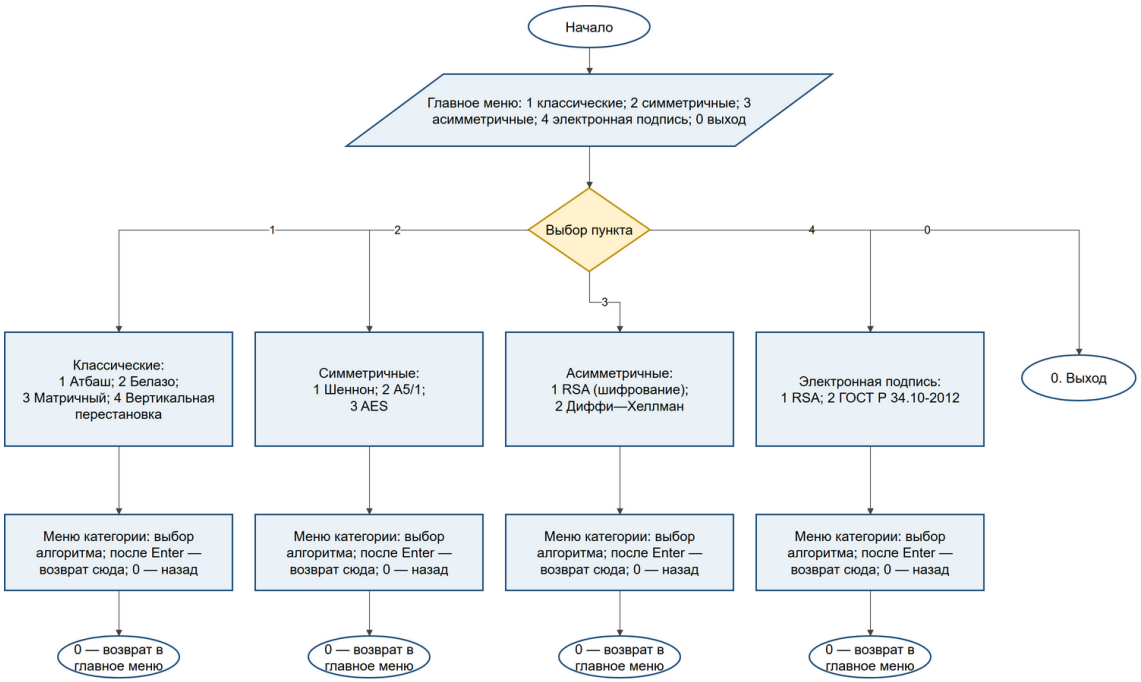


Рисунок 2. Блок-схема интерфейса всех шифров.

Блок А: Шифры однозначной замены

1. Шифр Атбаш

Шифр АТБАШ - шифр простой замены. Алгоритм представляет собой замену i-ой буквы алфавита на n-i+1 букву алфавита, где n – размерность алфавита.

Блок-схема:

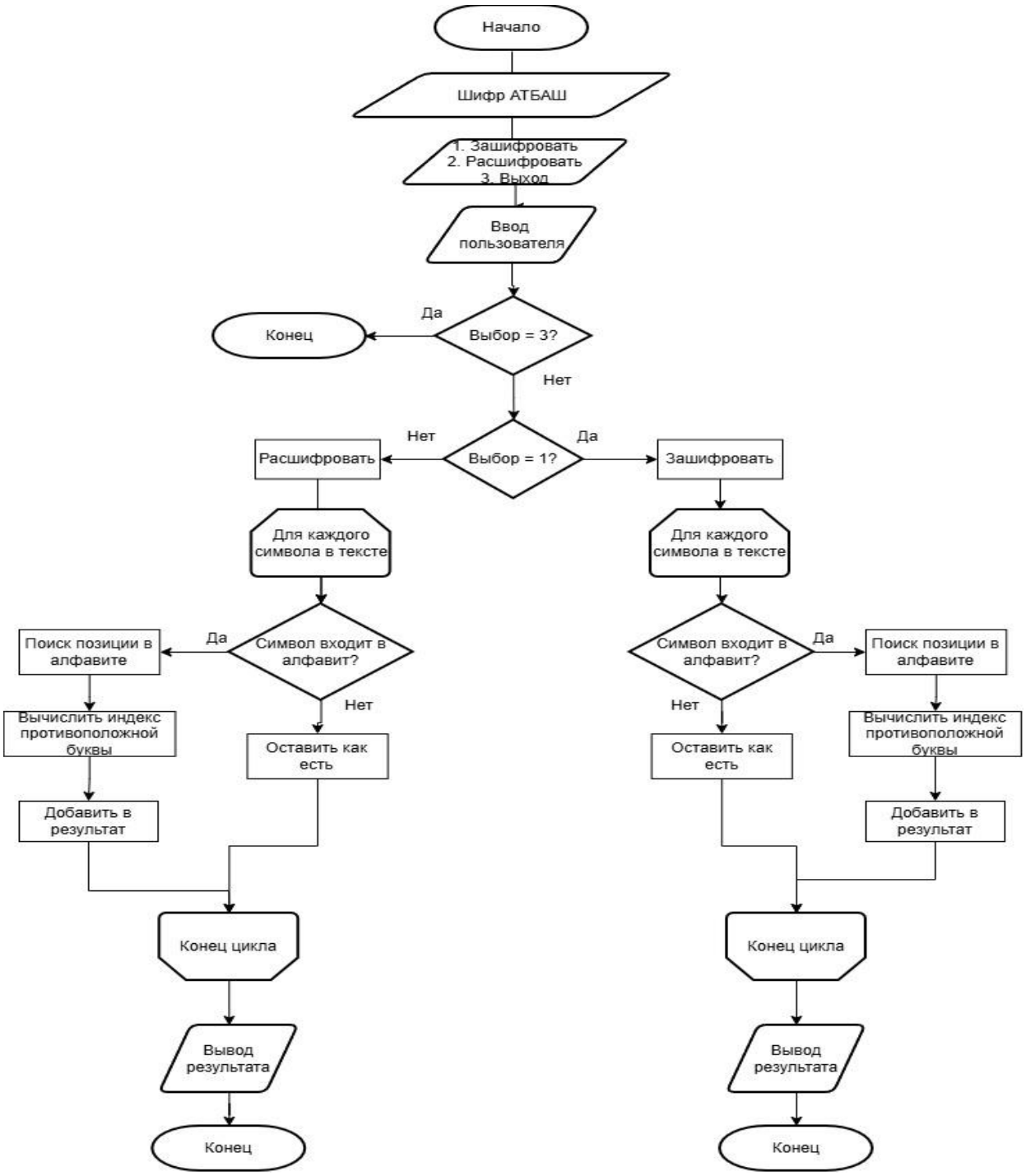

Начало
Шифр АТБАШ
1. Зашифровать
2. Расшифровать
3. Выход
Ввод пользователя
Выбор = 3?
Да
Конец
Нет
Выбор = 1?
Нет
Да
Расшифровать
Зашифровать
Для каждого символа в тексте
Для каждого символа в тексте
Символ входит в алфавит?
Да
Поиск позиции в алфавите
Нет
Вычислить индекс противоположной буквы
Оставить как есть
Добавить в результат
Символ входит в алфавит?
Да
Поиск позиции в алфавите
Нет
Оставить как есть
Вычислить индекс противоположной буквы
Добавить в результат
Конец цикла
Конец цикла
Вывод результата
Вывод результата
Конец
Конец

Рисунок 4. Блок-схема АТБАШ

Код программы:

```
"""
ШИФР АТБАШ
Русский алфавит (33 буквы с Ъ)
Первая буква алфавита заменяется последней, вторая — предпоследней и т.д.
"""

ALPHABET = 'АБВГДЕЁЖЗИЙКЛМНОПРСТУФХЦЧШЩЪЫЬЭЮЯ'
N = len(ALPHABET)

char_to_num = {char: i for i, char in enumerate(ALPHABET)}
num_to_char = {i: char for i, char in enumerate(ALPHABET)}

def encrypt(text):
    text = text.upper()
    result = []

    for char in text:
        if char == ' ':
            result.append(' ')
        elif char in char_to_num:
            m = char_to_num[char]
            L = (N - 1 - m) % N
            result.append(num_to_char[L])
        else:
            result.append(char)

    return ''.join(result)

def decrypt(ciphertext):
    return encrypt(ciphertext)  # Атбаш симметричен

while True:
    print("\n" + "=" * 50)
    print("ШИФР АТБАШ")
    print("=" * 50)
    print("1. Зашифровать")
    print("2. Расшифровать")
    print("3. Выход")

    choice = input("\nВыберите действие: ")

    if choice == "1":
        text = input("Введите текст для шифрования: ")
        encrypted = encrypt(text)

        print("\nРезультат:")
        print(encrypted)

    elif choice == "2":
        text = input("Введите текст для расшифрования: ")
        decrypted = decrypt(text)

        print("\nРезультат:")
        print(decrypted)

    elif choice == "3":
        print("Выход из программы...")
        break

    else:
        print("Неверный выбор!")
```

Результат работы программы:

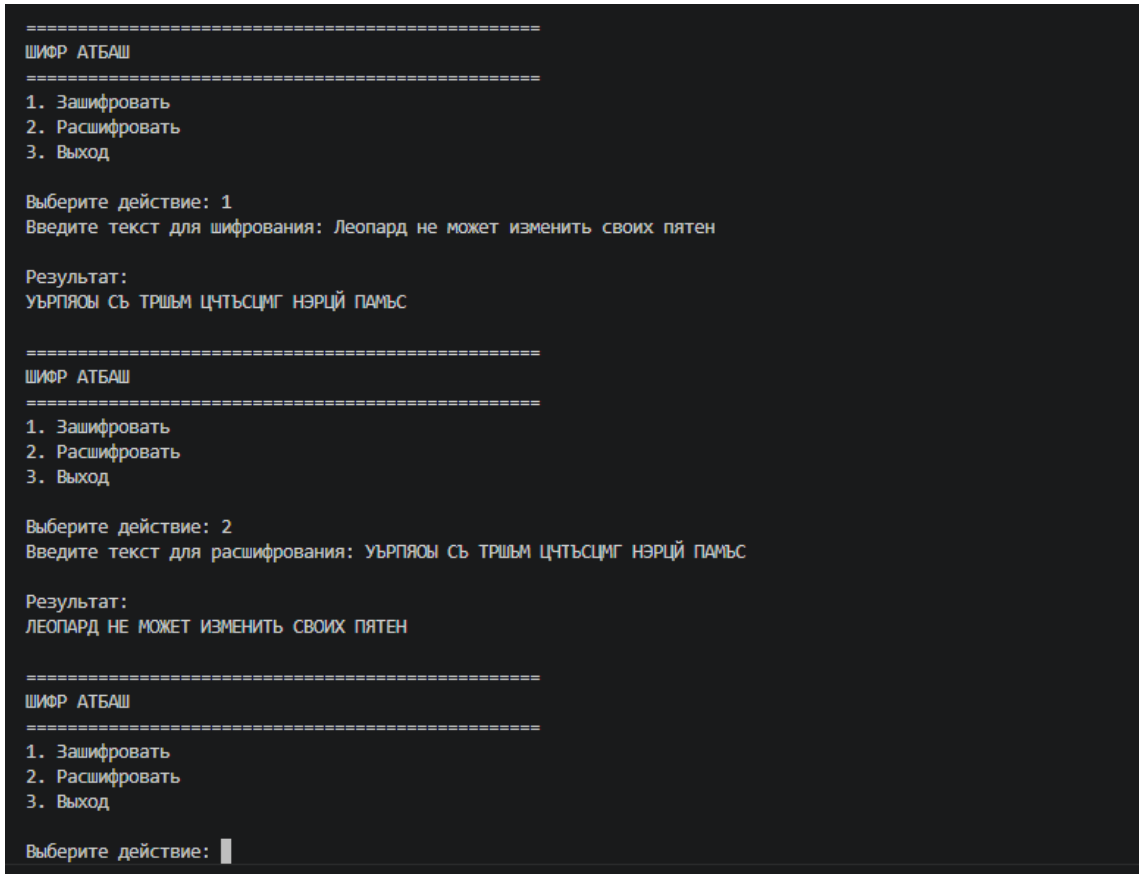


Рисунок 5. Результат выполнения шифра АТБАШ

Тест на 1000 символов:

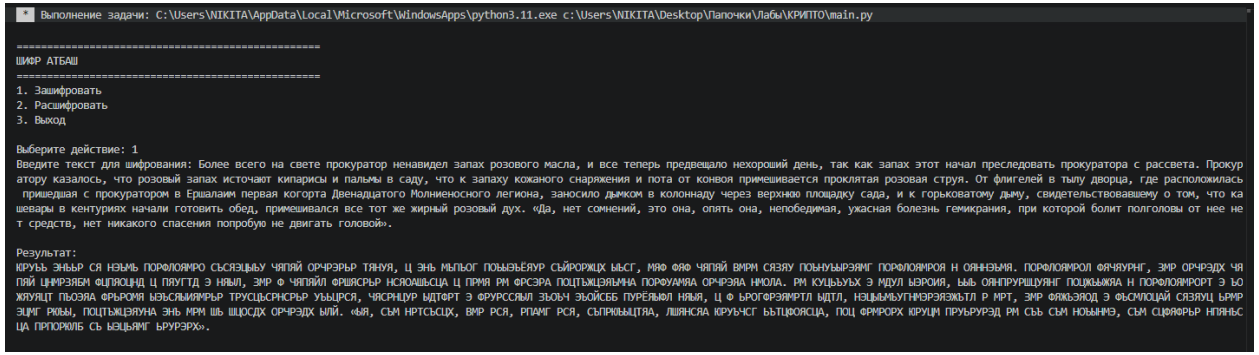


Рисунок 7. Шифрование АТБАШ на 1000 символов

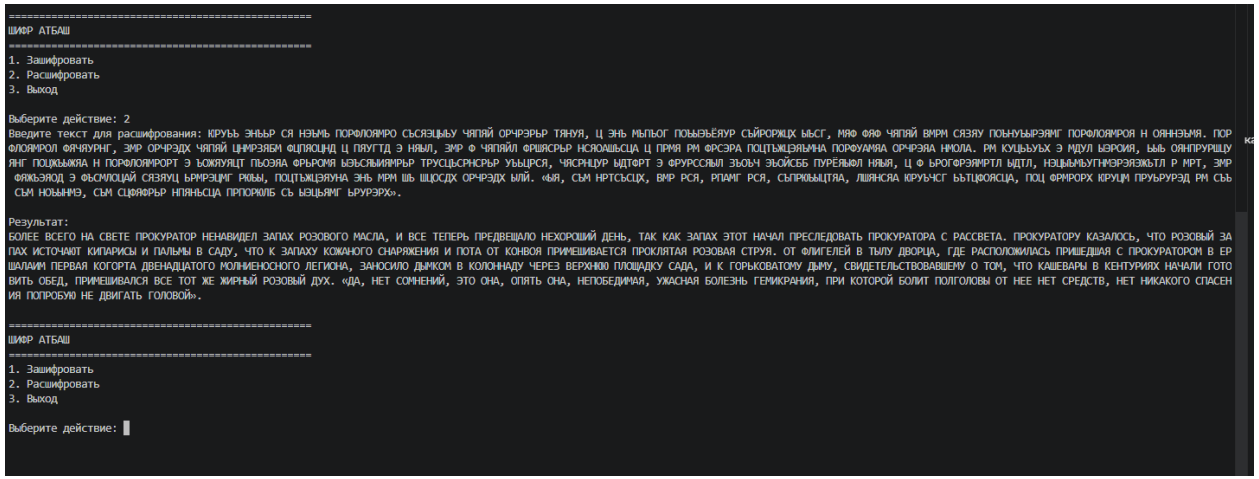


Рисунок 8. Расшифровывание АТБАШ на 1000 символов

Блок B: Шифры многозначной замены

5. Шифр Белазо

Шифр Белазо — это шифр подстановки. Он является усовершенствованием шифра Цезаря и предшественником шифра Виженера. В отличие от шифра Цезаря, где сдвиг постоянен для всего сообщения, шифр Белазо использует ключевое слово, которое определяет переменный сдвиг для каждого символа.

Обозначения:

P - открытый текст

C - зашифрованный текст

K - ключ

m - мощность алфавита

i - позиция символа в тексте (начало 0)

$C_i = (P_i + K\ \{i \bmod |K|\}) \bmod m$ - формула для шифра Белазо

Блок-схема:

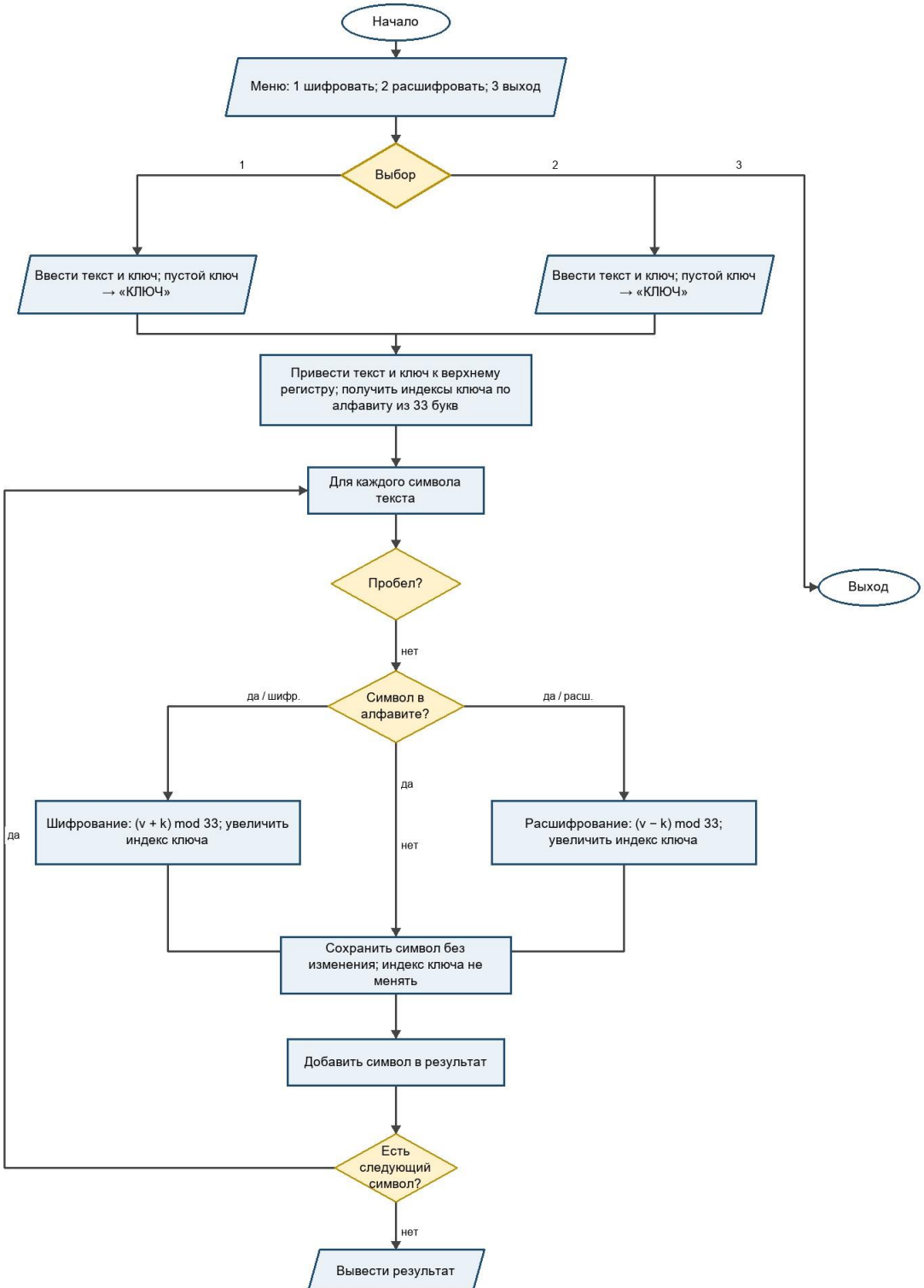

Начало
Меню: 1 шифровать; 2 расшифровать; 3 выход
Выбор
1
2
3
Ввести текст и ключ; пустой ключ → «КЛЮЧ»
Ввести текст и ключ; пустой ключ → «КЛЮЧ»
Привести текст и ключ к верхнему регистру; получить индексы ключа по алфавиту из 33 букв
Для каждого символа текста
Пробел?
Выход
нет
Символ в алфавите?
да / шифр.
да / расш.
да
нет
Шифрование: (v + k) mod 33; увеличить индекс ключа
Расшифрование: (v − k) mod 33; увеличить индекс ключа
да
Сохранить символ без изменения; индекс ключа не менять
Добавить символ в результат
Есть следующий символ?
нет
Вывести результат

Рисунок 9. Блок-схема шифра Белазо.

Код программы:

```
# -*- coding: utf-8 -*-
"""
ПРОСТОЙ ШИФР БЕЛАЗО
"""
ALPHABET = 'АБВГДЕЁЖЗИЙКЛМНОПРСТУФХЦЧШЩЪЫЬЭЮЯ'
N = 33
char_to_num = {char: i for i, char in enumerate(ALPHABET)}
num_to_char = {i: char for i, char in enumerate(ALPHABET)}
KEYWORD = "КЛЮЧ"
def get_key_nums(keyword):
    return [char_to_num[char] for char in keyword.upper() if char in char_to_num]
def encrypt(text, keyword=KEYWORD):
    text = text.upper()
    key_nums = get_key_nums(keyword)
    key_length = len(key_nums)
    result = []
    key_idx = 0
    for char in text:
        if char == ' ':
            result.append(' ')
        elif char in char_to_num:
            vi = char_to_num[char]
            ki = key_nums[key_idx % key_length]
            wi = (vi + ki) % N
            result.append(num_to_char[wi])
            key_idx += 1
        else:
            result.append(char)
    return ''.join(result)
def decrypt(ciphertext, keyword=KEYWORD):
    ciphertext = ciphertext.upper()
    key_nums = get_key_nums(keyword)
    key_length = len(key_nums)
    result = []
    key_idx = 0
    for char in ciphertext:
        if char == ' ':
            result.append(' ')
        elif char in char_to_num:
            wi = char_to_num[char]
            ki = key_nums[key_idx % key_length]
            vi = (wi - ki) % N
            result.append(num_to_char[vi])
            key_idx += 1
        else:
            result.append(char)
    return ''.join(result)
while True:
    print("\n" + "=" * 50)
```

```
print("ПРОСТОЙ ШИФР БЕЛАЗО")
print("=" * 50)
print("1. Зашифровать")
print("2. Расшифровать")
print("3. Выход")
choice = input("\nВыберите действие: ")
if choice == "1":
    text = input("Введите текст для шифрования: ")
    key = input("Введите ключ (Enter = КЛЮЧ): ").strip()
    if not key:
        key = KEYWORD
    encrypted = encrypt(text, key)
    print("\nРезультат шифрования:")
    print(encrypted)
elif choice == "2":
    text = input("Введите текст для расшифрования: ")
    key = input("Введите ключ (Enter = КЛЮЧ): ").strip()
    if not key:
        key = KEYWORD
    decrypted = decrypt(text, key)
    print("\nРезультат расшифрования:")
    print(decrypted)
elif choice == "3":
    print("Выход из программы...")
    break
else:
    print("Неверный выбор!")
```

Результат работы программы:



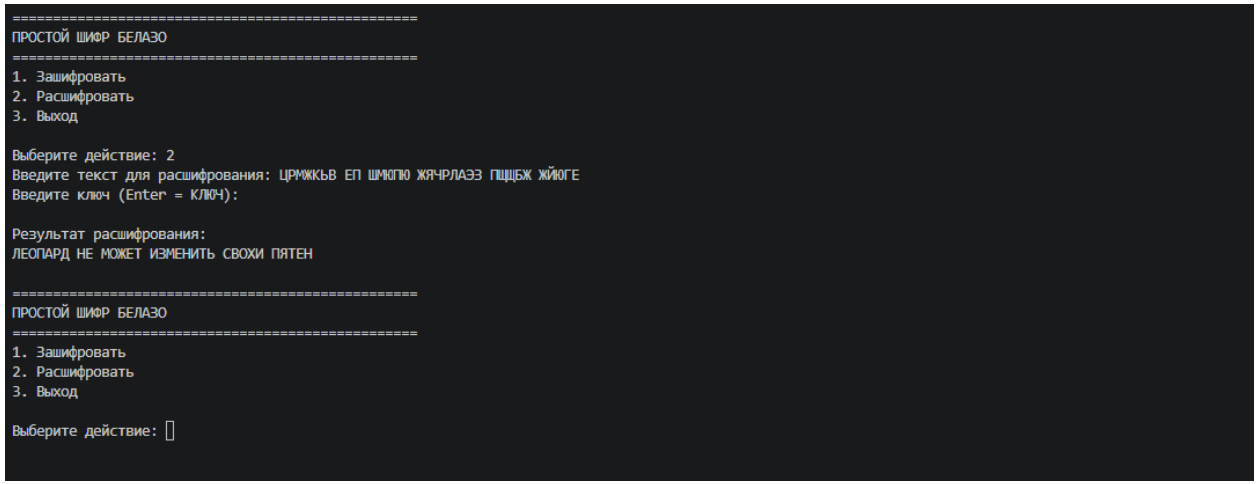


Рисунок 10. Результат работы шифра Белазо.

Тест на 1000 знаков:

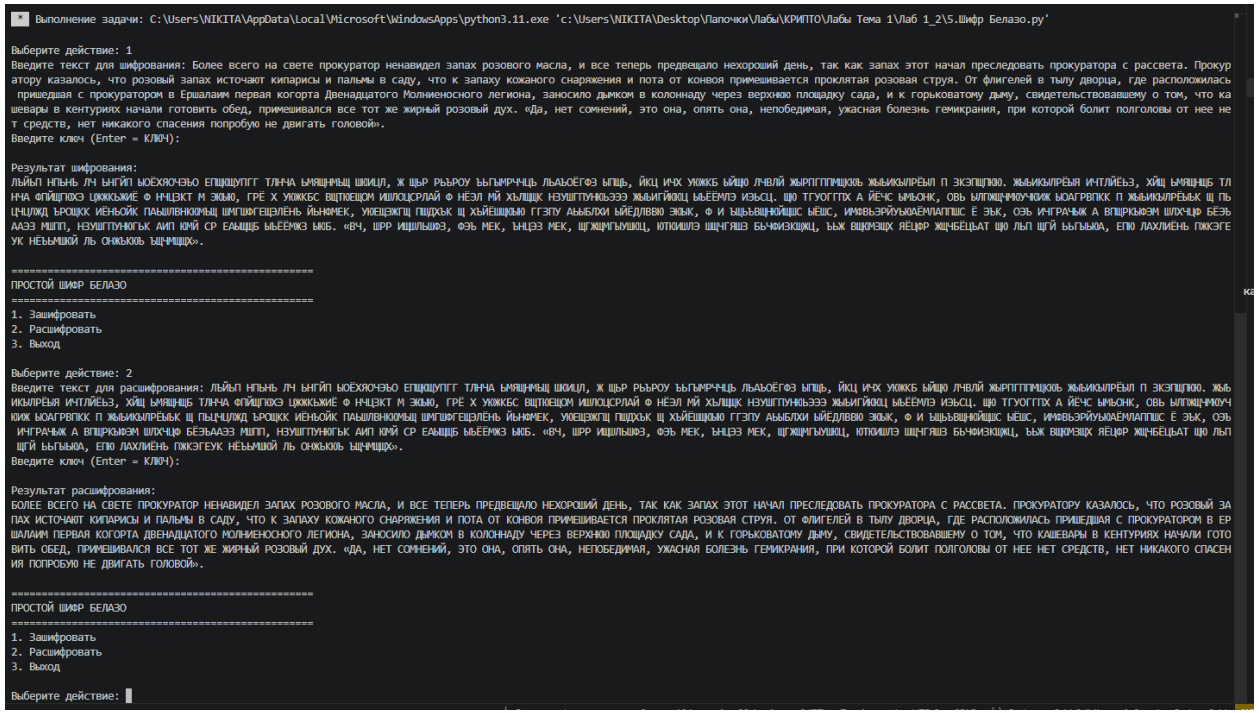


Рисунок 12. Тест на 1000 символов шифра Белазо.

Блок С: Шифры блочной замены

8. Матричный шифр

Матричный шифр (также известен как шифр Хилла) — это полиграммный шифр подстановки. Он является одним из первых шифров, которые могли работать с блоками символов (биграммами, триграммами).

Ключевая особенность: шифр Хилла использует матричное умножение для блоков букв. Пробелы и знаки препинания не входят в алфавит и остаются на исходных позициях; размер алфавита и вычисления по модулю 32 не меняются.

Обозначения:

$C = (K \times P) \bmod m$

P — вектор-столбец открытого текста (размер n×1)

K — ключевая матрица (размер n×n)

C — вектор-столбец зашифрованного текста (размер n×1)

m — мощность алфавита (32 для русского языка)

mod m — операция взятия остатка по модулю m

Блок-схема:

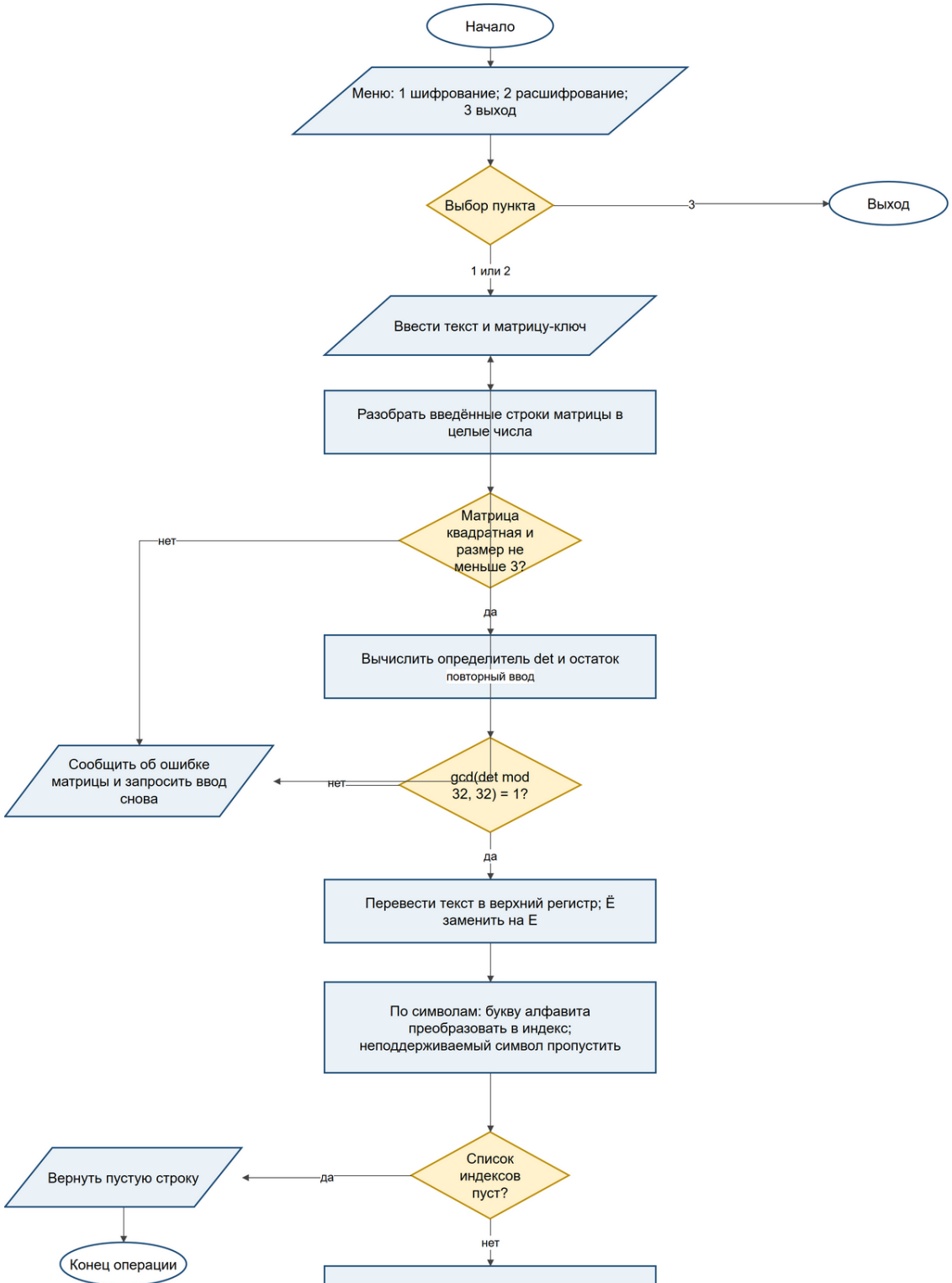

Начало
Меню: 1 шифрование; 2 расшифрование; 3 выход
Выбор пункта
3
Выход
1 или 2
Ввести текст и матрицу-ключ
Разобрать введённые строки матрицы в целые числа
Матрица квадратная и размер не меньше 3?
нет
да
Вычислить определитель det и остаток
повторный ввод
Сообщить об ошибке матрицы и запросить ввод снова
gcd(det mod 32, 32) = 1?
нет
да
Перевести текст в верхний регистр; Ё заменить на Е
По символам: букву алфавита преобразовать в индекс; неподдерживаемый символ пропустить
Список индексов пуст?
Вернуть пустую строку
да
нет
Конец операции

Рисунок 13. Блок-схема матричного шифра.

Код программы:

```
# МАТРИЧНЫЙ ШИФР (Шифр Хилла)
# Вариант 8: Матрица-ключ не меньше 3×3

import numpy as np
import math

# Русский алфавит (32 буквы, без Ё)
RUSSIAN_ALPHABET = 'АБВГДЕЖЗИЙКЛМНОПРСТУФХЦЧШЩЪЫЬЭЮЯ'
ALPHABET_SIZE = 32


class MatrixCipher:
    """Матричный шифр (шифр Хилла)"""

    def __init__(self, matrix):
        self.matrix = np.array(matrix, dtype=int)
        self.size = len(matrix)
        self.mod = ALPHABET_SIZE
        self.inverse_matrix = self._mod_inverse_matrix()

    def _mod_inverse(self, a, m):
        """Находит обратное число по модулю"""
        a = a % m
        for x in range(1, m):
            if (a * x) % m == 1:
                return x
        return None

    def _mod_inverse_matrix(self):
        """Находит обратную матрицу по модулю"""
        det = int(round(np.linalg.det(self.matrix)))
        det_mod = det % self.mod

        # Проверяем, что определитель обратим по модулю
        det_inv = self._mod_inverse(det_mod, self.mod)
        if det_inv is None:
            raise ValueError(f"Матрица необратима по модулю {self.mod}")

        # Для матрицы 3x3 вычисляем присоединенную матрицу
        if self.size == 3:
            adjugate = self._adjugate_3x3(self.matrix)
        else:
            adjugate = np.round(det * np.linalg.inv(self.matrix)).astype(int)

        # Умножаем на обратный определитель по модулю
        inverse = (det_inv * adjugate) % self.mod
        return inverse.astype(int)

    def _adjugate_3x3(self, matrix):
        """Вычисляет присоединенную матрицу для 3x3"""
        m = matrix
        adj = np.zeros((3, 3), dtype=int)
```

```
        # Формулы для присоединенной матрицы 3x3
        adj[0][0] = (m[1][1] * m[2][2] - m[1][2] * m[2][1])
        adj[1][0] = -(m[1][0] * m[2][2] - m[1][2] * m[2][0])
        adj[2][0] = (m[1][0] * m[2][1] - m[1][1] * m[2][0])

        adj[0][1] = -(m[0][1] * m[2][2] - m[0][2] * m[2][1])
        adj[1][1] = (m[0][0] * m[2][2] - m[0][2] * m[2][0])
        adj[2][1] = -(m[0][0] * m[2][1] - m[0][1] * m[2][0])

        adj[0][2] = (m[0][1] * m[1][2] - m[0][2] * m[1][1])
        adj[1][2] = -(m[0][0] * m[1][2] - m[0][2] * m[1][0])
        adj[2][2] = (m[0][0] * m[1][1] - m[0][1] * m[1][0])

        return adj

    def _text_to_numbers(self, text):
        """Преобразует текст в числа (А=0, Б=1, ...)"""
        text = text.upper()
        numbers = []
        for char in text:
            if char in RUSSIAN_ALPHABET:
                numbers.append(RUSSIAN_ALPHABET.index(char))
            elif char == 'Ё':
                numbers.append(RUSSIAN_ALPHABET.index('Е'))
            elif char.isalpha():
                # Для английских букв
                numbers.append(ord(char) % self.mod)
        return numbers

    def _numbers_to_text(self, numbers):
        """Преобразует числа обратно в текст"""
        text = ''
        for num in numbers:
            text += RUSSIAN_ALPHABET[num % self.mod]
        return text

    @staticmethod
    def _is_encrypted_character(char):
        """Проверяет, преобразуется ли символ текущим алфавитным правилом."""
        return char.upper() in RUSSIAN_ALPHABET or char.upper() == 'Ё' or char.isalpha()

    def _restore_format(self, template, transformed):
        """Возвращает преобразованные буквы на исходные места, сохраняя разделители."""
        result = []
        letter_index = 0
        for char in template:
            if self._is_encrypted_character(char):
                if letter_index < len(transformed):
                    result.append(transformed[letter_index])
                    letter_index += 1
            else:
                result.append(char)
        # Остаток — это символы дополнения блока.
        result.extend(transformed[letter_index:])
        return ''.join(result)

    def encrypt(self, text):
        """Шифрует буквы, оставляя пробелы и знаки на прежних местах."""
        numbers = self._text_to_numbers(text)
```

```
        if not numbers:
            return text

        # Дополняем буквенную последовательность до длины, кратной размеру матрицы.
        while len(numbers) % self.size != 0:
            numbers.append(RUSSIAN_ALPHABET.index('ь'))

        encrypted = []
        for i in range(0, len(numbers), self.size):
            block = numbers[i:i + self.size]
            encrypted_block = np.dot(self.matrix, block) % self.mod
            encrypted.extend(encrypted_block)

        encrypted_letters = self._numbers_to_text(encrypted)
        return self._restore_format(text, encrypted_letters)

    def decrypt(self, text, remove_padding=True):
        """Расшифровывает буквы, сохраняя пробелы и знаки на прежних местах."""
        numbers = self._text_to_numbers(text)
        if not numbers:
            return text

        # Длина буквенной последовательности должна делиться на размер матрицы.
        if len(numbers) % self.size != 0:
            while len(numbers) % self.size != 0:
                numbers.append(RUSSIAN_ALPHABET.index('ь'))

        decrypted = []
        for i in range(0, len(numbers), self.size):
            block = numbers[i:i + self.size]
            decrypted_block = np.dot(self.inverse_matrix, block) % self.mod
            decrypted.extend(decrypted_block)

        decrypted_letters = self._numbers_to_text(decrypted)
        if remove_padding:
            decrypted_letters = decrypted_letters.rstrip('ь')
        return self._restore_format(text, decrypted_letters)


def validate_matrix(matrix):
    """Проверяет, что матрица подходит для шифрования"""
    try:
        matrix = np.array(matrix)
        det = int(round(np.linalg.det(matrix)))

        # Определитель не должен быть равен 0
        if det == 0:
            return False, f"Определитель матрицы = 0 (матрица необратима)"

        # Определитель должен быть взаимно прост с размером алфавита
        det_mod = det % ALPHABET_SIZE
        if math.gcd(det_mod, ALPHABET_SIZE) != 1:
            return False, f"Определитель матрицы ({det}) не взаимно прост с {ALPHABET_SIZE}"

        return True, f"Матрица валидна (определитель = {det})"
    except Exception as e:
        return False, f"Ошибка при проверке матрицы: {e}"


def print_matrix(matrix):
```

```
    """Красиво выводит матрицу"""
    print("\nМатрица-ключ:")
    for row in matrix:
        print("  " + " ".join(f"{x:3}" for x in row))


def main():

    while True:

        print("\n" + "=" * 50)
        print("МАТРИЧНЫЙ ШИФР (Шифр Хилла)")
        print("=" * 50)
        print("1. Зашифровать")
        print("2. Расшифровать")
        print("3. Выход")

        choice = input("\nВыберите действие: ").strip()

        if choice == "1":

            text = input("Введите текст для шифрования: ")

            print("\nВведите матрицу-ключ")
            print("Пример: 3 2 1, 1 1 1, 1 2 3")

            try:

                matrix_str = input("Матрица: ").strip()

                rows = matrix_str.split(',')
                matrix = []

                for row in rows:
                    matrix.append(
                        list(map(int, row.strip().split()))
                    )

                is_valid, message = validate_matrix(matrix)

                if not is_valid:
                    print(f"Ошибка: {message}")
                    continue

                cipher = MatrixCipher(matrix)

                encrypted = cipher.encrypt(text)

                print("\nРезультат шифрования:")
                print(encrypted)

            except Exception as e:
                print(f"Ошибка: {e}")

        elif choice == "2":

            text = input("Введите текст для расшифрования: ")

            print("\nВведите ту же матрицу-ключ")
```

```
            try:

                matrix_str = input("Матрица: ").strip()

                rows = matrix_str.split(',')
                matrix = []

                for row in rows:
                    matrix.append(
                        list(map(int, row.strip().split()))
                    )

                is_valid, message = validate_matrix(matrix)

                if not is_valid:
                    print(f"Ошибка: {message}")
                    continue

                cipher = MatrixCipher(matrix)

                decrypted = cipher.decrypt(text)

                print("\nРезультат расшифрования:")
                print(decrypted)

            except Exception as e:
                print(f"Ошибка: {e}")

        elif choice == "3":

            print("Выход из программы...")
            break

        else:

            print("Неверный выбор!")


if __name__ == "__main__":
    main()
```

Результат работы программы:

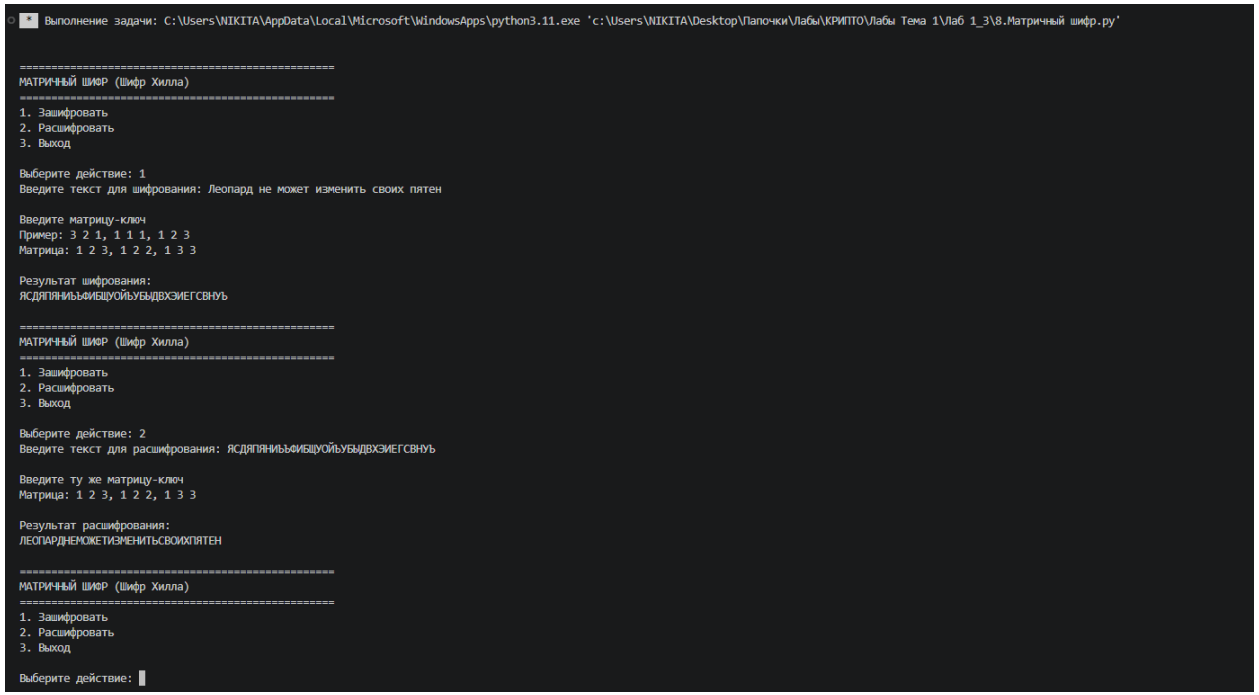


Рисунок 14. Результат работы матричного шифра.

Тест на 1000 знаков:

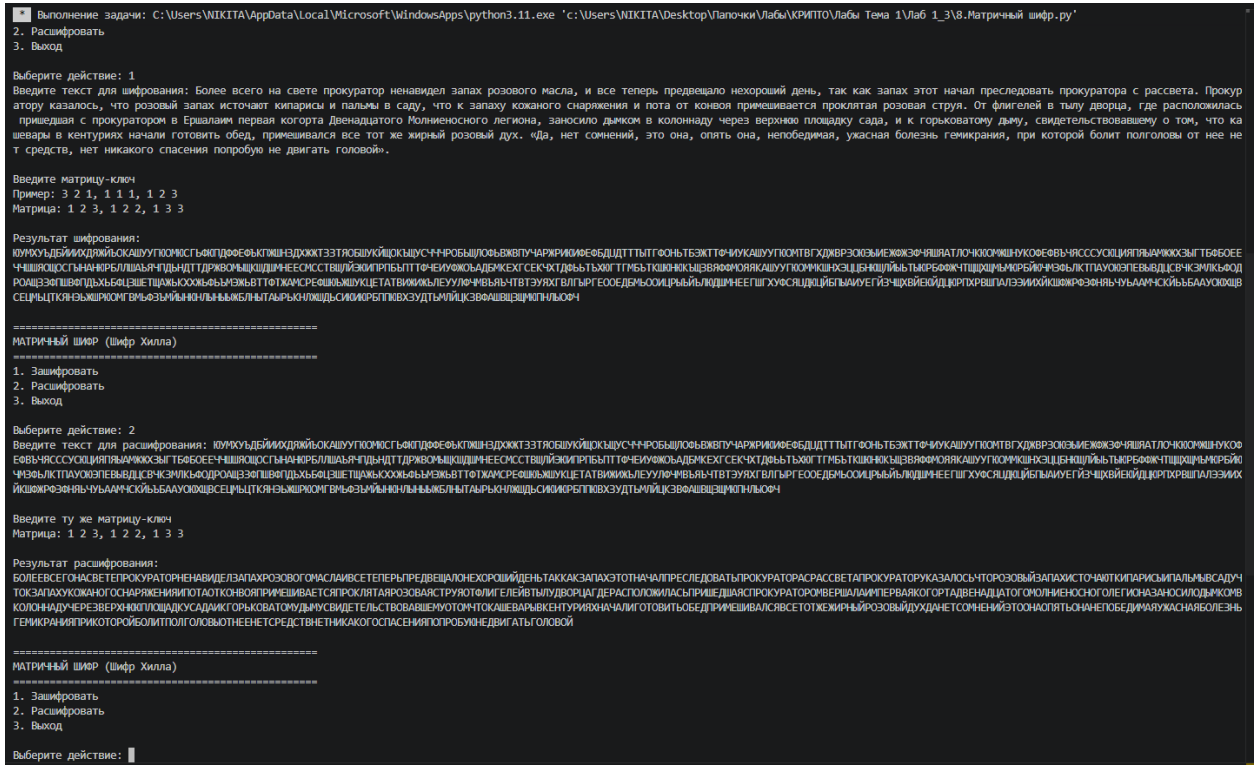


Рисунок 16. Тест на 1000 символов матричного шифра.

Блок D: Шифры перестановки

10. Вертикальная перестановка

Шифр вертикальной перестановки — это метод шифрования, при котором символы исходного сообщения записываются в таблицу по строкам, а затем считываются по столбцам в порядке, определяемом ключевым словом. Это классический транспозиционный шифр, который не заменяет символы, а только меняет их порядок.

Блок-схема:

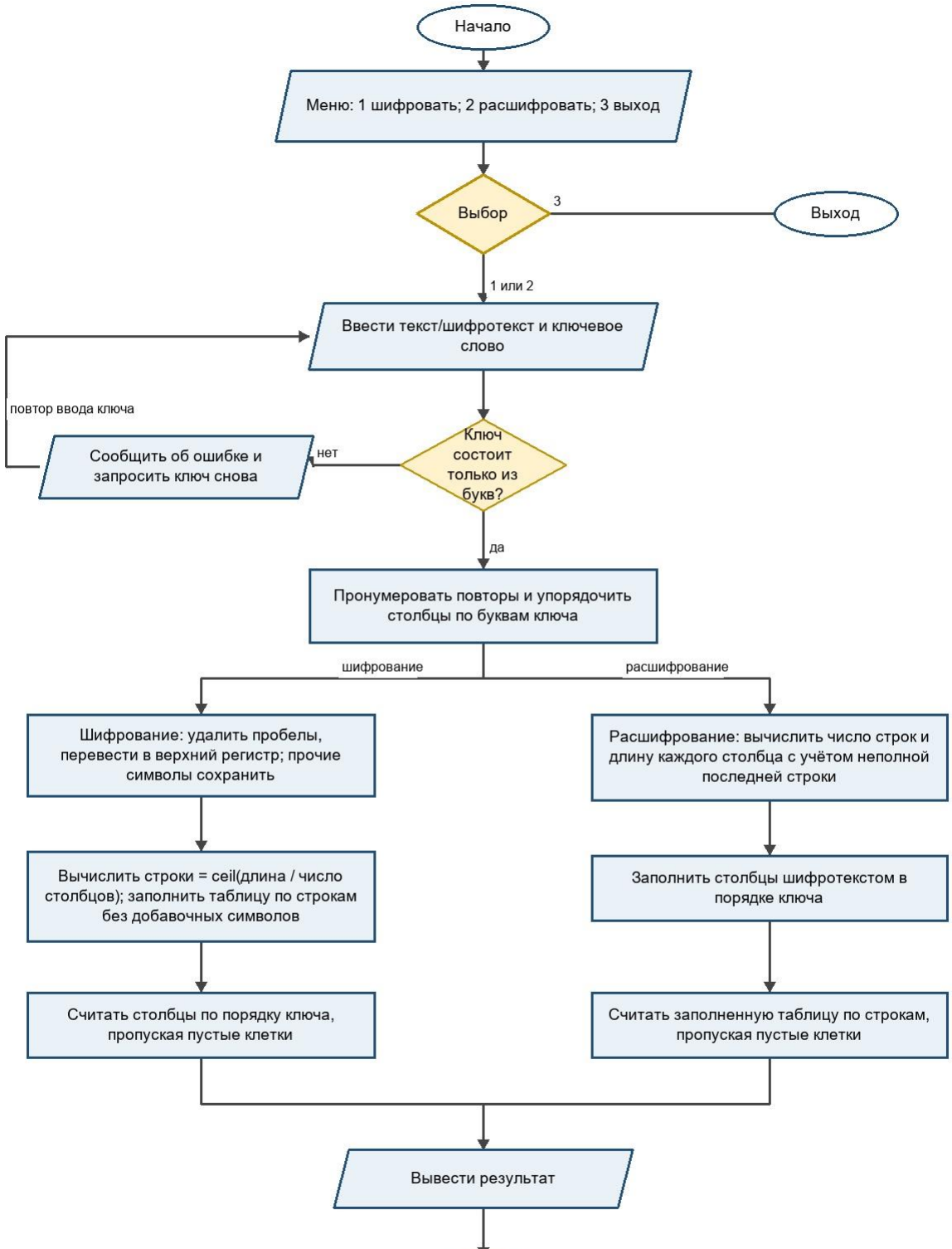

Начало
Меню: 1 шифровать; 2 расшифровать; 3 выход
Выбор
3
Выход
1 или 2
Ввести текст/шифротекст и ключевое слово
повтор ввода ключа
Сообщить об ошибке и запросить ключ снова
нет
Ключ состоит только из букв?
да
Пронумеровать повторы и упорядочить столбцы по буквам ключа
шифрование
расшифрование
Шифрование: удалить пробелы, перевести в верхний регистр; прочие символы сохранить
Вычислить строки = ceil(длина / число столбцов); заполнить таблицу по строкам без добавочных символов
Считать столбцы по порядку ключа, пропуская пустые клетки
Расшифрование: вычислить число строк и длину каждого столбца с учётом неполной последней строки
Заполнить столбцы шифротекстом в порядке ключа
Считать заполненную таблицу по строкам, пропуская пустые клетки
Вывести результат

Рисунок 17. Блок-схема вертикальной перестановки.

Код программы:

```
# Шифр вертикальной перестановки (Вариант 10)
# Ключ - слово "МАГМА" (буквы могут повторяться)
# Пустые клетки НЕ ЗАПОЛНЯЮТСЯ

def proverka_klucha(key):
    """Проверка ключа-слова"""
    if not key.isalpha():
        print("❌ Ошибка: Ключ должен состоять только из букв!")
        return False

    print(f"✅ Ключ '{key}' принят!")
    return True



def poluchit_poryadok_stolbcov(key):
    """
    Получает порядок столбцов на основе алфавитного порядка букв ключа
    Для повторяющихся букв использует порядковый номер (1,2,3...)
```

```
"""
print(f"\n🔍 Анализ ключа '{key}':")
print("-" * 50)
print("Исходная позиция | Буква | Номер повтора | Новый порядок")
print("-" * 50)

# Считаем сколько раз встретилась каждая буква
letter_count = {}
letters_with_pos = []

for i, letter in enumerate(key):
    # Увеличиваем счетчик для буквы
    if letter in letter_count:
        letter_count[letter] += 1
    else:
        letter_count[letter] = 1

    # Запоминаем букву, позицию и номер повтора
    letters_with_pos.append((letter, i, letter_count[letter]))

# Сортируем: сначала по букве, потом по номеру повтора
# Для одинаковых букв меньший номер повтора идет первым
```

```
sorted_letters = sorted(letters_with_pos, key=lambda x: (x[0], x[2]))

# Создаем словарь: исходная позиция -> новый порядок
order = {}
for new_pos, (letter, old_pos, repeat_num) in enumerate(sorted_letters):
    order[old_pos] = new_pos + 1  # +1 для человеко-читаемого формата

# Создаем ключ-цифру
numeric_key = ''.join(str(order[i]) for i in range(len(key)))

# Выводим таблицу анализа
for i in range(len(key)):
    letter = key[i]
    # Находим номер повтора для этой позиции
    repeat = 1
    for j in range(i):
        if key[j] == letter:
            repeat += 1
    print(f"       {i+1}         |   {letter}   |       {repeat}       |       {order[i]}")

print("-" * 50)
print(f"👉 Ключ для шифрования: {numeric_key}")
```

```
    return numeric_key, sorted_letters


def shifrovat(text, word_key):
    """Шифрование вертикальной перестановкой (без заполнения пустых клеток)"""
    print("\n" + "="*70)
    print("🔐 НАЧИНАЕМ ШИФРОВАНИЕ")
    print("="*70)

    # Получаем числовой ключ из слова
    numeric_key, sorted_letters = poluchit_poryadok_stolbcov(word_key)

    # Шаг 1: Подготавливаем текст
    original_text = text
    text = text.replace(" ", "").upper()
    print(f"\n📝 Исходный текст: '{original_text}'")
    print(f"1. Текст без пробелов (upper): {text}")
    print(f"   Длина текста: {len(text)} символов")

    # Шаг 2: Определяем размер таблицы
    cols = len(word_key)  # количество столбцов = длина ключа
```

```
rows = (len(text) + cols - 1) // cols  # количество строк

print(f"2. Таблица будет размером {rows} строк × {cols} столбцов")
print(f"   ⚠ Пустые клетки останутся пустыми (не заполняются)")

# Шаг 3: Записываем текст в таблицу по строкам
print("\n3. Заполняем таблицу по строкам (пустые клетки - '.'):")
print("-" * 70)

# Печатаем заголовок с ключом
print("    ", end="")
for i, letter in enumerate(word_key):
    print(f"  {letter}({i+1}) ", end="")
print("\n   " + "-" * (cols * 7))

matrix = []
index = 0
total_chars = len(text)

for i in range(rows):
    row = []
    print(f"   |", end="")
```

```
    for j in range(cols):
        if index < total_chars:
            row.append(text[index])
            print(f"  {text[index]}  |", end="")
            index += 1
        else:
            row.append(None)  # Пустая клетка (None, чтобы не путать с точкой из текста)
            print(f"  .  |", end="")
    matrix.append(row)
    print(f"  строка {i+1}")
    print("    " + "-" * (cols * 7))

# Шаг 4: Читаем по столбцам в порядке, определенном ключом
print("\n4. Читаем столбцы в порядке сортировки букв ключа:")
print("    (пропуская пустые клетки)")

result = ""
print("\n   Порядок чтения столбцов:")
for new_pos, (letter, col, repeat_num) in enumerate(sorted_letters):
    column_text = ""
    print(f"    {new_pos+1}. Столбец {col+1} (буква '{letter}' №{repeat_num}): ", end="")
    for row in range(rows):
```

```
            if matrix[row][col] is not None:  # Пропускаем пустые клетки
                result += matrix[row][col]
                column_text += matrix[row][col]
        print(column_text)

    print("\n" + "="*70)
    print(f"🔐 РЕЗУЛЬТАТ ШИФРОВАНИЯ: {result}")
    print("="*70)

    return result, numeric_key, sorted_letters, rows, cols


def rasshifrovat(cipher, word_key, numeric_key, sorted_letters, rows, cols):
    """Расшифрование вертикальной перестановки (с пустыми клетками)"""
    print("\n" + "="*70)
    print("🔓 НАЧИНАЕМ РАСШИФРОВАНИЕ")
    print("="*70)

    print(f"1. Таблица {rows}×{cols} (с пустыми клетками)")
    print(f"2. Шифротекст: {cipher}")
    print(f"3. Ключ-слово: {word_key}")
    print(f"4. Числовой ключ: {numeric_key}")
```

```
# Создаем пустую таблицу
matrix = [[None for _ in range(cols)] for _ in range(rows)]

# Сначала нужно определить, в каких столбцах есть пустые клетки
# Вычисляем сколько символов в каждом столбце
total_chars = len(cipher)
chars_per_column = [rows] * cols  # по умолчанию во всех столбцах rows символов

# В последних столбцах может быть меньше символов
empty_in_last_row = cols * rows - total_chars
if empty_in_last_row > 0:
    # Пустые клетки в последней строке, начиная с последних столбцов
    for i in range(empty_in_last_row):
        chars_per_column[cols - 1 - i] -= 1

print(f"\n   Количество символов в каждом столбце: {chars_per_column}")

# Заполняем по столбцам в порядке, определенном ключом
index = 0
print("\n5. Заполняем таблицу по столбцам в порядке сортировки букв:")
```

```
for new_pos, (letter, col, repeat_num) in enumerate(sorted_letters):
    print(f"\n    {new_pos+1}. Столбец {col+1} (буква '{letter}' №{repeat_num}): ", end="")
    # Сколько символов нужно записать в этот столбец
    chars_in_this_col = chars_per_column[col]
    for row in range(chars_in_this_col):
        matrix[row][col] = cipher[index]
        print(f"{cipher[index]}", end="")
        index += 1
    # Остальные клетки в столбце остаются '.'

# Показываем заполненную таблицу
print("\n\n6. Заполненная таблица:")
print("-" * 70)
print("    ", end="")
for i, letter in enumerate(word_key):
    print(f"  {letter}({i+1}) ", end="")
print("\n   " + "-" * (cols * 7))

for i in range(rows):
    print(f"   |", end="")
    for j in range(cols):
        print(f"  {matrix[i][j] if matrix[i][j] is not None else '.'}  |", end="")
```

```
        print(f"  строка {i+1}")
        print("    " + "-" * (cols * 7))

    # Читаем по строкам (пропуская пустые клетки)
    print("\n7. Читаем результат по строкам (пропуская пустые клетки):")
    result = ""
    for i in range(rows):
        row_text = ""
        for j in range(cols):
            if matrix[i][j] is not None:
                result += matrix[i][j]
                row_text += matrix[i][j]
        print(f"   Строка {i+1}: {row_text}")

    print(f"\n8. Результат: {result}")
    return result


# ============ ГЛАВНАЯ ПРОГРАММА ============

def main():
```

```
while True:

    print("\n" + "=" * 50)
    print("ШИФР ВЕРТИКАЛЬНОЙ ПЕРЕСТАНОВКИ")
    print("=" * 50)
    print("1. Зашифровать")
    print("2. Расшифровать")
    print("3. Выход")

    choice = input("\nВыберите действие: ").strip()

    if choice == "1":

        text = input("Введите текст для шифрования: ")

        while True:
            key = input("Введите ключевое слово: ").upper()

            if proverka_klucha(key):
                break

        cipher, numeric_key, sorted_letters, rows, cols = shifrovat(text, key)
```

```
    print("\nРезультат шифрования:")
    print(cipher)

elif choice == "2":

    cipher = input("Введите шифротекст: ")

    while True:
        key = input("Введите ключевое слово: ").upper()

        if proverka_klucha(key):
            break

    numeric_key, sorted_letters = poluchit_poryadok_stolbcov(key)

    cols = len(key)
    rows = (len(cipher) + cols - 1) // cols

    decrypted = rasshifrovat(
        cipher,
        key,
```

```
            numeric_key,
            sorted_letters,
            rows,
            cols
        )

        print("\nРезультат расшифрования:")
        print(decrypted)

    elif choice == "3":

        print("Выход из программы...")
        break

    else:

        print("Неверный выбор!")


if __name__ == "__main__":
    main()
```

Результат работы программы:

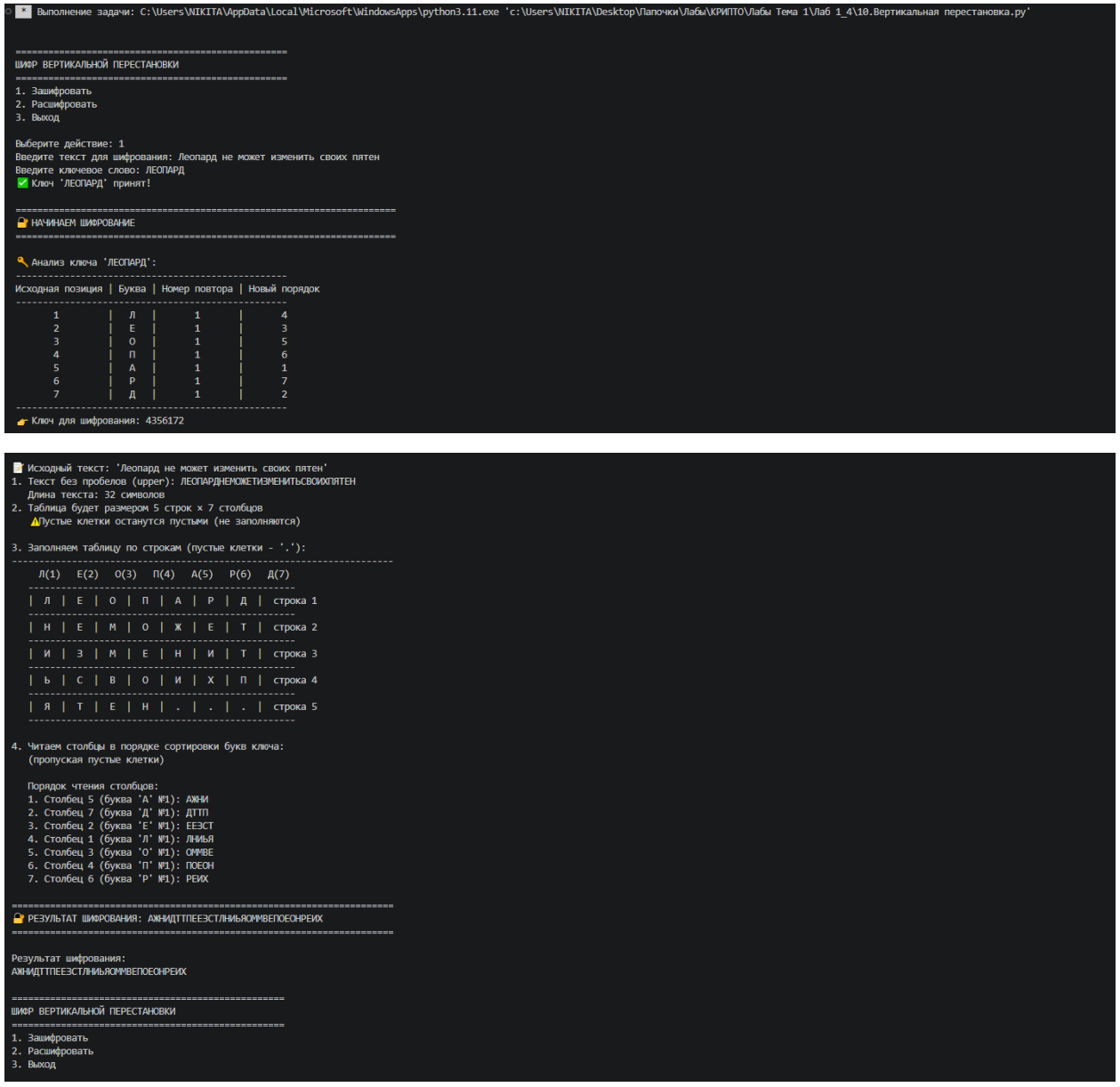
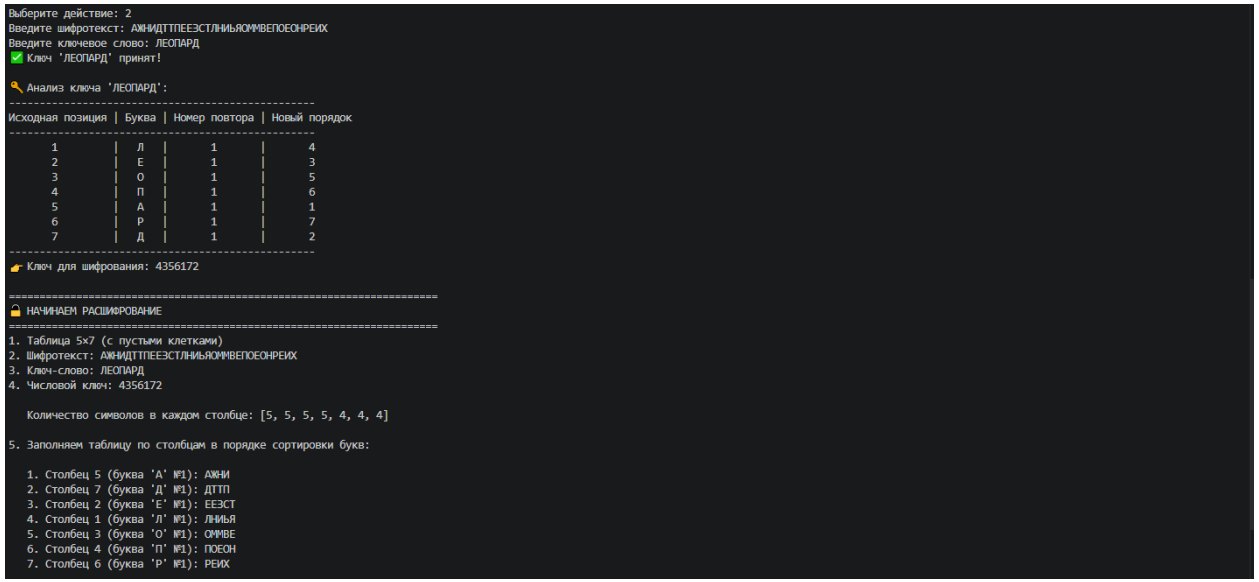


Рисунок 18. Работа шифра вертикальной перестановки.



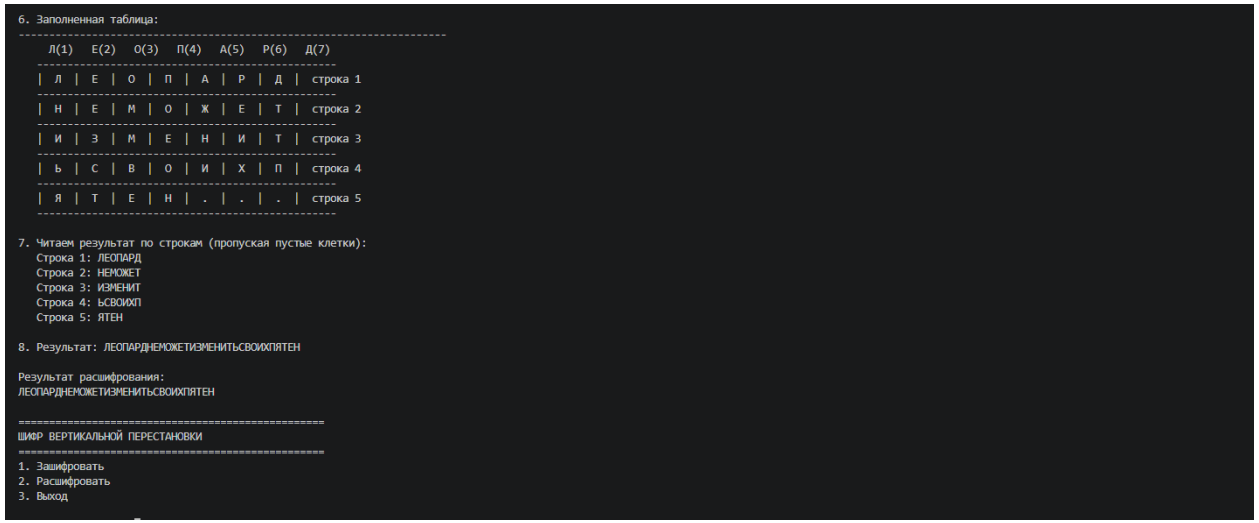


Рисунок 19. Работа шифра вертикальной перестановки (2).

Блок E: Шифры гаммирования

13. Одноразовый блокнот Шеннона

Открытый текст сообщения m записывают, как последовательность бит или символов m = m0m1…mn-1, а двоичную или символьную шифрующую последовательность k той же самой длины — как k = k0k1…kn-1. Шифртекст c = c0c1…cn-1 определяется соотношением ci = mi ki при 0≤i≤n-1, где обозначает операцию «исключающее ИЛИ» (ассемблерная операция XOR) по модулю два или сложение по любому другому модулю в случае символьной гаммы.

Для генерации ПСЧ могут использоваться конгруэнтные генераторы, например, линейный генератор: T(i+1) = (aT(i) + c) mod m, где a и c – константы, T(0) – исходная величина, выбранная в качестве порождающего числа. Период ограничен модулем и не может быть больше m. Максимальный период равен m.

Период генерируемых чисел зависит от величины a и c и является максимальным тогда и только тогда, когда c – нечетное число, a (mod 4) = 1. В случае символьной гаммы величина m определяется размером алфавита, а период гаммы также параметрами a и c:

a – нечетное число

c – взаимно прост с модулем m

b = a – 1 кратно p для каждого простого p, делителя m b кратно 4, если m кратно 4.

Блок-схема:

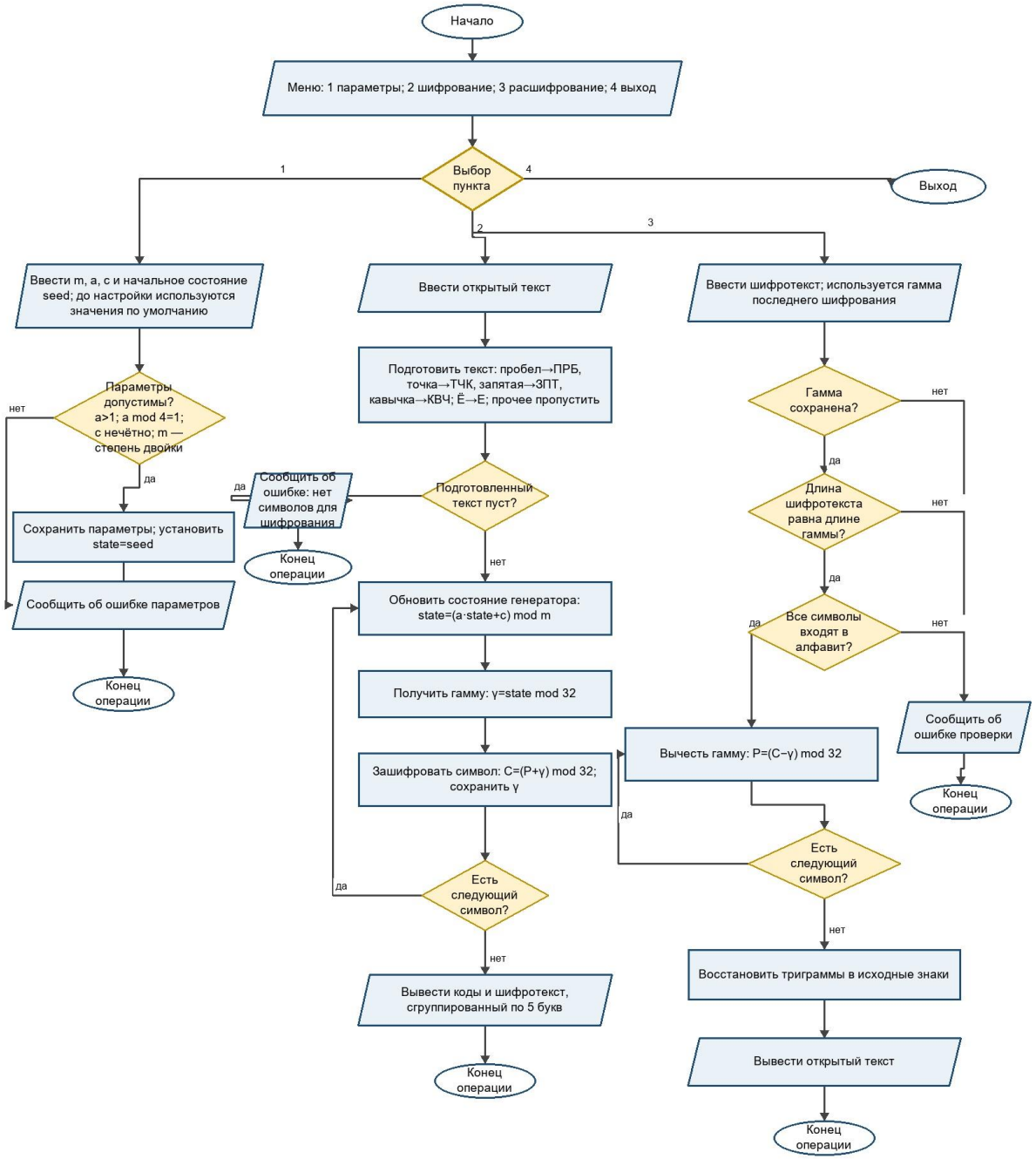

Начало
Меню: 1 параметры; 2 шифрование; 3 расшифрование; 4 выход
Выбор пункта
1
2
3
4
Выход
Ввести m, a, c и начальное состояние seed; до настройки используются значения по умолчанию
Параметры допустимы? a>1; a mod 4=1; c нечётно; m — степень двойки
нет
да
Сохранить параметры; установить state=seed
Сообщить об ошибке параметров
Конец операции
Ввести открытый текст
Подготовить текст: пробел→ПРБ, точка→ТЧК, запятая→ЗПТ, кавычка→КВЧ; Ё→Е; прочее пропустить
Подготовленный текст пуст?
да
Сообщить об ошибке: нет символов для шифрования
Конец операции
нет
Обновить состояние генератора: state=(a·state+c) mod m
Получить гамму: γ=state mod 32
Зашифровать символ: C=(P+γ) mod 32; сохранить γ
Есть следующий символ?
да
нет
Вывести коды и шифротекст, сгруппированный по 5 букв
Конец операции
Ввести шифротекст; используется гамма последнего шифрования
Гамма сохранена?
нет
да
Длина шифротекста равна длине гаммы?
нет
да
Все символы входят в алфавит?
да
нет
Сообщить об ошибке проверки
Конец операции
Вычесть гамму: P=(C−γ) mod 32
Есть следующий символ?
да
нет
Восстановить триграммы в исходные знаки
Вывести открытый текст
Конец операции

Рисунок 22. Блок-схема блокнота Шеннона.

Код программы:

```
       import sys
class ShannonCipher:
    def __init__(self):
        self.a = 1664525
        self.c = 1013904223
        self.m = 256
        self.state = 12345
        self.alphabet = "АБВГДЕЖЗИЙКЛМНОПРСТУФХЦЧШЩЪЫЬЭЮЯ"
        self.last_gamma = []
    def get_char_index(self, ch):
        try:
            return self.alphabet.index(ch)
        except ValueError:
            return -1
    def is_russian_char(self, ch):
        return ch.upper() in self.alphabet or ch.upper() == 'Ё'
    def prepare_text(self, text):
        result = []
        for ch in text:
            if ch == ' ':
                result.append("ПРБ")
            elif ch == '.':
                result.append("ТЧК")
            elif ch == ',':
                result.append("ЗПТ")
            elif ch == '"':
                result.append("КВЧ")
            else:
                upper_ch = ch.upper()
                if upper_ch == 'Ё':
                    upper_ch = 'Е'
                if upper_ch in self.alphabet:
                    result.append(upper_ch)
        return ''.join(result)
    def restore_text(self, text):
        result = []
        i = 0
        length = len(text)
        while i < length:
            if i + 2 < length:
                triple = text[i:i+3]
                if triple == "ПРБ":
                    result.append(' ')
                    i += 3
                    continue
                elif triple == "ТЧК":
                    result.append('.')
                    i += 3
                    continue
                elif triple == "ЗПТ":
                    result.append(',')
                    i += 3
                    continue
                elif triple == "КВЧ":
                    result.append('"')
                    i += 3
```

```
                continue
            result.append(text[i])
            i += 1
        return ''.join(result)
    def lcg(self):
        self.state = (self.a * self.state + self.c) % self.m
        return self.state
    def set_default_parameters(self):
        self.a = 1664525
        self.c = 1013904223
        self.m = 256
        self.state = 12345
        print("Установлены параметры по умолчанию:")
        print(f"a = {self.a}, c = {self.c}, m = {self.m}, seed = {self.state}")
    def set_parameters(self, a, c, m, seed):
        valid = True
        errors = []
        if a <= 1:
            valid = False
            errors.append("- a должно быть больше 1")
        if a % 4 != 1:
            valid = False
            errors.append("- a mod 4 должно быть равно 1")
        if c % 2 == 0:
            valid = False
            errors.append("- c должно быть нечетным")
        if (m & (m - 1)) != 0:
            valid = False
            errors.append("- m должно быть степенью двойки")
        if not valid:
            print("\nОШИБКА: Параметры не соответствуют требованиям:")
            for err in errors:
                print(err)
            return False
        self.a = a
        self.c = c
        self.m = m
        self.state = seed
        print("\nПараметры приняты и обеспечивают максимальный период")
        print(f"a = {self.a}, c = {self.c}, m = {self.m}, T(0) = {self.state}")
        return True
    def encrypt(self, text):
        prepared = self.prepare_text(text)
        if not prepared:
            print("Ошибка: текст пуст!")
            return
        self.last_gamma.clear()
        numbers = []
        letters = []
        for ch in prepared:
            idx = self.get_char_index(ch)
            if idx == -1:
                print(f"Ошибка: символ {ch} не найден в алфавите!")
                return
            gamma = self.lcg() % 32
            self.last_gamma.append(gamma)
            enc_idx = (idx + gamma) % 32
```

```
            enc_ch = self.alphabet[enc_idx]
            numbers.append(enc_idx)
            letters.append(enc_ch)
        raw_cipher = ''.join(letters)
        # Группировка по 5 символов
        grouped = ' '.join(raw_cipher[i:i+5] for i in range(0, len(raw_cipher), 5))
        print("\nЗашифрованный текст: ")
        print("В виде цифр:", ' '.join(str(x) for x in numbers))
        print(f"В виде букв: {grouped}")
    def decrypt(self, encrypted_text, gamma):
        cleaned_text = ''.join(encrypted_text.split())
        if len(cleaned_text) != len(gamma):
            print(f"Ошибка: длина текста ({len(cleaned_text)}) не совпадает с длиной гаммы ({len(gamma)})!")
            return
        temp_chars = []
        for i, ch in enumerate(cleaned_text):
            idx = self.get_char_index(ch)
            if idx == -1:
                print(f"Ошибка: символ {ch} не найден в алфавите!")
                return
            dec_idx = (idx - gamma[i] + 32) % 32
            temp_chars.append(self.alphabet[dec_idx])
        temp = ''.join(temp_chars)
        result = self.restore_text(temp)
        print("\nРасшифрованный текст:")
        print(result)
    def get_last_gamma(self):
        return self.last_gamma
    def print_alphabet(self):
        print("\nАлфавит (32 буквы, индексы 0-31):")
        for i, ch in enumerate(self.alphabet):
            print(f"{i}:{ch}", end=' ')
            if (i + 1) % 8 == 0:
                print()
        print()
def process_user_input(cipher):
    print("\n1. Задать параметры генератора")
    print("2. Зашифровать текст")
    print("3. Расшифровать текст")
    print("4. Выход")
    choice = input("Ваш выбор: ").strip()
    if choice == '1':
        try:
            m = int(input("Введите модуль m (степень двойки): "))
            a = int(input("Введите множитель a (a > 1, a mod 4 = 1): "))
            c = int(input("Введите приращение c (нечетное): "))
            seed = int(input("Введите начальное значение T(0): "))
            cipher.set_parameters(a, c, m, seed)
        except ValueError:
            print("Ошибка: введите целые числа.")
    elif choice == '2':
        text = input("Введите текст для шифрования: ")
        cipher.encrypt(text)
    elif choice == '3':
        encrypted = input("Введите зашифрованный текст: ")
        gamma = cipher.get_last_gamma()
```

```
            if not gamma:
                print("Ошибка: сначала нужно зашифровать текст!")
            else:
                cipher.decrypt(encrypted, gamma)
        elif choice == '4':
            print("Программа завершена.")
            sys.exit(0)
        else:
            print("Неверный выбор!")

def main():
    print("Одноразовый блокнот Шеннона")
    cipher = ShannonCipher()
    while True:
        process_user_input(cipher)

if __name__ == "__main__":
    main()
```

Результат работы программы:

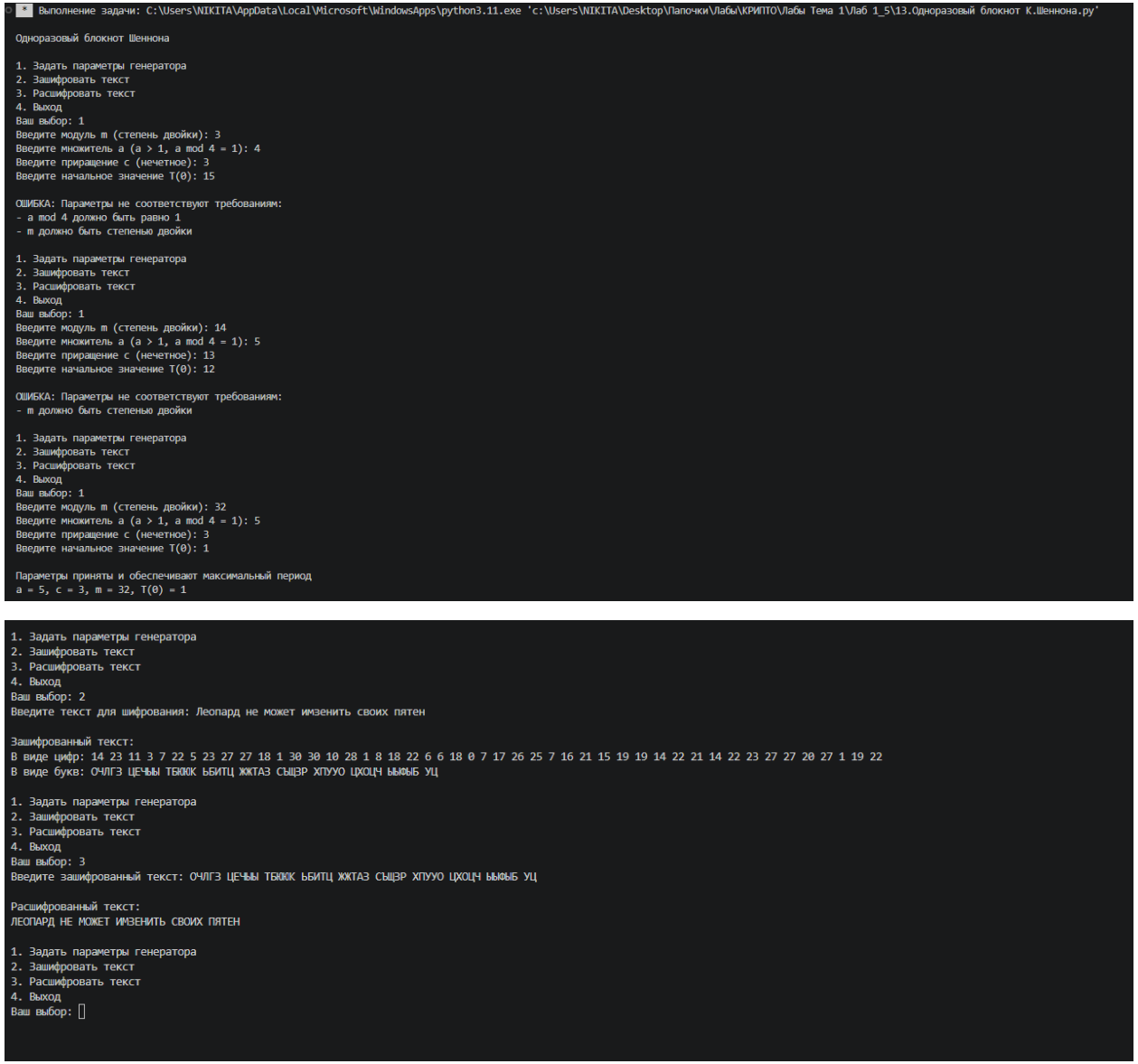


Рисунок 23. Результат выполнения блокнота Шеннона.

Результат теста на 1000 знаков:

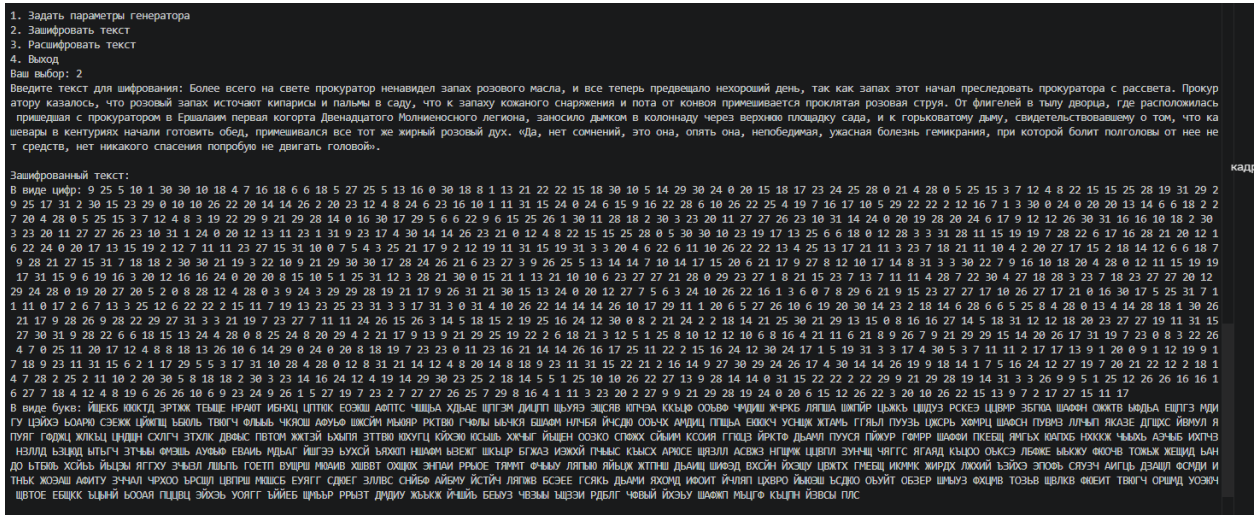


Рисунок 25. Тест на 1000 знаков блокнота Шеннона.

Блок F: Поточные шифры

15. Поточный шифр A5/1

A5/1 — это поточный шифр, используемый в стандарте GSM (Global System for Mobile Communications) для шифрования голосовых вызовов и передачи данных между мобильным телефоном и базовой станцией. Он генерирует гамму (псевдослучайную последовательность битов) с помощью трёх линейных регистров сдвига с обратной связью (LFSR - Linear Feedback Shift Register), которые тактируются неравномерно на основе мажоритарной функции.

Ключевая особенность: шифртекстом является результат сложения гаммы с исходным текстом по модулю 2 (XOR). Генератор гаммы (рисунок 72) в алгоритме A5/1 обеспечивает криптостойкость за счёт неравномерного тактирования регистров.

Блок-схема:

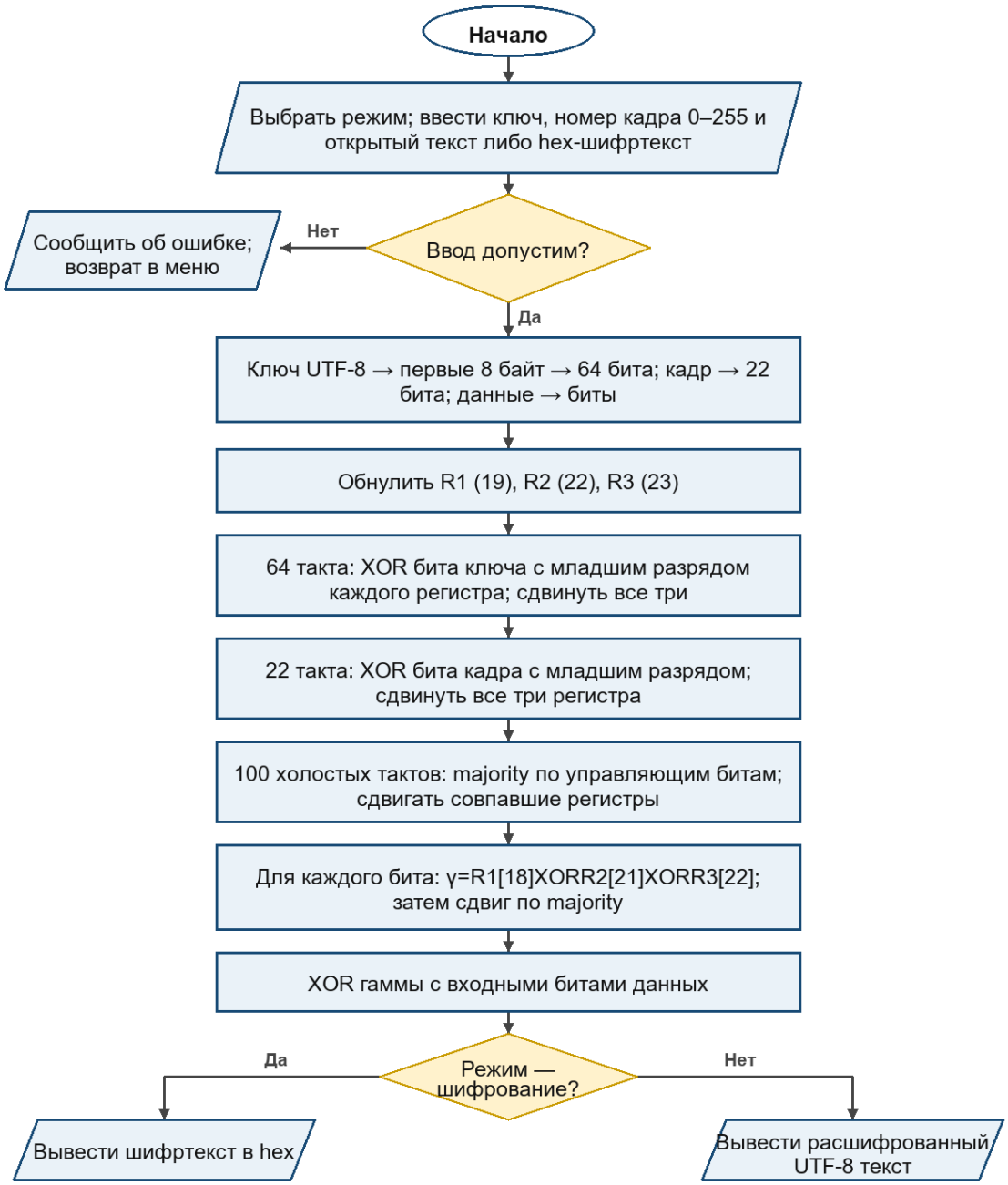

Начало
Выбрать режим; ввести ключ, номер кадра 0–255 и открытый текст либо hex-шифртекст
Ввод допустим?
Нет
Сообщить об ошибке; возврат в меню
Да
Ключ UTF-8 → первые 8 байт → 64 бита; кадр → 22 бита; данные → биты
Обнулить R1 (19), R2 (22), R3 (23)
64 такта: XOR бита ключа с младшим разрядом каждого регистра; сдвинуть все три
22 такта: XOR бита кадра с младшим разрядом; сдвинуть все три регистра
100 холостых тактов: majority по управляющим битам; сдвигать совпавшие регистры
Для каждого бита: γ=R1[18]XORR2[21]XORR3[22]; затем сдвиг по majority
XOR гаммы с входными битами данных
Режим — шифрование?
Да
Нет
Вывести шифртекст в hex
Вывести расшифрованный UTF-8 текст

Рисунок 26. Блок-схема шифра A5/1.



Рисунок 27. Генерация гаммы для A5/1.

Код программы:

```
import random

R1_LEN = 19
R2_LEN = 22
R3_LEN = 23

R1_FEEDBACK = [18, 17, 16, 13]
R2_FEEDBACK = [21, 20]
R3_FEEDBACK = [22, 21, 20, 7]

R1_CLOCK = 8
R2_CLOCK = 10
R3_CLOCK = 10

def majority(b1, b2, b3):
    return (b1 & b2) | (b1 & b3) | (b2 & b3)

def clock_register(reg, length, feedback_bits):
    out_bit = reg[length - 1]
    feedback = 0
    for idx in feedback_bits:
```

```
        feedback ^= reg[idx]
    for i in range(length - 1, 0, -1):
        reg[i] = reg[i - 1]
    reg[0] = feedback
    return out_bit

def string_to_64bit_key(text):
    """Преобразует текст в 64 бита (первые 8 байт UTF-8, дополняя нулями)."""
    data = text.encode('utf-8')[:8]
    if len(data) < 8:
        data += b'\x00' * (8 - len(data))
    bits = []
    for byte in data:
        for i in range(7, -1, -1):
            bits.append((byte >> i) & 1)
    return bits

def int_to_frame(num):
    """
    Преобразует целое число от 0 до 255 в список из 22 битов:
    """
    if not (0 <= num <= 255):
```

```
        raise ValueError("Номер кадра должен быть в диапазоне 0 … 255")
    # 8 бит числа
    low_bits = []
    for i in range(7, -1, -1):
        low_bits.append((num >> i) & 1)
    # 14 нулевых старших битов + 8 бит числа = 22 бита
    frame_bits = [0] * 14 + low_bits
    return frame_bits

def a5_1_keystream(key_bits, frame_bits, keystream_length):
    """Генерация гаммы A5/1."""
    r1 = [0] * R1_LEN
    r2 = [0] * R2_LEN
    r3 = [0] * R3_LEN

    # Загрузка 64-битного ключа
    for i in range(64):
        bit = key_bits[i]
        r1[0] ^= bit
        r2[0] ^= bit
        r3[0] ^= bit
        clock_register(r1, R1_LEN, R1_FEEDBACK)
```

```
    clock_register(r2, R2_LEN, R2_FEEDBACK)
    clock_register(r3, R3_LEN, R3_FEEDBACK)

# Загрузка 22-битного номера кадра
for i in range(22):
    bit = frame_bits[i]
    r1[0] ^= bit
    r2[0] ^= bit
    r3[0] ^= bit
    clock_register(r1, R1_LEN, R1_FEEDBACK)
    clock_register(r2, R2_LEN, R2_FEEDBACK)
    clock_register(r3, R3_LEN, R3_FEEDBACK)

# Холостые такты
for _ in range(100):
    maj = majority(r1[R1_CLOCK], r2[R2_CLOCK], r3[R3_CLOCK])
    if r1[R1_CLOCK] == maj:
        clock_register(r1, R1_LEN, R1_FEEDBACK)
    if r2[R2_CLOCK] == maj:
        clock_register(r2, R2_LEN, R2_FEEDBACK)
    if r3[R3_CLOCK] == maj:
        clock_register(r3, R3_LEN, R3_FEEDBACK)
```

```
    # Генерация гаммы
    keystream = []
    for _ in range(keystream_length):
        maj = majority(r1[R1_CLOCK], r2[R2_CLOCK], r3[R3_CLOCK])
        out_bit = r1[R1_LEN - 1] ^ r2[R2_LEN - 1] ^ r3[R3_LEN - 1]
        keystream.append(out_bit)
        if r1[R1_CLOCK] == maj:
            clock_register(r1, R1_LEN, R1_FEEDBACK)
        if r2[R2_CLOCK] == maj:
            clock_register(r2, R2_LEN, R2_FEEDBACK)
        if r3[R3_CLOCK] == maj:
            clock_register(r3, R3_LEN, R3_FEEDBACK)
    return keystream

def text_to_bits(text):
    bits = []
    for byte in text.encode('utf-8'):
        for i in range(7, -1, -1):
            bits.append((byte >> i) & 1)
    return bits
```

```
def bits_to_text(bits):
    bytes_list = []
    for i in range(0, len(bits), 8):
        if i + 7 < len(bits):
            byte = 0
            for j in range(8):
                byte |= (bits[i + j] << (7 - j))
            bytes_list.append(byte)
    return bytes(bytes_list).decode('utf-8', errors='ignore')

def bits_to_hex(bits):
    while len(bits) % 4 != 0:
        bits.append(0)
    hex_str = []
    for i in range(0, len(bits), 4):
        val = (bits[i] << 3) | (bits[i+1] << 2) | (bits[i+2] << 1) | bits[i+3]
        hex_str.append(f"{val:X}")
    return ''.join(hex_str)

def hex_to_bits(hex_str):
    bits = []
    for ch in hex_str:
```

```
        val = int(ch, 16)
        bits.append((val >> 3) & 1)
        bits.append((val >> 2) & 1)
        bits.append((val >> 1) & 1)
        bits.append(val & 1)
    return bits


def encrypt():
    print("\nШИФРОВАНИЕ")
    key_text = input("Введите ключ (текст): ")
    try:
        frame_num = int(input("Введите номер кадра (целое число от 0 до 255): "))
    except ValueError:
        print("Ошибка: необходимо ввести целое число")
        return

    plaintext = input("Введите текст для шифрования: ")
    if not plaintext:
        print("Текст не введён")
        return

    key_bits = string_to_64bit_key(key_text)
```

```
    try:
        frame_bits = int_to_frame(frame_num)
    except ValueError as e:
        print(e)
        return

    plain_bits = text_to_bits(plaintext)
    keystream = a5_1_keystream(key_bits, frame_bits, len(plain_bits))
    cipher_bits = [plain_bits[i] ^ keystream[i] for i in range(len(plain_bits))]

    print("\nРЕЗУЛЬТАТ")
    print(f"Ключ (hex): {bits_to_hex(key_bits)}")
    print(f"Номер кадра: {frame_num}")
    print(f"Зашифрованный текст (hex): {bits_to_hex(cipher_bits)}")

def decrypt():
    print("\nРАСШИФРОВАНИЕ")
    key_text = input("Введите ключ (текст): ")
    try:
        frame_num = int(input("Введите номер кадра (целое число от 0 до 255): "))
    except ValueError:
        print("Ошибка: необходимо ввести целое число")
```

```
    return

cipher_hex = input("Введите зашифрованный текст в hex: ").strip()
if not cipher_hex:
    print("Текст не введён")
    return

key_bits = string_to_64bit_key(key_text)
try:
    frame_bits = int_to_frame(frame_num)
except ValueError as e:
    print(e)
    return

cipher_bits = hex_to_bits(cipher_hex)
keystream = a5_1_keystream(key_bits, frame_bits, len(cipher_bits))
plain_bits = [cipher_bits[i] ^ keystream[i] for i in range(len(cipher_bits))]
plaintext = bits_to_text(plain_bits)

print("\nРЕЗУЛЬТАТ")
print(f"Ключ (hex): {bits_to_hex(key_bits)}")
print(f"Номер кадра: {frame_num}")
```

```
    print(f"Расшифрованный текст: {plaintext}")


def main():
    while True:
        print("\nПОТОЧНЫЙ ШИФР A5/1")
        print("1. Зашифровать текст")
        print("2. Расшифровать текст")
        print("3. Выход")
        choice = input("Выберите действие: ")

        if choice == '1':
            encrypt()
        elif choice == '2':
            decrypt()
        elif choice == '3':
            print("Программа завершена.")
            break
        else:
            print("Неверный выбор, попробуйте снова.")

if __name__ == "__main__":
    main()
```

Результат работы программы:



Рисунок 28. Результат контрольного запуска шифра A5/1, вариант 20.

Контрольный пример пересчитан по текущему исходному коду A5/1.

Таблица 1. Параметры тестового запуска.

Параметр	Значение из запуска
Ключ, введённый пользователем	АВГУСТ
Байты ключа, использованные кодом (hex)	D0 90 D0 92 D0 93 D0 A3
Номер кадра	155
22 бита кадра, созданные int_to_frame()	0000000000000010011011
Сообщение	Леопард не может изменить своих пятен
Кодирование сообщения	UTF-8; 69 байт, 552 бита

Примечание. string_to_64bit_key() берёт первые 8 байт UTF-8 и дополняет нулями, если их меньше 8. Для ключа «АВГУСТ» это байты первых четырёх букв. int_to_frame() ограничивает номер кадра диапазоном 0–255 и добавляет 14 нулевых старших битов.

Таблица 2. Состояния регистров после этапов инициализации.

Этап	R1 (19 бит)	R2 (22 бита)	R3 (23 бита)
После загрузки ключа	0101100011001101110	11011101000000010000110	11000001000111011110111
После загрузки кадра	0010100110001011110	01011111000000011000101	10100100111010111100001
После 100 тактов разогрева	0011100100000010101	00111001100101000100001	00100011111110000111001

Состояния записаны в порядке элементов массивов программы: reg[0] → reg[n−1], не в условной записи от старшего бита к младшему.

Таблица 3. Первые 16 тактов генерации гаммы и шифрования.

	Входы S1S2S3	Большинство	Тактируются	Гамма z_i	Открытый p_i	Шифр c_i
	001	0	R1/R2	1	1	0
	101	1	R1/R3	1	1	0
	001	0	R1/R2	0	0	0
	011	1	R2/R3	1	1	0
	011	1	R2/R3	0	0	0
	001	0	R1/R2	0	0	0
	101	1	R1/R3	0	0	0
	101	1	R1/R3	1	0	1
	100	0	R2/R3	1	1	0
0	110	1	R1/R2	0	0	0
1	010	0	R1/R3	0	0	0
2	010	0	R1/R3	0	1	1
3	011	1	R2/R3	0	1	1
4	010	0	R1/R3	1	0	1
5	110	1	R1/R2	0	1	1
6	000	0	R1/R2/R3	0	1	1

На каждом такте программа вычисляет z_i = R1[18] XOR R2[21] XOR R3[22], затем $c_i = p_i$ XOR z_i. Сдвигаются только регистры, управляющий бит которых совпал с большинством.

Полный шифртекст, выведенный программой (hex):

```
011FF38D15D7BDCCF3C2E90BAD973F37FD945E9DDA3E7676
3232A577CFF10D61E640045D16EC1184077EEFE8C606D44E
E1365A43F43C9D384B15F173C741ABFD57ED0B8FE7
```

Проверка: повторное расшифрование с тем же ключом и номером кадра возвращает исходное сообщение. Таблица воспроизводит поведение учебной программы и не заявляет о соответствии промышленной GSM-реализации A5/1.

Тест на 1000 знаков:



Рисунок 29. Тест на 1000 знаков шифра A5/1.

Блок G: Комбинационные шифры

19. Шифр AES

AES (Advanced Encryption Standard) — это симметричный блочный шифр, принятый в качестве стандарта шифрования в США в 2001 году. Он пришёл на смену устаревшему DES (Data Encryption Standard). AES является одним из наиболее распространённых алгоритмов симметричного шифрования в мире, используется в протоколах SSL/TLS, Wi-Fi (WPA2), файловых архиваторах, банковских системах и во многих других приложениях.

Ключевая особенность: AES работает с блоками данных фиксированного размера (128 бит = 16 байт) и поддерживает ключи длиной 128, 192 или 256 бит.

Блок-схема:

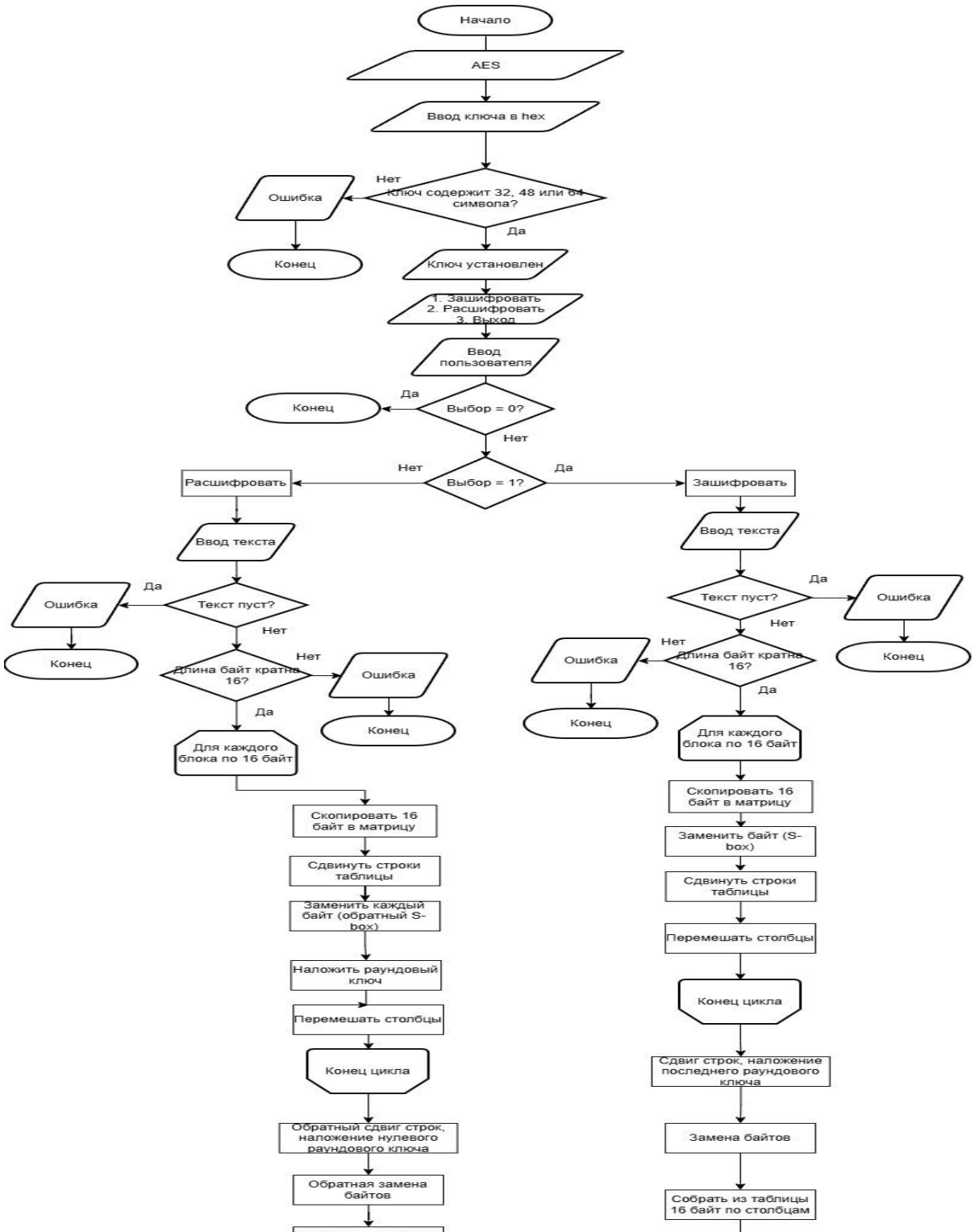

Начало
AES
Ввод ключа в hex
Нет
Ключ содержит 32, 48 или 64 символа?
Ошибка
Конец
Да
Ключ установлен
1. Зашифровать
2. Расшифровать
3. Выход
Ввод пользователя
Да
Выбор = 0?
Конец
Нет
Нет
Выбор = 1?
Да
Расшифровать
Зашифровать
Ввод текста
Ввод текста
Да
Текст пуст?
Ошибка
Конец
Нет
Нет
Длина байт кратна 16?
Ошибка
Конец
Да
Для каждого блока по 16 байт
Скопировать 16 байт в матрицу
Сдвинуть строки таблицы
Заменить каждый байт (обратный S-box)
Наложить раундовый ключ
Перемешать столбцы
Конец цикла
Обратный сдвиг строк, наложение нулевого раундового ключа
Обратная замена байтов
Да
Текст пуст?
Ошибка
Конец
Нет
Нет
Длина байт кратна 16?
Ошибка
Конец
Да
Для каждого блока по 16 байт
Скопировать 16 байт в матрицу
Заменить байт (S-box)
Сдвинуть строки таблицы
Перемешать столбцы
Конец цикла
Сдвиг строк, наложение последнего раундового ключа
Замена байтов
Собрать из таблицы 16 байт по столбцам

Рисунок 30. Блок-схема шифра AES.

Код программы:

```
import sys


# Таблицы S-box, обратный S-box и Rcon (как в оригинале)
SBOX = [
    0x63, 0x7c, 0x77, 0x7b, 0xf2, 0x6b, 0x6f, 0xc5, 0x30, 0x01, 0x67, 0x2b, 0xfe, 0xd7, 0xab, 0x76,
    0xca, 0x82, 0xc9, 0x7d, 0xfa, 0x59, 0x47, 0xf0, 0xad, 0xd4, 0xa2, 0xaf, 0x9c, 0xa4, 0x72, 0xc0,
    0xb7, 0xfd, 0x93, 0x26, 0x36, 0x3f, 0xf7, 0xcc, 0x34, 0xa5, 0xe5, 0xf1, 0x71, 0xd8, 0x31, 0x15,
    0x04, 0xc7, 0x23, 0xc3, 0x18, 0x96, 0x05, 0x9a, 0x07, 0x12, 0x80, 0xe2, 0xeb, 0x27, 0xb2, 0x75,
    0x09, 0x83, 0x2c, 0x1a, 0x1b, 0x6e, 0x5a, 0xa0, 0x52, 0x3b, 0xd6, 0xb3, 0x29, 0xe3, 0x2f, 0x84,
    0x53, 0xd1, 0x00, 0xed, 0x20, 0xfc, 0xb1, 0x5b, 0x6a, 0xcb, 0xbe, 0x39, 0x4a, 0x4c, 0x58, 0xcf,
    0xd0, 0xef, 0xaa, 0xfb, 0x43, 0x4d, 0x33, 0x85, 0x45, 0xf9, 0x02, 0x7f, 0x50, 0x3c, 0x9f, 0xa8,
    0x51, 0xa3, 0x40, 0x8f, 0x92, 0x9d, 0x38, 0xf5, 0xbc, 0xb6, 0xda, 0x21, 0x10, 0xff, 0xf3, 0xd2,
    0xcd, 0x0c, 0x13, 0xec, 0x5f, 0x97, 0x44, 0x17, 0xc4, 0xa7, 0x7e, 0x3d, 0x64, 0x5d, 0x19, 0x73,
    0x60, 0x81, 0x4f, 0xdc, 0x22, 0x2a, 0x90, 0x88, 0x46, 0xee, 0xb8, 0x14, 0xde, 0x5e, 0x0b, 0xdb,
    0xe0, 0x32, 0x3a, 0x0a, 0x49, 0x06, 0x24, 0x5c, 0xc2, 0xd3, 0xac, 0x62, 0x91, 0x95, 0xe4, 0x79,
    0xe7, 0xc8, 0x37, 0x6d, 0x8d, 0xd5, 0x4e, 0xa9, 0x6c, 0x56, 0xf4, 0xea, 0x65, 0x7a, 0xae, 0x08,
    0xba, 0x78, 0x25, 0x2e, 0x1c, 0xa6, 0xb4, 0xc6, 0xe8, 0xdd, 0x74, 0x1f, 0x4b, 0xbd, 0x8b, 0x8a,
    0x70, 0x3e, 0xb5, 0x66, 0x48, 0x03, 0xf6, 0x0e, 0x61, 0x35, 0x57, 0xb9, 0x86, 0xc1, 0x1d, 0x9e,
    0xe1, 0xf8, 0x98, 0x11, 0x69, 0xd9, 0x8e, 0x94, 0x9b, 0x1e, 0x87, 0xe9, 0xce, 0x55, 0x28, 0xdf,
    0x8c, 0xa1, 0x89, 0x0d, 0xbf, 0xe6, 0x42, 0x68, 0x41, 0x99, 0x2d, 0x0f, 0xb0, 0x54, 0xbb, 0x16
]
```

```
INV_SBOX = [
    0x52, 0x09, 0x6a, 0xd5, 0x30, 0x36, 0xa5, 0x38, 0xbf, 0x40, 0xa3, 0x9e, 0x81, 0xf3, 0xd7, 0xfb,
    0x7c, 0xe3, 0x39, 0x82, 0x9b, 0x2f, 0xff, 0x87, 0x34, 0x8e, 0x43, 0x44, 0xc4, 0xde, 0xe9, 0xcb,
    0x54, 0x7b, 0x94, 0x32, 0xa6, 0xc2, 0x23, 0x3d, 0xee, 0x4c, 0x95, 0x0b, 0x42, 0xfa, 0xc3, 0x4e,
    0x08, 0x2e, 0xa1, 0x66, 0x28, 0xd9, 0x24, 0xb2, 0x76, 0x5b, 0xa2, 0x49, 0x6d, 0x8b, 0xd1, 0x25,
    0x72, 0xf8, 0xf6, 0x64, 0x86, 0x68, 0x98, 0x16, 0xd4, 0xa4, 0x5c, 0xcc, 0x5d, 0x65, 0xb6, 0x92,
    0x6c, 0x70, 0x48, 0x50, 0xfd, 0xed, 0xb9, 0xda, 0x5e, 0x15, 0x46, 0x57, 0xa7, 0x8d, 0x9d, 0x84,
    0x90, 0xd8, 0xab, 0x00, 0x8c, 0xbc, 0xd3, 0x0a, 0xf7, 0xe4, 0x58, 0x05, 0xb8, 0xb3, 0x45, 0x06,
    0xd0, 0x2c, 0x1e, 0x8f, 0xca, 0x3f, 0x0f, 0x02, 0xc1, 0xaf, 0xbd, 0x03, 0x01, 0x13, 0x8a, 0x6b,
    0x3a, 0x91, 0x11, 0x41, 0x4f, 0x67, 0xdc, 0xea, 0x97, 0xf2, 0xcf, 0xce, 0xf0, 0xb4, 0xe6, 0x73,
    0x96, 0xac, 0x74, 0x22, 0xe7, 0xad, 0x35, 0x85, 0xe2, 0xf9, 0x37, 0xe8, 0x1c, 0x75, 0xdf, 0x6e,
    0x47, 0xf1, 0x1a, 0x71, 0x1d, 0x29, 0xc5, 0x89, 0x6f, 0xb7, 0x62, 0x0e, 0xaa, 0x18, 0xbe, 0x1b,
    0xfc, 0x56, 0x3e, 0x4b, 0xc6, 0xd2, 0x79, 0x20, 0x9a, 0xdb, 0xc0, 0xfe, 0x78, 0xcd, 0x5a, 0xf4,
    0x1f, 0xdd, 0xa8, 0x33, 0x88, 0x07, 0xc7, 0x31, 0xb1, 0x12, 0x10, 0x59, 0x27, 0x80, 0xec, 0x5f,
    0x60, 0x51, 0x7f, 0xa9, 0x19, 0xb5, 0x4a, 0x0d, 0x2d, 0xe5, 0x7a, 0x9f, 0x93, 0xc9, 0x9c, 0xef,
    0xa0, 0xe0, 0x3b, 0x4d, 0xae, 0x2a, 0xf5, 0xb0, 0xc8, 0xeb, 0xbb, 0x3c, 0x83, 0x53, 0x99, 0x61,
    0x17, 0x2b, 0x04, 0x7e, 0xba, 0x77, 0xd6, 0x26, 0xe1, 0x69, 0x14, 0x63, 0x55, 0x21, 0x0c, 0x7d
]

RCON = [0x00, 0x01, 0x02, 0x04, 0x08, 0x10, 0x20, 0x40, 0x80, 0x1b, 0x36]
```

```
def xtime(x):
    return ((x << 1) ^ (0x1b if (x & 0x80) else 0x00)) & 0xff


class AES:
    def __init__(self):
        self.Nk = 0
        self.Nr = 0
        self.w = [0] * (60 * 4)

    def set_key(self, key):
        key_len = len(key)
        self.Nk = key_len // 4
        self.Nr = {4: 10, 6: 12, 8: 14}[self.Nk]
        for i in range(self.Nk):
            self.w[4*i + 0] = key[4*i + 0]
            self.w[4*i + 1] = key[4*i + 1]
            self.w[4*i + 2] = key[4*i + 2]
            self.w[4*i + 3] = key[4*i + 3]
        for i in range(self.Nk, 4 * (self.Nr + 1)):
            temp = [self.w[4*(i-1)+0], self.w[4*(i-1)+1], self.w[4*(i-1)+2], self.w[4*(i-1)+3]]
            if i % self.Nk == 0:
                temp = [temp[1], temp[2], temp[3], temp[0]]
```

```
            temp = [SBOX[b] for b in temp]
            temp[0] ^= RCON[i // self.Nk]
        elif self.Nk > 6 and i % self.Nk == 4:
            temp = [SBOX[b] for b in temp]
        self.w[4*i + 0] = self.w[4*(i - self.Nk) + 0] ^ temp[0]
        self.w[4*i + 1] = self.w[4*(i - self.Nk) + 1] ^ temp[1]
        self.w[4*i + 2] = self.w[4*(i - self.Nk) + 2] ^ temp[2]
        self.w[4*i + 3] = self.w[4*(i - self.Nk) + 3] ^ temp[3]

def add_round_key(self, state, rnd):
    for i in range(4):
        for j in range(4):
            state[j][i] ^= self.w[4 * (rnd * 4 + i) + j]

def sub_bytes(self, state):
    for i in range(4):
        for j in range(4):
            state[i][j] = SBOX[state[i][j]]

def inv_sub_bytes(self, state):
    for i in range(4):
        for j in range(4):
```

```
            state[i][j] = INV_SBOX[state[i][j]]

    def shift_rows(self, state):
        state[1] = state[1][1:] + state[1][:1]
        state[2] = state[2][2:] + state[2][:2]
        state[3] = state[3][3:] + state[3][:3]

    def inv_shift_rows(self, state):
        state[1] = state[1][-1:] + state[1][:-1]
        state[2] = state[2][-2:] + state[2][:-2]
        state[3] = state[3][-3:] + state[3][:-3]

    def mix_columns(self, state):
        for j in range(4):
            a = state[0][j]
            b = state[1][j]
            c = state[2][j]
            d = state[3][j]
            state[0][j] = xtime(a) ^ xtime(b) ^ b ^ c ^ d
            state[1][j] = a ^ xtime(b) ^ xtime(c) ^ c ^ d
            state[2][j] = a ^ b ^ xtime(c) ^ xtime(d) ^ d
            state[3][j] = xtime(a) ^ a ^ b ^ c ^ xtime(d)
```

```
def inv_mix_columns(self, state):
    """Обратное преобразование столбцов AES по матрице 14/11/13/9."""
    def gf_mul(value, factor):
        result = 0
        while factor:
            if factor & 1:
                result ^= value
            value = xtime(value)
            factor >>= 1
        return result

    for col in range(4):
        a, b, c, d = (state[row][col] for row in range(4))
        state[0][col] = gf_mul(a, 14) ^ gf_mul(b, 11) ^ gf_mul(c, 13) ^ gf_mul(d, 9)
        state[1][col] = gf_mul(a, 9) ^ gf_mul(b, 14) ^ gf_mul(c, 11) ^ gf_mul(d, 13)
        state[2][col] = gf_mul(a, 13) ^ gf_mul(b, 9) ^ gf_mul(c, 14) ^ gf_mul(d, 11)
        state[3][col] = gf_mul(a, 11) ^ gf_mul(b, 13) ^ gf_mul(c, 9) ^ gf_mul(d, 14)

def encrypt_block(self, plain):
    state = [[0]*4 for _ in range(4)]
    for i in range(4):
```

```
            for j in range(4):
                state[j][i] = plain[i*4 + j]
        self.add_round_key(state, 0)
        for rnd in range(1, self.Nr):
            self.sub_bytes(state)
            self.shift_rows(state)
            self.mix_columns(state)
            self.add_round_key(state, rnd)
        self.sub_bytes(state)
        self.shift_rows(state)
        self.add_round_key(state, self.Nr)
        out = bytearray(16)
        for i in range(4):
            for j in range(4):
                out[i*4 + j] = state[j][i]
        return bytes(out)

    def decrypt_block(self, cipher):
        state = [[0]*4 for _ in range(4)]
        for i in range(4):
            for j in range(4):
                state[j][i] = cipher[i*4 + j]
```

```
        self.add_round_key(state, self.Nr)
        for rnd in range(self.Nr - 1, 0, -1):
            self.inv_shift_rows(state)
            self.inv_sub_bytes(state)
            self.add_round_key(state, rnd)
            self.inv_mix_columns(state)
        self.inv_shift_rows(state)
        self.inv_sub_bytes(state)
        self.add_round_key(state, 0)
        out = bytearray(16)
        for i in range(4):
            for j in range(4):
                out[i*4 + j] = state[j][i]
        return bytes(out)

class AESCipher:
    def __init__(self):
        self.aes = AES()
        self.key_set = False

    def set_key_from_hex(self, key_hex):
        key_hex = key_hex.strip().lower()
```

```
    if len(key_hex) not in (32, 48, 64):
        print("Ошибка: длина ключа должна быть 32, 48 или 64 hex-символа (16, 24 или 32 байта).")
        return False
    try:
        key_bytes = bytes.fromhex(key_hex)
    except ValueError:
        print("Ошибка: неверный hex-формат ключа.")
        return False
    self.aes.set_key(key_bytes)
    self.key_set = True
    bits = len(key_bytes) * 8
    print(f"Ключ установлен ({bits} бит): {key_hex.upper()}")
    return True

def encrypt_hex(self, plain_hex):
    if not self.key_set:
        print("Ошибка: ключ не установлен!")
        return ""
    try:
        data = bytes.fromhex(plain_hex)
    except ValueError:
        print("Ошибка: неверный hex-формат открытого текста.")
```

```
            return ""
        # Требуем, чтобы длина данных была кратна 16 байтам
        if len(data) % 16 != 0:
            print("Ошибка: длина открытого текста должна быть кратна 16 байтам (32 hex-символа).")
            return ""
        cipher = bytearray()
        for i in range(0, len(data), 16):
            block = data[i:i+16]
            enc_block = self.aes.encrypt_block(block)
            cipher.extend(enc_block)
        return cipher.hex().upper()

    def decrypt_hex(self, cipher_hex):
        if not self.key_set:
            print("Ошибка: ключ не установлен!")
            return ""
        try:
            cipher = bytes.fromhex(cipher_hex)
        except ValueError:
            print("Ошибка: неверный hex-формат зашифрованного текста.")
            return ""
        if len(cipher) % 16 != 0:
```

```
            print("Ошибка: длина зашифрованного текста должна быть кратна 16 байтам.")
            return ""
        plain = bytearray()
        for i in range(0, len(cipher), 16):
            block = cipher[i:i+16]
            dec_block = self.aes.decrypt_block(block)
            plain.extend(dec_block)
        return plain.hex().upper()


def main():
    cipher = AESCipher()
    while True:
        print("\nAES")
        print("1. Установить ключ (hex)")
        print("2. Зашифровать hex")
        print("3. Расшифровать hex")
        print("4. Выход")
        choice = input("Ваш выбор: ").strip()
        if choice == '1':
            key_hex = input("Введите ключ в hex: ").strip()
            cipher.set_key_from_hex(key_hex)
        elif choice == '2':
```

```
            plain_hex = input("Введите открытый текст в hex: ").strip()
            enc = cipher.encrypt_hex(plain_hex)
            if enc:
                print(f"Зашифрованный текст: {enc}")
        elif choice == '3':
            cipher_hex = input("Введите зашифрованный текст в hex: ").strip()
            dec = cipher.decrypt_hex(cipher_hex)
            if dec:
                print(f"Расшифрованный текст: {dec}")
        elif choice == '4':
            print("Программа завершена.")
            break
        else:
            print("Неверный выбор.")

if __name__ == "__main__":
    main()
```

Результат работы программы:



Рисунок 31. Результат контрольного запуска AES, вариант 20.

Блок H: Ассиметричные шифры

21. Шифр RSA

RSA — это один из первых и наиболее известных асимметричных криптографических алгоритмов, названный в честь его создателей: Рона Ривеста, Ади Шамира и Леонарда Адлемана (1977 год). Алгоритм основан на вычислительной сложности факторизации больших целых чисел (разложения числа на простые множители).

Ключевая особенность: RSA использует два ключа: открытый (для шифрования) и закрытый (для расшифрования). Невозможно вычислить закрытый ключ из открытого за разумное время при достаточно больших размерах ключа.

Блок-схема:

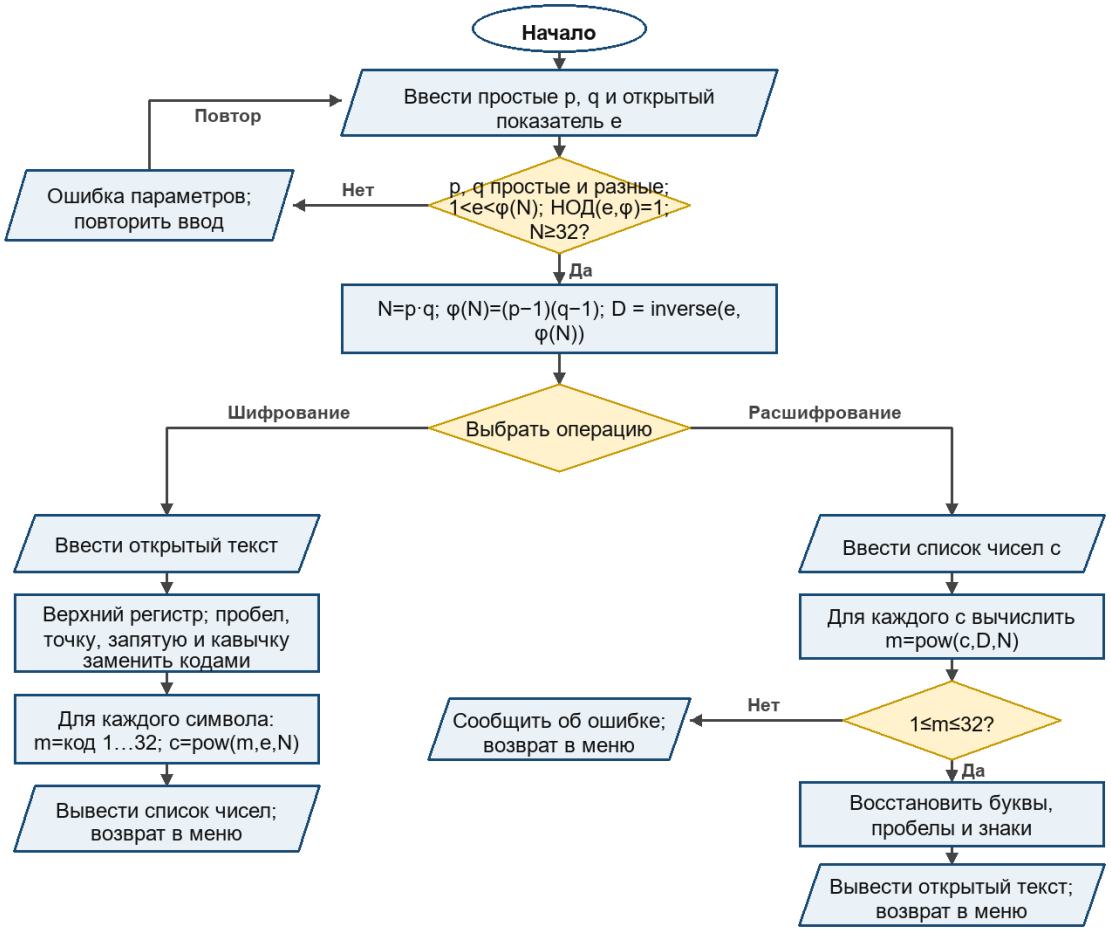


Рисунок 32. Блок-схема шифра RSA.

Код программы:

```
import sys
import math

# ---- Вспомогательные математические функции ----
def mod_pow(a, exp, mod):
    """Быстрое возведение в степень по модулю"""
    result = 1
    a %= mod
    while exp > 0:
        if exp & 1:
            result = (result * a) % mod
        a = (a * a) % mod
        exp >>= 1
    return result

def mod_inverse(a, m):
    """Расширенный алгоритм Евклида для нахождения обратного элемента"""
    m0, x0, x1 = m, 0, 1
    if m == 1:
        return 0
    while a > 1:
        q = a // m
        a, m = m, a % m
        x0, x1 = x1 - q * x0, x0
    return x1 + m0 if x1 < 0 else x1

def is_prime(n):
    """Проверка простоты (для небольших чисел)"""
    if n < 2:
        return False
    if n == 2:
        return True
    if n % 2 == 0:
        return False
    i = 3
    while i * i <= n:
        if n % i == 0:
            return False
        i += 2
    return True

def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

class RSACipher:
    def __init__(self):
        self.p = self.q = self.N = self.phi = self.E = self.D = 0
        self.key_set = False
        self.alphabet = "АБВГДЕЖЗИЙКЛМНОПРСТУФХЦЧШЩЪЫЬЭЮЯ"   # 32 буквы

    def to_upper_russian(self, c):
        """Приведение русской буквы к верхнему регистру, Ё -> Е"""
        if 'а' <= c <= 'я' or 'А' <= c <= 'Я':
            return c.upper()
        if c == 'ё' or c == 'Ё':
            return 'Е'
        return c

    def is_russian_char(self, c):
        """Проверка, является ли символ русской буквой (включая Ё)"""
```

```
    return ('А' <= c <= 'Я') or ('а' <= c <= 'я') or c in ('Ё', 'ё')
def prepare_text(self, text):
    """Подготовка текста: пробел -> ПРБ, точка -> ТЧК, запятая -> ЗПТ, кавычка -> КВЧ"""
    result = []
    for ch in text:
        if ch == ' ':
            result.append("ПРБ")
        elif ch == '.':
            result.append("ТЧК")
        elif ch == ',':
            result.append("ЗПТ")
        elif ch == '"':
            result.append("КВЧ")
        else:
            upper = self.to_upper_russian(ch)
            if self.is_russian_char(upper):
                result.append(upper)
    return ''.join(result)
def restore_text(self, text):
    """Восстановление пробелов, точек, запятых и кавычек после расшифровки"""
    result = []
    i = 0
    n = len(text)
    while i < n:
        if i + 2 < n:
            triple = text[i:i+3]
            if triple == "ПРБ":
                result.append(' ')
                i += 3
                continue
            elif triple == "ТЧК":
                result.append('.')
                i += 3
                continue
            elif triple == "ЗПТ":
                result.append(',')
                i += 3
                continue
            elif triple == "КВЧ":
                result.append('"')
                i += 3
                continue
        result.append(text[i])
        i += 1
    return ''.join(result)
def char_to_int(self, c):
    """Буква -> число 1..32 (нумерация с 1)"""
    try:
        return self.alphabet.index(c) + 1
    except ValueError:
        return -1
def int_to_char(self, n):
    """Число 1..32 -> буква"""
    if 1 <= n <= len(self.alphabet):
        return self.alphabet[n-1]
    return '?'
# ---- Установка ключа ----
def set_key(self, prime_p, prime_q, exp_e):
    if not is_prime(prime_p):
```

```
            raise ValueError("p не является простым числом.")
        if not is_prime(prime_q):
            raise ValueError("q не является простым числом.")
        if prime_p == prime_q:
            raise ValueError("p и q должны быть разными.")
        N = prime_p * prime_q
        phi = (prime_p - 1) * (prime_q - 1)
        if exp_e <= 1 or exp_e >= phi:
            raise ValueError("E должно быть в диапазоне 1 < E < ϕ(N).")
        if gcd(exp_e, phi) != 1:
            raise ValueError("E и ϕ(N) должны быть взаимно просты.")
        if N < 32:
            raise ValueError(f"Модуль N = {N} слишком мал для кодирования алфавита.")
        private_exponent = mod_inverse(exp_e, phi)
        if exp_e == private_exponent:
            raise ValueError("Открытый и секретный показатели совпадают; выберите другое E.")
        self.p = prime_p
        self.q = prime_q
        self.N = N
        self.phi = phi
        self.E = exp_e
        self.D = private_exponent
        self.key_set = True
        print("\nКЛЮЧИ УСТАНОВЛЕНЫ")
        print(f"p = {self.p}, q = {self.q}")
        print(f"N = {self.N}")
        print(f"ϕ(N) = {self.phi}")
        print(f"Открытый ключ (E, N) = ({self.E}, {self.N})")
        print(f"Секретный ключ D = {self.D}")
        return True
    # ---- Шифрование ----
    def encrypt(self, plain_text):
        if not self.key_set:
            print("Ошибка: ключ не установлен!")
            return ""
        prepared = self.prepare_text(plain_text)
        if not prepared:
            print("Ошибка: текст пуст!")
            return ""
        print(f"\nТекст после обработки: {prepared}")
        cipher_nums = []
        print("\n--- ШИФРОВАНИЕ ---")
        print("Символ -> Код -> Шифр")
        print("-------------------------------")
        for ch in prepared:
            m = self.char_to_int(ch)
            if m == -1:
                print(f"Ошибка: символ {ch} не найден в алфавите!")
                return ""
            ciph = mod_pow(m, self.E, self.N)
            cipher_nums.append(ciph)
            print(f"   {ch}    ->    {m:2d}     -> {ciph}")
        # Формируем строку с числами через пробел
        result = ' '.join(str(x) for x in cipher_nums)
        print("\nЗАШИФРОВАННЫЙ ТЕКСТ")
        print(result)
```

```
        return result
    # ---- Расшифровка ----
    def decrypt(self, cipher_text):
        if not self.key_set:
            print("Ошибка: ключ не установлен!")
            return ""
        # Разбираем строку с числами, разделёнными пробелами
        parts = cipher_text.strip().split()
        if not parts:
            print("Ошибка: не удалось разобрать зашифрованный текст!")
            return ""
        try:
            cipher_nums = [int(x) for x in parts]
        except ValueError:
            print("Ошибка: зашифрованный текст должен содержать только целые числа!")
            return ""
        print("\n--- РАСШИФРОВАНИЕ ---")
        print("Шифр -> Код -> Символ")
        print("-------------------------------")
        decrypted_chars = []
        for ciph in cipher_nums:
            m = mod_pow(ciph, self.D, self.N)
            if m < 1 or m > len(self.alphabet):
                print(f"Ошибка: расшифрованное число {m} вне диапазона 1..32!")
                return ""
            ch = self.int_to_char(m)
            decrypted_chars.append(ch)
            print(f"{ciph} ->    {m:2d}     -> {ch}")
        decrypted_str = ''.join(decrypted_chars)
        result = self.restore_text(decrypted_str)
        print("\nРАСШИФРОВАННЫЙ ТЕКСТ")
        print(result)
        return result

def process_user_input(cipher):
    print("\nRSA")
    print("1. Установить ключ (p, q, e)")
    print("2. Зашифровать текст")
    print("3. Расшифровать текст")
    print("4. Выход")
    choice = input("Ваш выбор: ").strip()
    if choice == '1':
        try:
            p = int(input("Введите простое число p (например, 23): "))
            q = int(input("Введите простое число q (например, 29): "))
            e = int(input("Введите открытый ключ e (например, 3 или 65537): "))
            cipher.set_key(p, q, e)
        except ValueError:
            print("Ошибка: введите целые числа.")
    elif choice == '2':
        text = input("Введите текст для шифрования: ")
        cipher.encrypt(text)
    elif choice == '3':
        text = input("Введите зашифрованный текст (числа через пробел): ")
        cipher.decrypt(text)
    elif choice == '4':
        print("Программа завершена.")
        sys.exit(0)
    else:
```

```
            print("Неверный выбор!")

    def main():
        cipher = RSACipher()
        while True:
            process_user_input(cipher)

    if __name__ == "__main__":
        main()
```

Результат работы программы:

```
Python — контрольный запуск

RSA — фактический вывод реализации из репозитория
Вариант 20: текстовая проверка
Исходный текст: Леопард не может изменить своих пятен.
Параметры: p=61, q=53, e=17, N=3233, ϕ(N)=3120
Вычисленный закрытый показатель: D=2753
Открытый ключ (E, N): (17, 3233)
Выполнено шифрование и расшифрование по коду RSA.
Результат: ЛЕОПАРД НЕ МОЖЕТ ИЗМЕНИТЬ СВОИХ ПЯТЕН.
Проверка: результат совпадает с исходным текстом
```

```
Введите текст для шифрования: Леопард не может изменить своих пятен

Текст после обработки: ЛЕОПАРДПРБНЕПРБМОЖЕТПРБИЗМЕНИТЬПРБСВОИХПРБПЯТЕН

--- ШИФРОВАНИЕ ---
Символ -> Код -> Шифр
--------------------------------
  Л   ->   12   -> 12
  Е   ->    6   -> 18
  О   ->   15   -> 9
  П   ->   16   -> 4
  А   ->    1   -> 1
  Р   ->   17   -> 29
  Д   ->    5   -> 26
  П   ->   16   -> 4
  Р   ->   17   -> 29
  Б   ->    2   -> 8
  Н   ->   14   -> 5
  Е   ->    6   -> 18
  П   ->   16   -> 4
  Р   ->   17   -> 29
  Б   ->    2   -> 8
  М   ->   13   -> 19
  О   ->   15   -> 9
  Ж   ->    7   -> 13
  Е   ->    6   -> 18
  Т   ->   19   -> 28
  П   ->   16   -> 4
  Р   ->   17   -> 29
  Б   ->    2   -> 8
  И   ->    9   -> 3
  З   ->    8   -> 17
  М   ->   13   -> 19
  Е   ->    6   -> 18
  Н   ->   14   -> 5
  И   ->    9   -> 3
  Т   ->   19   -> 28
  Ь   ->   29   -> 2
  П   ->   16   -> 4
  Р   ->   17   -> 29
  Б   ->    2   -> 8
  С   ->   18   -> 24
  В   ->    3   -> 27
  О   ->   15   -> 9
  И   ->    9   -> 3
  Х   ->   22   -> 22
  П   ->   16   -> 4
  Р   ->   17   -> 29
  Б   ->    2   -> 8
  П   ->   16   -> 4
  Я   ->   32   -> 32
  Т   ->   19   -> 28
```

```
ЗАШИФРОВАННЫЙ ТЕКСТ
12 18 9 4 1 29 26 4 29 8 5 18 4 29 8 19 9 13 18 28 4 29 8 3 17 19 18 5 3 28 2 4 29 8 24 27 9 3 22 4 29 8 4 32 28 18 5

RSA
1. Установить ключ (p, q, e)
2. Зашифровать текст
3. Расшифровать текст
4. Выход
Ваш выбор: 3
Введите зашифрованный текст (числа через пробел): 12 18 9 4 1 29 26 4 29 8 5 18 4 29 8 19 9 13 18 28 4 29 8 3 17 19 18 5 3 28 2 4 29 8 24 27 9 3 22 4 29 8 4 32 28 18 5

--- РАСШИФРОВАНИЕ ---
Шифр -> Код -> Символ
```

```
* Выполнение задачи: C:\Users\NIKITA\AppData\Local\Microsoft\WindowsApps\python3.11.exe 'c:\Users\NIKITA\Desktop\Папочки\Лабы\КРИПТО\Лабы Тема 2\Лаб 2_1\21-23.RSA_Elgamal_ECC.py'
------------------------------
12 ->    12     -> Л
18 ->     6     -> Е
9 ->     15     -> О
4 ->     16     -> П
1 ->      1     -> А
29 ->    17     -> Р
26 ->     5     -> Д
4 ->     16     -> П
29 ->    17     -> Р
8 ->      2     -> Б
5 ->     14     -> Н
18 ->     6     -> Е
4 ->     16     -> П
29 ->    17     -> Р
8 ->      2     -> Б
19 ->    13     -> М
9 ->     15     -> О
13 ->     7     -> Ж
18 ->     6     -> Е
28 ->    19     -> Т
4 ->     16     -> П
29 ->    17     -> Р
8 ->      2     -> Б
3 ->      9     -> И
17 ->     8     -> З
19 ->    13     -> М
18 ->     6     -> Е
5 ->     14     -> Н
3 ->      9     -> И
28 ->    19     -> Т
2 ->     29     -> Ь
4 ->     16     -> П
29 ->    17     -> Р
8 ->      2     -> Б
24 ->    18     -> С
27 ->     3     -> В
9 ->     15     -> О
3 ->      9     -> И
22 ->    22     -> Х
4 ->     16     -> П
29 ->    17     -> Р
8 ->      2     -> Б
4 ->     16     -> П
32 ->    32     -> Я
28 ->    19     -> Т
18 ->     6     -> Е
5 ->     14     -> Н

РАСШИФРОВАННЫЙ ТЕКСТ
ЛЕОПАРД НЕ МОЖЕТ ИЗМЕНИТЬ СВОИХ ПЯТЕН
```

Рисунок 33. Результат контрольного запуска RSA, вариант 20.

Результат шифра в тетради:

23 Шифр RSA

$E = 3$, $p = 3$, $q = 11$

$n = p \cdot q = 3 \cdot 11 = 33$

$\varphi(n) = 2 \cdot 10 = 20$

$3d \equiv 1 \pmod{20}$

$d = 7$.

Наш текст: 11, 17, 1, 18, 9, 3, 28, 13, 9, 18, 12, 15, 3, 1, 13, 9, 16, 1, 18, 19, 6, 17, 14, 1, 11, 18, 6, 16, 15, 13, 1, 18, 12, 9, 25, 29, 19, 24, 11.

$C_i = M_i^e \bmod n$

$11^3 \bmod 33 = 11$

$17^3 \bmod 33 = 29$

$1^3 \bmod 33 = 1$

$18^3 \bmod 33 = 24$

$9^3 \bmod 33 = 3$

$3^3 \bmod 33 = 27$

$28^3 \bmod 33 = 7$

$13^3 \bmod 33 = 19$

$12^3 \bmod 33 = 12$

$15^3 \bmod 33 = 9$

$3^3 \bmod 33 = 27$

$1^3 \bmod 33 = 1$

$13^3 \bmod 33 = 19$

$9^3 \bmod 33 = 3$

$16^3 \bmod 33 = 4$

$1^3 \bmod 33 = 1$

Рисунок 34. Шифр RSA в тетради.

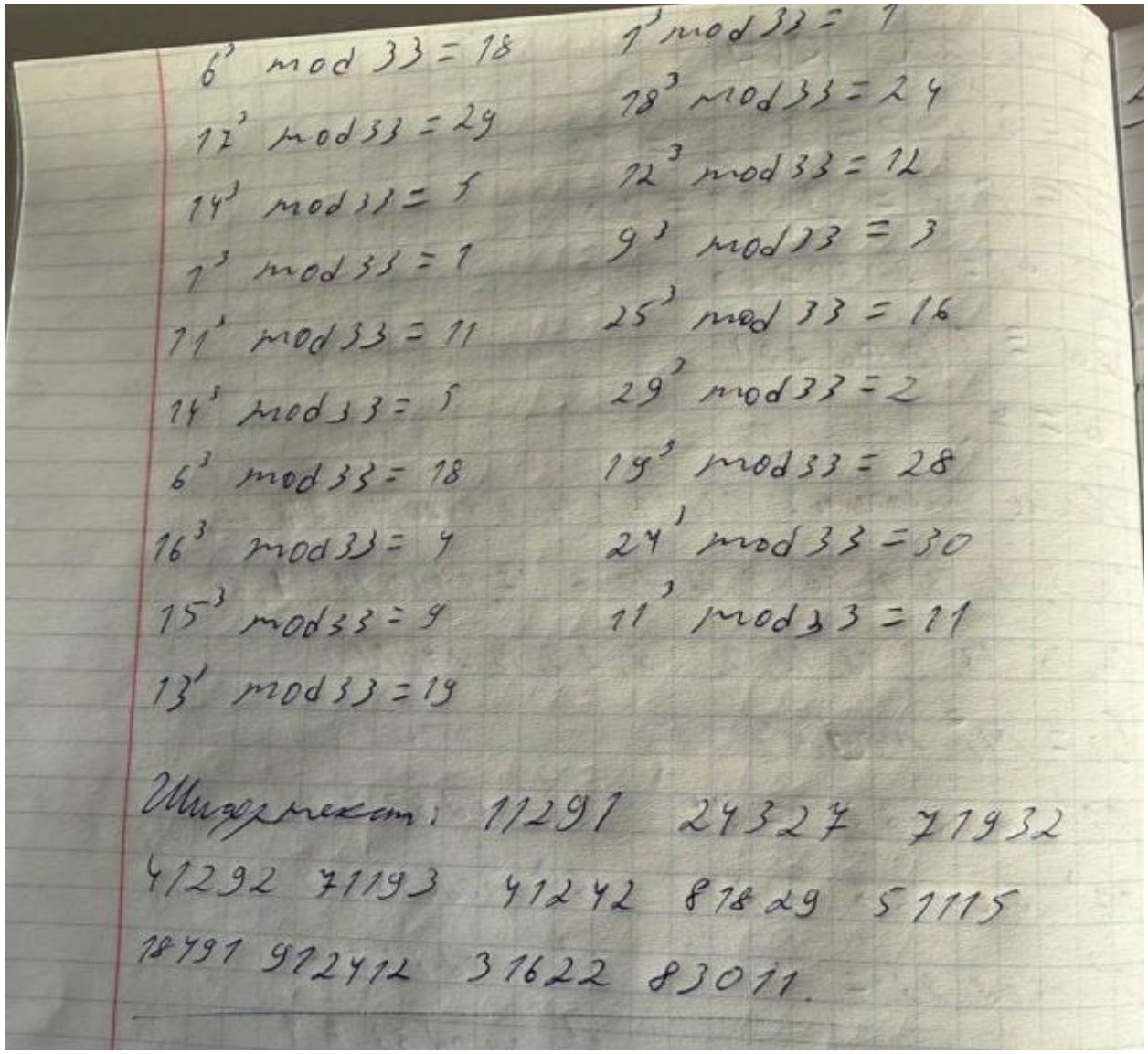


Рисунок 35. Шифр RSA в тетради (2).

Блок I: алгоритмы цифровых подписей

24. Цифровая подпись RSA

RSA - первый алгоритм цифровой подписи, который был разработан в 1977 году в Массачусетском технологическом институт и назван по первым буквам фамилий ее разработчиков (Ronald Rivest, Adi Shamir и Leonard Adleman). RSA основывается на сложности разложения большого числа n на простые множители.

1. Берутся два очень больших простых числа P и Q и находятся произведение простых чисел $N=P\times Q$ и функция Эйлера от этого произведения $\varphi(N)=(P-1)\times(Q-1)$.

2. Выбирается случайное целое число E: $1 < E < \varphi(N)$ и взаимно простое с $\varphi(N)$, и вычисляется D = 1 MOD $\varphi(N))/E$.

3. Потом E и N публикуются как открытый ключ, D сохраняется в тайне.

4. Если M — сообщение, а h(M) = m – хеш-код сообщения, длина которого должна быть в интервале (1,N), то электронная цифровая подпись S получается шифрованием хеш-кода сообщения m возведением в степень D по модулю N: S = mD MOD N.

5. Получателю отправляется сообщение M и подпись S.

6. Получатель сообщения хеширует M, получает хеш-код m’.

7. Проверяет подпись S: расшифровывает хеш-код, возведя S в степень E (число E ему известно) по модулю N: m = SE MOD N.

8. Сравнивает m и m’: если m = m’ - подпись верна.

Блок – схема алгоритма:

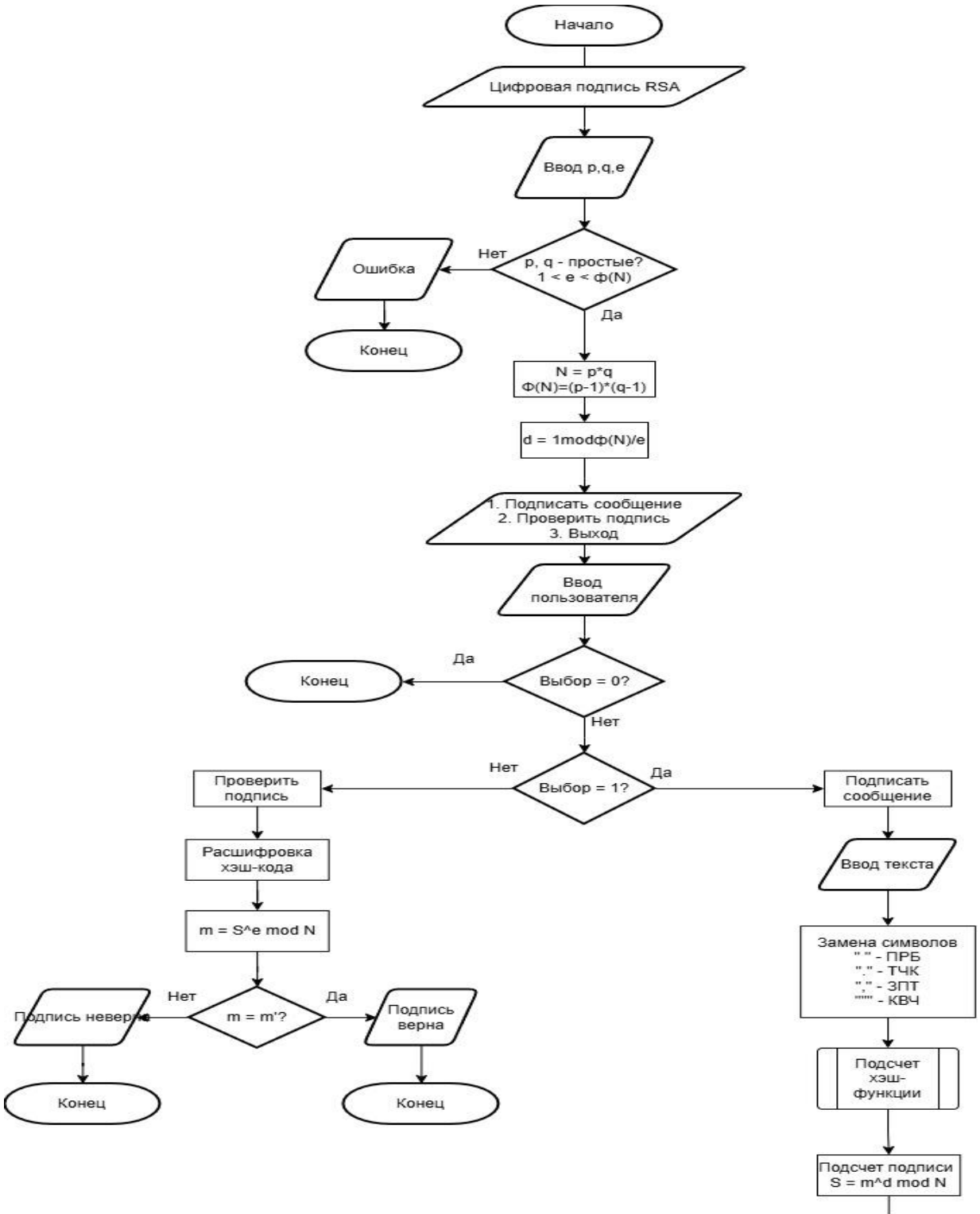

Начало
Цифровая подпись RSA
Ввод p,q,e
p, q - простые?
1 < e < ф(N)
Нет
Ошибка
Конец
Да
N = p*q
Ф(N)=(p-1)*(q-1)
d = 1modф(N)/e
1. Подписать сообщение
2. Проверить подпись
3. Выход
Ввод пользователя
Выбор = 0?
Да
Конец
Нет
Выбор = 1?
Нет
Да
Проверить подпись
Расшифровка хэш-кода
m = S^e mod N
m = m'?
Нет
Да
Подпись неверна
Конец
Подпись верна
Конец
Подписать сообщение
Ввод текста
Замена символов
" " - ПРБ
"." - ТЧК
"," - ЗПТ
""" - КВЧ
Подсчет хэш-функции
Подсчет подписи
S = m^d mod N

Рисунок 37. Блок-схема цифровой подписи RSA.

Код программы:

```
import random

def is_prime(n: int, k: int = 10) -> bool:
    if n < 2:
        return False
    if n in (2, 3):
        return True
    if n % 2 == 0:
        return False
    r, d = 0, n - 1
    while d % 2 == 0:
        r += 1
        d //= 2
    for _ in range(k):
        a = random.randint(2, n - 2)
        x = pow(a, d, n)
        if x == 1 or x == n - 1:
            continue
        for _ in range(r - 1):
            x = pow(x, 2, n)
            if x == n - 1:
                break
        else:
            return False
    return True

def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def mod_inverse(a, m):
    m0, x0, x1 = m, 0, 1
    if m == 1:
        return 0
    while a > 1:
        q = a // m
        a, m = m, a % m
        x0, x1 = x1 - q * x0, x0
    return x1 + m0 if x1 < 0 else x1

def prepare_text(message: str) -> str:
    result = ""
    for ch in message.upper():
        if ch == ' ':
            result += "ПРБ"
        elif ch == '.':
            result += "ТЧК"
        else:
            result += ch
    return result

def square_hash(message: str, N: int) -> int:
    alphabet = "АБВГДЕЖЗИЙКЛМНОПРСТУФХЦЧШЩЪЫЬЭЮЯ"
    h = 0
```

```
    print(f"Начальное значение h0 = {h}")
    for ch in message:
        if ch in alphabet:
            code = alphabet.index(ch) + 1
        else:
            continue
        h = (h + code) % N
        h = (h * h) % N
    print(f"Итоговый хеш: {h}")
    return h

class RSASignature:
    def __init__(self):
        self.p = None
        self.q = None
        self.N = None
        self.phi = None
        self.e = None
        self.d = None
        self.keys_ready = False
        self.last_message = ""
        self.last_hash = 0
        self.last_signature = 0

    def set_keys(self, p, q, e):
        if not is_prime(p):
            raise ValueError("p не является простым числом")
        if not is_prime(q):
            raise ValueError("q не является простым числом")
        if p == q:
            raise ValueError("p и q должны быть разными")
        self.p = p
        self.q = q
        self.N = p * q
        self.phi = (p - 1) * (q - 1)

        if not (1 < e < self.phi):
            raise ValueError("e должен быть в диапазоне 1 < e < ϕ(N)")
        if gcd(e, self.phi) != 1:
            raise ValueError("e и ϕ(N) не взаимно просты")
        self.e = e

        self.d = mod_inverse(self.e, self.phi)

        if self.e == self.d:
            raise ValueError("Открытый и секретный ключи совпадают")
        if self.N <= 32:
            raise ValueError(f"Модуль N = {self.N} должен быть больше 32")

        self.keys_ready = True
        print("\nКлючи установлены")
        print(f"p = {p}, q = {q}")
        print(f"N = {self.N}")
        print(f"ϕ(N) = {self.phi}")
        print(f"Открытый ключ e = {e}")
        print(f"Секретный ключ d = {self.d}")

    def sign(self, message):
        if not self.keys_ready:
            raise RuntimeError("Ключи не установлены")
```

```
        prepared = prepare_text(message)
        h = square_hash(prepared, self.N)
        signature = pow(h, self.d, self.N)
        self.last_message = message
        self.last_hash = h
        self.last_signature = signature
        print(f"\nПодпись = {h}^{self.d} mod {self.N} = {signature}")
        return signature

    def verify(self):
        if not self.keys_ready:
            raise RuntimeError("Ключи не установлены")
        if not self.last_message:
            raise RuntimeError("Нет подписанного сообщения")
        prepared = prepare_text(self.last_message)
        h = square_hash(prepared, self.N)
        decrypted = pow(self.last_signature, self.e, self.N)
        print(f"\nПроверка подписи")
        print(f"Сообщение: {self.last_message}")
        print(f"Хэш сообщения (число): {h}")
        print(f"Подпись: {self.last_signature}")
        print(f"Вычисляем (подпись)^e mod N = {decrypted}")
        print(f"Сравниваем с хэшем: {h} == {decrypted} => {h == decrypted}")
        return h == decrypted

def main():
    rsa = RSASignature()
    while True:
        print("\nЦИФРОВАЯ ПОДПИСЬ RSA")
        print("1. Ввести ключи (p, q, e)")
        print("2. Подписать сообщение")
        print("3. Проверить подпись")
        print("4. Выход")
        choice = input("Ваш выбор: ").strip()

        if choice == '1':
            try:
                p = int(input("Введите простое число p: "))
                q = int(input("Введите простое число q: "))
                e = int(input("Введите открытый ключ e: "))
                rsa.set_keys(p, q, e)
            except Exception as ex:
                print(f"Ошибка: {ex}")

        elif choice == '2':
            if not rsa.keys_ready:
                print("Сначала установите ключи (пункт 1).")
            else:
                msg = input("Введите сообщение для подписи: ")
                try:
                    rsa.sign(msg)
                except Exception as ex:
                    print(f"Ошибка: {ex}")

        elif choice == '3':
            if not rsa.keys_ready:
                print("Сначала установите ключи (пункт 1).")
            elif not rsa.last_message:
                print("Сначала подпишите сообщение (пункт 2).")
            else:
```

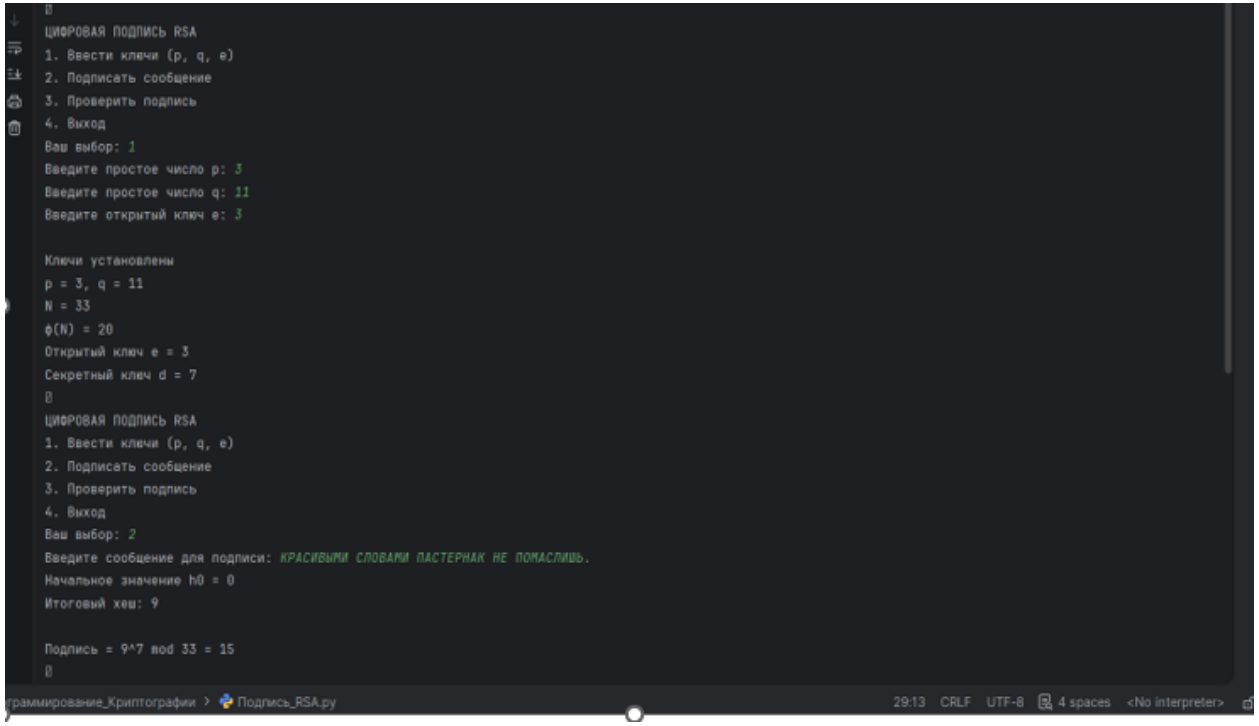
```
                try:
                    valid = rsa.verify()
                    if valid:
                        print("\nПОДПИСЬ ВЕРНА")
                    else:
                        print("\nПОДПИСЬ НЕВЕРНА")
                except Exception as ex:
                    print(f"Ошибка: {ex}")

            elif choice == '4':
                print("Программа завершена.")
                break
            else:
                print("Неверный выбор")

            input("\nНажмите Enter, чтобы вернуться в меню...")

    if __name__ == "__main__":
        main()
```

Результат работы программы:



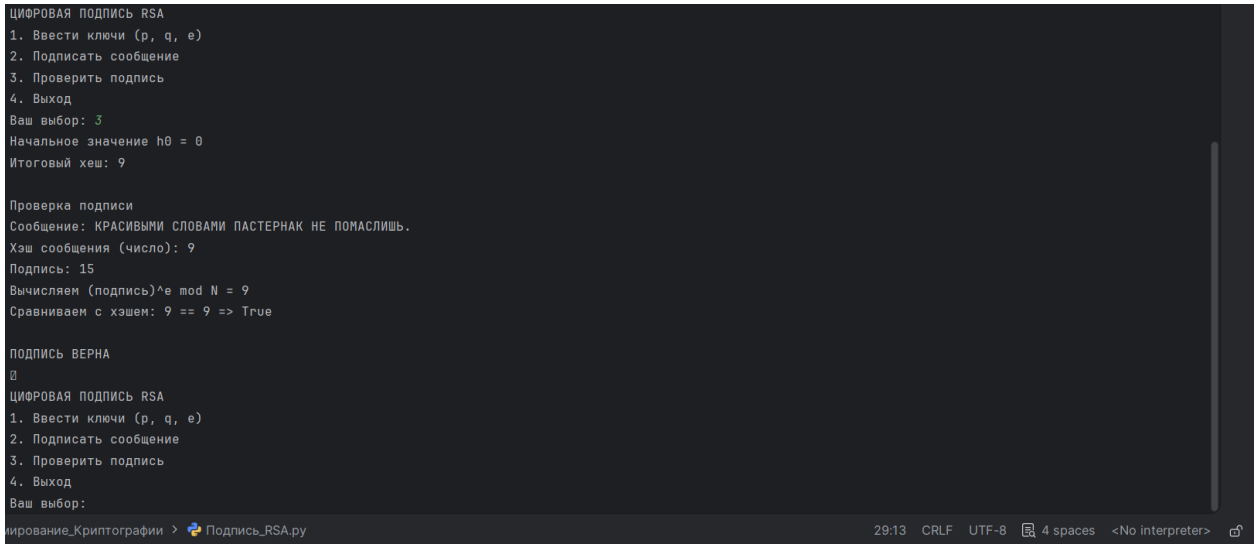


Рисунок 38. Результат подписи RSA.

Результат подписи в тетради:

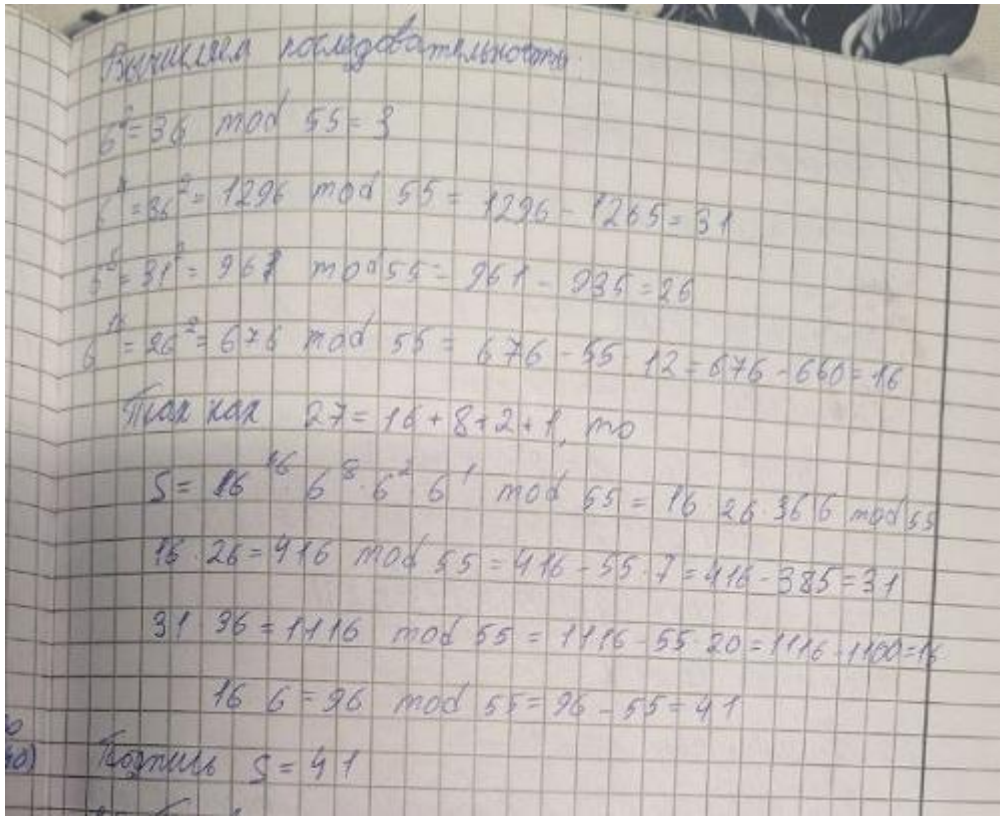


Рисунок 39. Работа в тетради подписи RSA.

Блок J: Стандарты цифровых подписей

27. Цифровая подпись ГОСТ Р 34.10-2012

Алгоритм реализует формирование и проверку электронной цифровой подписи на эллиптической кривой. В программе параметры кривой вводятся пользователем; точка G проверяется, а её порядок q вычисляется по точке.

Открытый ключ Y_A=x_A·G. Для сообщения вычисляется хеш e по учебному правилу из методички, после чего по случайному k формируется подпись (r,s). При проверке вычисляется точка C′ и сравниваются r′ и r.

Параметры контрольного примера: p=11, a=b=1, G=(0,1), q=7. Для основного запуска программа позволяет задать параметры вручную.

Блок-схема:

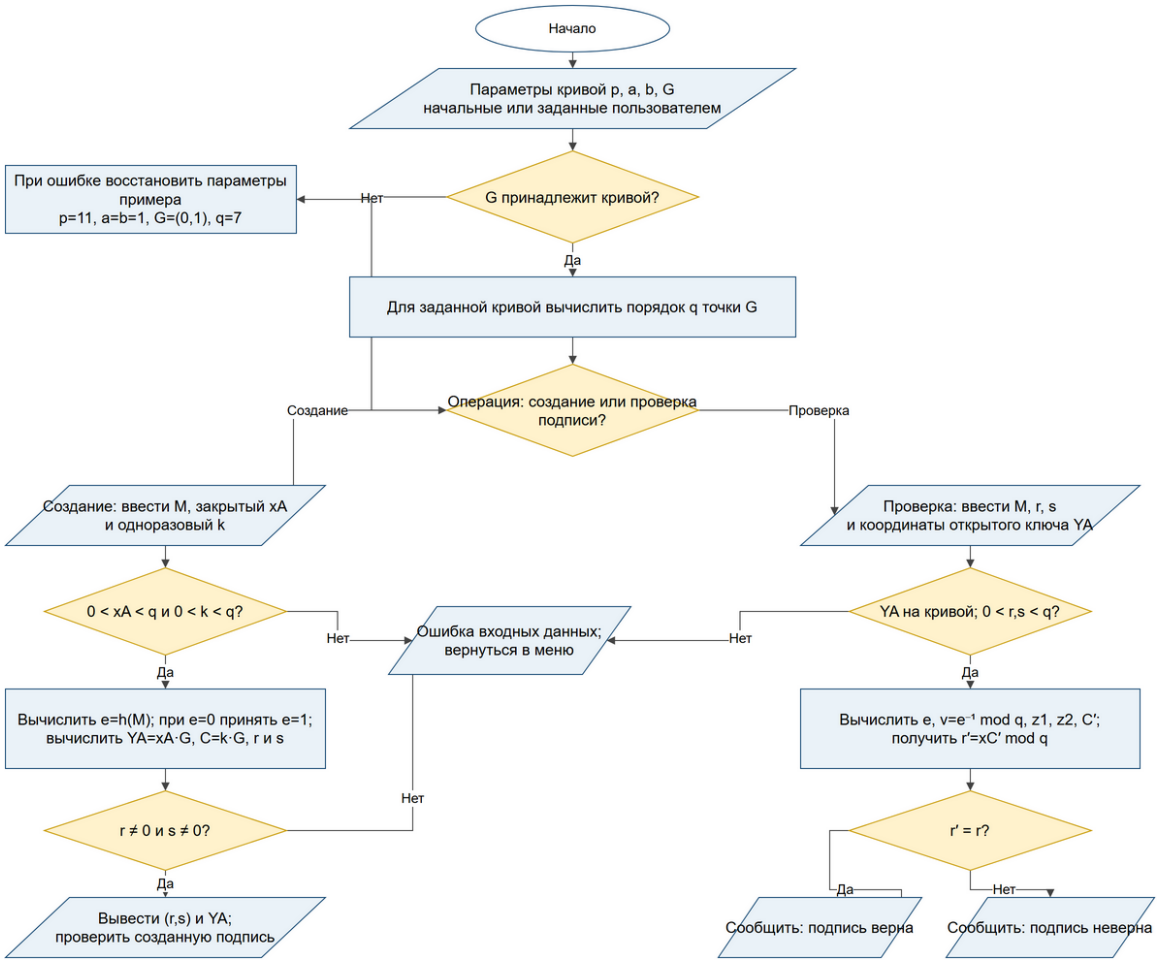


Рисунок 40. Блок-схема цифровой подписи ГОСТ Р 34.10-2012.

Код программы:

"""

Лабораторная работа 3_2

Алгоритм 27: ЭЦП по ГОСТ Р 34.10-2012

Вариант: "Леопард не может изменить своих пятен"

Источник: ЛАБКРИПТ_2022, Блок J, стр. 37-39

Задание: "PDF/Задание по ГОСТ Р 34.10-2012..pdf"

Параметры из методички (пример):

Кривая $E_p(a,b)$: $y^2 \equiv x^3 + x + 1 \pmod{11}$

$a = 1, b = 1, p = 11$

Порядок подгруппы $q = 7$

Генератор $G = (0, 1)$

Хэш-функция квадратичной свёртки ($h_0 = 0$):

$h_i = (h_{i-1} + M_i)^2 \bmod q$

где M_i — коды (ord) символов сообщения

Подпись (отправитель):

1. Выбрать секретный ключ $0 < x_A < q$
2. Вычислить $Y_A = x_A * G$ (открытый ключ)
3. Вычислить $e = h(\text{сообщение}) \bmod q$, если $e=0 \rightarrow e=1$
4. Выбрать случайное $0 < k < q$
5. Вычислить $C = k * G$, $r = x_C \bmod q$
6. $s = (k * e + r * x_A) \bmod q$
7. Подпись: (r, s)

Проверка (получатель):

1. Вычислить e = h(сообщение) mod q, если e=0 → e=1
2. Проверить 0 < r < q и 0 < s < q
3. $v = e^{-1} \bmod q$
4. $z_1 = s * v \bmod q$
5. $z_2 = -r * v \bmod q$
6. $C' = z_1*G + z_2*Y_A$
7. r' = x_{C'} mod q; если r' = r — подпись верна

"""

```
import random
from math import isqrt


# ──────────────────────────────────────────────────────────────
#  Эллиптическая кривая
# ──────────────────────────────────────────────────────────────


class EllipticCurve:
    def __init__(self, p, a, b):
        self.p = p
        self.a = a
        self.b = b
```

```
def is_on_curve(self, point):
    if point is None:
        return True
    x, y = point
    return (y * y - x ** 3 - self.a * x - self.b) % self.p == 0

def neg(self, P):
    if P is None:
        return None
    return P[0], (-P[1]) % self.p

def add(self, P, Q):
    if P is None:
        return Q
    if Q is None:
        return P
    x1, y1 = P
    x2, y2 = Q
    p = self.p
    if x1 == x2:
        if (y1 + y2) % p == 0:
            return None  # точка на бесконечности
```

```
            lam = (3 * x1 * x1 + self.a) * pow(2 * y1, p - 2, p) % p
        else:
            lam = (y2 - y1) * pow(x2 - x1, p - 2, p) % p
        x3 = (lam * lam - x1 - x2) % p
        y3 = (lam * (x1 - x3) - y1) % p
        return x3, y3

    def mul(self, k, P):
        """Скалярное умножение методом двоичного разложения."""
        result = None
        addend = P
        while k > 0:
            if k & 1:
                result = self.add(result, addend)
            addend = self.add(addend, addend)
            k >>= 1
        return result


def point_order(curve, P):
    """Вычисляет порядок точки P перебором кратных P.
```

```
    Для кривой над простым полем порядок группы точек не превосходит
    p + 1 + 2*sqrt(p) (граница Хассе), поэтому перебор имеет верхнюю границу.
    """
    if curve.p <= 1:
        raise ValueError("Модуль p должен быть больше 1.")
    if P is None or not curve.is_on_curve(P):
        raise ValueError("Точка G должна лежать на заданной кривой.")

    max_order = curve.p + 1 + isqrt(4 * curve.p)
    multiple = None
    for order in range(1, max_order + 1):
        multiple = curve.add(multiple, P)
        if multiple is None:
            return order
    raise ValueError("Не удалось вычислить порядок G; проверьте параметры кривой.")


# ──────────────────────────────────────────────────────────────
#  Хэш-функция квадратичной свёртки (ЛАБКРИПТ_2022, стр. 1)
#  h_0 = 0
#  h_i = (h_{i-1} + M_i)² mod p
# ──────────────────────────────────────────────────────────────
```

```
def hash_func(message: str, q: int) -> int:
    """
    Упрощённая хэш-функция квадратичной свёртки по методичке:
      h_0 = 0
      h_i = (h_{i-1} + ord(M_i))² mod q
    Если h = 0 — возвращаем 1 (защита от нулевого хэша).
    """
    h = 0
    for ch in message:
        h = (h + ord(ch)) ** 2 % q
    return h if h != 0 else 1



# ——————————————————————————————————————————————
#  Генерация ключей
# ——————————————————————————————————————————————

def keygen(curve, P, q):
    d = random.randint(1, q - 1)   # закрытый ключ
    Q = curve.mul(d, P)            # открытый ключ Q = d·P
    return d, Q
```

```
# ──────────────────────────────────────────────────────────────
#  Создание подписи
# ──────────────────────────────────────────────────────────────

def sign(message, curve, P, q, d, k=None):
    e = hash_func(message, q)
    if not 0 < d < q:
        raise ValueError(f"Закрытый ключ d должен быть в диапазоне 1..{q - 1}.")
    if k is not None:
        if not 0 < k < q:
            raise ValueError(f"Одноразовый ключ k должен быть в диапазоне 1..{q - 1}.")
        C = curve.mul(k, P)
        if C is None:
            raise ValueError("Выбранный k дал точку на бесконечности; введите другое k.")
        r = C[0] % q
        s = (k * e + r * d) % q
        if r == 0 or s == 0:
            raise ValueError("Для выбранного k получено r=0 или s=0; введите другое k.")
        return r, s, e, k
    while True:
```

```
        k = random.randint(1, q - 1)
        C = curve.mul(k, P)
        if C is None:
            continue
        r = C[0] % q
        if r == 0:
            continue
        s = (k * e + r * d) % q
        if s == 0:
            continue
        return r, s, e, k


# ──────────────────────────────────────────────────────────────
#  Проверка подписи
# ──────────────────────────────────────────────────────────────

def verify(message, r, s, curve, P, Q_pub, q):
    if not (0 < r < q and 0 < s < q):
        return False
    e = hash_func(message, q)
    v = pow(e, q - 2, q)       # e^{-1} mod q
```

```
    z1 = s * v % q
    z2 = (-r * v) % q
    C = curve.add(curve.mul(z1, P), curve.mul(z2, Q_pub))
    if C is None:
        return False
    return C[0] % q == r


# ──────────────────────────────────────────────────────────────
#  Главное меню
# ──────────────────────────────────────────────────────────────

# Параметры по умолчанию из примера методички (ЛАБКРИПТ_2022, стр. 37)
#   E_p(a,b): y² ≡ x³ + x + 1 (mod 11)
#   a=1, b=1, p=11, q=7, G=(0,1)
DEFAULT_P = 11
DEFAULT_A = 1
DEFAULT_B = 1
DEFAULT_G = (0, 1)
DEFAULT_Q = 7
```

```
def print_params(curve, G, q):
    print(f"  Кривая     : y² ≡ x³ + {curve.a}x + {curve.b}  (mod {curve.p})")
    print(f"  Генератор  : G = {G}")
    print(f"  Порядок    : q = {q}")



def main():
    print("=" * 60)
    print("ГОСТ Р 34.10-2012  —  Цифровая подпись (эллиптические кривые)")
    print("=" * 60)
    print("Параметры из методички (ЛАБКРИПТ_2022, Блок J, задание 27):")
    print(f"  Кривая E_p(a,b): y² ≡ x³ + ax + b  (mod p)")
    print(f"  Хэш: h_0=0, h_i=(h_{{i-1}}+ord(M_i))² mod q")
    print()

    # Загружаем параметры по умолчанию
    curve = EllipticCurve(p=DEFAULT_P, a=DEFAULT_A, b=DEFAULT_B)
    P_BASE = DEFAULT_G
    Q_ORDER = DEFAULT_Q

    while True:
        print("\n" + "-" * 60)
```

```
print("Текущие параметры кривой:")
print_params(curve, P_BASE, Q_ORDER)
print("-" * 60)
print("1 — Задать параметры кривой вручную")
print("2 — Создать подпись")
print("3 — Проверить подпись")
print("4 — Показать пример из методички")
print("0 — Выход")

choice = input("Ваш выбор: ").strip()

if choice == "0":
    break

try:
    if choice == "1":
        print(f"\nВведите параметры кривой y² ≡ x³ + ax + b (mod p)")
        p = int(input("  p (простое): "))
        a = int(input("  a: "))
        b = int(input("  b: "))
        gx = int(input("  Gx (x-координата генератора): "))
        gy = int(input("  Gy (y-координата генератора): "))
```

```
    curve = EllipticCurve(p=p, a=a, b=b)
    P_BASE = (gx, gy)
    if not curve.is_on_curve(P_BASE):
        print("  ОШИБКА: точка G не лежит на заданной кривой!")
        # Возвращаем умолчания
        curve = EllipticCurve(p=DEFAULT_P, a=DEFAULT_A, b=DEFAULT_B)
        P_BASE = DEFAULT_G
        Q_ORDER = DEFAULT_Q
    else:
        Q_ORDER = point_order(curve, P_BASE)
        print(f"  Порядок точки G вычислен автоматически: q = {Q_ORDER}.")
        print("  Параметры установлены.")

elif choice == "2":
    print("\n--- Создание подписи ---")
    message = input("Введите сообщение: ")
    if not message:
        raise ValueError("Сообщение не должно быть пустым.")
    d = int(input(f"Введите закрытый ключ x_A (1..{Q_ORDER - 1}): "))
    k = int(input(f"Введите одноразовый ключ k (1..{Q_ORDER - 1}): "))
    if not (0 < d < Q_ORDER):
        raise ValueError(f"x_A должен быть в диапазоне 1..{Q_ORDER - 1}.")
```

```
    Q_pub = curve.mul(d, P_BASE)
    r, s, e, _ = sign(message, curve, P_BASE, Q_ORDER, d, k)
    print(f"\nХэш e = {e}")
    print(f"Открытый ключ Y_A = {Q_pub}")
    print(f"Подпись (r, s) = ({r}, {s})")
    ok = verify(message, r, s, curve, P_BASE, Q_pub, Q_ORDER)
    print(f"Проверка созданной подписи: {'ВЕРНА ✓' if ok else 'НЕ ВЕРНА ✗'}")

elif choice == "3":
    print("\n--- Проверка подписи ---")
    message = input("Введите исходное сообщение: ")
    r = int(input("Введите r: "))
    s = int(input("Введите s: "))
    qx = int(input("Введите координату Y_Ax: "))
    qy = int(input("Введите координату Y_Ay: "))
    Q_pub = (qx, qy)
    if not curve.is_on_curve(Q_pub):
        raise ValueError("Открытый ключ Y_A не принадлежит заданной эллиптической кривой.")
    valid = verify(message, r, s, curve, P_BASE, Q_pub, Q_ORDER)
    print(f"Результат проверки: {'ПОДПИСЬ ВЕРНА ✓' if valid else 'ПОДПИСЬ НЕВЕРНА ✗'}")

elif choice == "4":
```

```
print("\n" + "=" * 60)
print("ПРИМЕР ИЗ МЕТОДИЧКИ (ЛАБКРИПТ_2022, стр. 37)")
print("=" * 60)
print("Кривая E_p(a,b): y² ≡ x³ + x + 1 (mod 11)")
print("a = 1, b = 1, p = 11, q = 7, G = (0, 1)")
print()
demo_curve = EllipticCurve(p=11, a=1, b=1)
demo_G = (0, 1)
demo_q = 7
# Пример из PDF: x_A=4, k=5
demo_xA = 4
demo_k = 5
demo_msg = "ДЕВА"  # Из таблицы вариантов для примера
# Считаем хэш
demo_e = hash_func(demo_msg, demo_q)
demo_YA = demo_curve.mul(demo_xA, demo_G)
demo_C = demo_curve.mul(demo_k, demo_G)
demo_r = demo_C[0] % demo_q
demo_s = (demo_k * demo_e + demo_r * demo_xA) % demo_q
print(f"Сообщение   : {demo_msg!r}")
print(f"x_A (секрет): {demo_xA}")
print(f"k (разовый) : {demo_k}")
```

```
            print(f"Хэш e       : {demo_e}")
            print(f"Y_A = {demo_xA}·G = {demo_YA}")
            print(f"C = {demo_k}·G = {demo_C}")
            print(f"r = {demo_r}, s = {demo_s}")
            print(f"Подпись (r, s) = ({demo_r}, {demo_s})")
            ok = verify(demo_msg, demo_r, demo_s, demo_curve, demo_G, demo_YA, demo_q)
            print(f"Проверка: {'ВЕРНА ✓' if ok else 'НЕ ВЕРНА ✗'}")
            print("=" * 60)

        else:
            print("Выберите 0, 1, 2, 3 или 4.")

    except ValueError as exc:
        print(f"Ошибка ввода: {exc}")
    except Exception as exc:
        print(f"Ошибка: {exc}")


if __name__ == "__main__":
    main()
```

Контрольный пример по параметрам методички: кривая $y^2=x^3+x+1 \pmod{11}$, $G=(0,1)$, $q=7$. Программа вычисляет порядок G,

формирует подпись для сообщения и проверяет её по открытому ключу.

Тест на длинном сообщении: сформирована строка длиной 1000 символов из текста варианта 20; подпись проверена программой. В результате сравнения получено «подпись верна».

Ручной тест: в отчёте следует приложить фото расчётов на карточке задания с параметрами варианта. Автоматическая трассировка не подменяет это фото.

Блок К: обмен ключами

28. Обмен ключами по алгоритму Диффи-Хеллмана

Протокол Диффи-Хеллмана — это криптографический протокол, позволяющий двум и более сторонам получить общий секретный ключ по незащищённому каналу связи. Протокол был предложен Уитфилдом Диффи и Мартином Хеллманом в 1976 году и стал первым практическим методом асимметричного распределения ключей.

Ключевая особенность: Протокол не выполняет шифрование напрямую, а позволяет сторонам договориться о секретном ключе, который затем можно использовать для симметричного шифрования. Алгоритм основан на вычислительной сложности дискретного логарифмирования.

Блок-схема алгоритма:

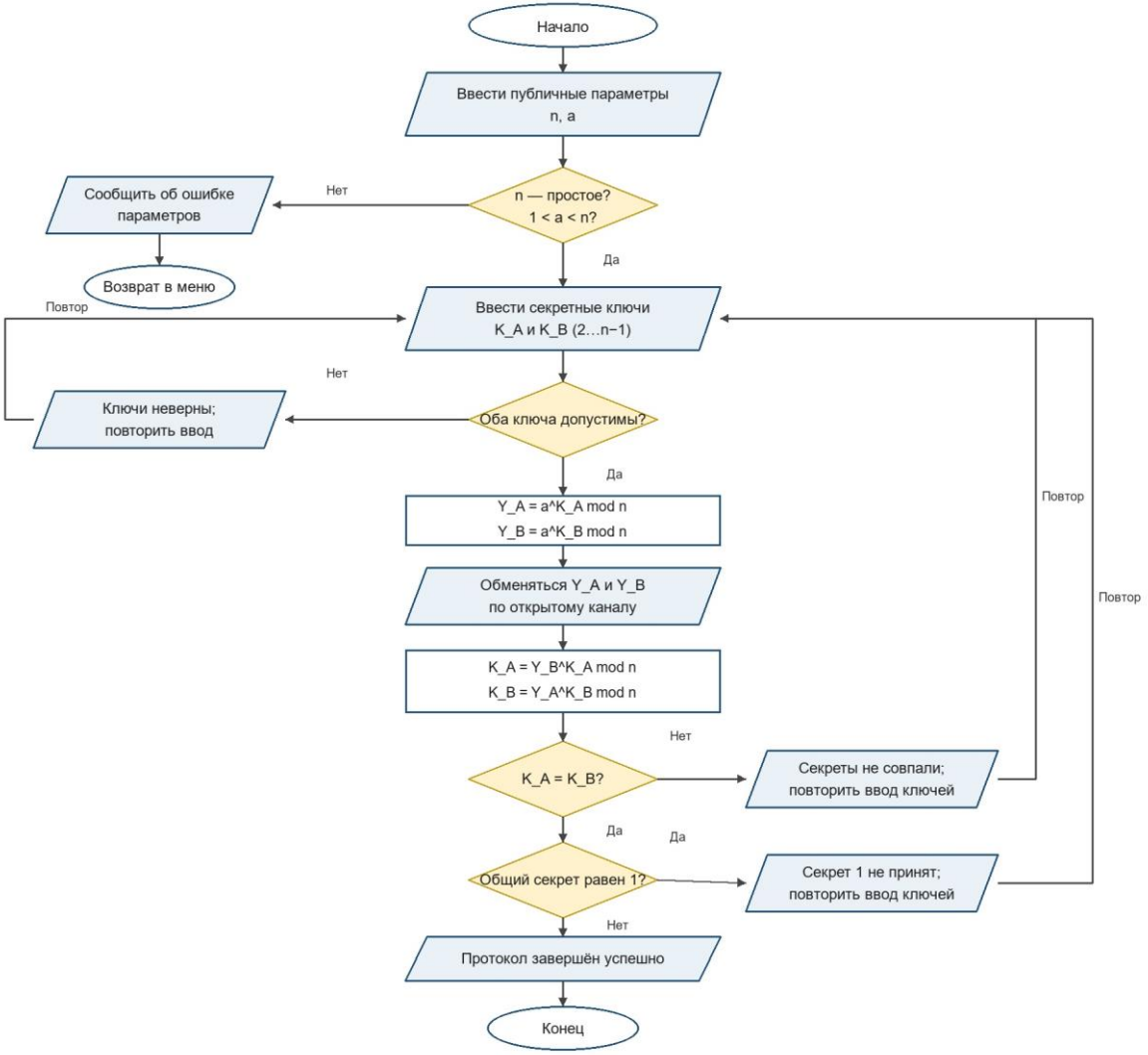


Рисунок 45. Блок-схема алгоритма Диффи-Хеллмана.

Код программы:

```
# ============================================================
# БЛОК К: ОБМЕН КЛЮЧАМИ
# Лаб 2_2 (задание 28) — Обмен ключами по Диффи-Хеллману
# Источник: ЛАБКРИПТ_2022, стр. 40
# ============================================================
#
# АЛГОРИТМ (по методичке):
#   Публичные параметры: n — большое простое, a (1 < a < n)
#   Секретные ключи:     K_A, K_B — случайные числа из [2, n-1]
#   Открытые ключи:      Y_A = a^K_A mod n,  Y_B = a^K_B mod n
#   Обмен:               Y_A и Y_B передаются по открытому каналу
#   Общий секрет:        K_A: K = Y_B^K_A mod n
#                        K_B: K = Y_A^K_B mod n
#   Проверка:            K_A == K_B (оба должны совпадать)
#
# ДОКАЗАТЕЛЬСТВО:
#   A: Y_B^K_A mod n = (a^K_B)^K_A mod n = a^(K_B*K_A) mod n = K
#   B: Y_A^K_B mod n = (a^K_A)^K_B mod n = a^(K_A*K_B) mod n = K
# ============================================================
import random
import sys

# ──────────────────────────────────────────────
#  Математические вспомогательные функции
# ──────────────────────────────────────────────
def mod_pow(base, exp, mod):
    """Быстрое возведение в степень по модулю (алгоритм «двоичного возведения»)."""
    result = 1
    base %= mod
    while exp > 0:
        if exp & 1:
            result = (result * base) % mod
        base = (base * base) % mod
        exp >>= 1
    return result

def is_prime(n):
    """Детерминированная проверка простоты методом пробных делений."""
    if n < 2:
        return False
    if n == 2:
        return True
    if n % 2 == 0:
        return False
    i = 3
    while i * i <= n:
        if n % i == 0:
            return False
        i += 2
    return True

def miller_rabin(n, k=10):
    """Тест Миллера–Рабина для больших чисел (вероятностный, k раундов)."""
    if n < 2:
        return False
    if n == 2 or n == 3:
        return True
    if n % 2 == 0:
```

```
        return False
    # Представляем n-1 = 2^r * d
    r, d = 0, n - 1
    while d % 2 == 0:
        r += 1
        d //= 2
    for _ in range(k):
        a = random.randrange(2, n - 1)
        x = mod_pow(a, d, n)
        if x == 1 or x == n - 1:
            continue
        for _ in range(r - 1):
            x = (x * x) % n
            if x == n - 1:
                break
        else:
            return False
    return True

def generate_prime(bits=512):
    """Генерация случайного простого числа заданной битности (тест Миллера–Рабина)."""
    while True:
        candidate = random.getrandbits(bits) | (1 << (bits - 1)) | 1  # нечётное, старший бит = 1
        if miller_rabin(candidate):
            return candidate

def find_primitive_root(p):
    """
    Находит первообразный корень (примитивный корень) по модулю простого p.
    Для простого p φ(p) = p-1. Перебираем кандидатов от 2.
    """
    phi = p - 1
    # Факторизация phi
    factors = set()
    temp = phi
    d = 2
    while d * d <= temp:
        if temp % d == 0:
            factors.add(d)
            while temp % d == 0:
                temp //= d
        d += 1
    if temp > 1:
        factors.add(temp)
    # Проверяем кандидатов
    for g in range(2, p):
        ok = True
        for factor in factors:
            if mod_pow(g, phi // factor, p) == 1:
                ok = False
                break
        if ok:
            return g
    return -1  # не найден (не должно случиться для простого p)

# ──────────────────────────────────────────────
#  Класс протокола Диффи-Хеллмана
# ──────────────────────────────────────────────
class DiffieHellman:
    """
    Протокол обмена ключами Диффи–Хеллмана.
```

```
Публичные параметры:  n (простое), a (1 < a < n)
Секретные ключи:      K_A, K_B ∈ [2, n-1]
Открытые ключи:       Y_A = a^K_A mod n,  Y_B = a^K_B mod n
Общий секрет:         K = Y_B^K_A mod n  =  Y_A^K_B mod n
"""
def __init__(self):
    self.n = None      # публичный модуль (большое простое)
    self.a = None      # публичное основание a
    self.K_A = None    # секретный ключ пользователя A
    self.K_B = None    # секретный ключ виртуального пользователя B
    self.Y_A = None    # открытый ключ A
    self.Y_B = None    # открытый ключ B
    self.shared_K_A = None   # общий секрет, вычисленный стороной A
    self.shared_K_B = None   # общий секрет, вычисленный стороной B
# ── Установка публичных параметров ──────────────────────
def set_params(self, n, a):
    """Устанавливает публичные параметры n и a с проверками."""
    # Не используем пробное деление для больших n: проверяем простоту
    # тестом Миллера–Рабина с достаточным числом раундов.
    if not miller_rabin(n, k=32):
        print("  Ошибка: n должно быть простым числом!")
        return False
    if not (1 < a < n):
        print(f"  Ошибка: a должно быть в диапазоне (1 < a < n={n})!")
        return False
    self.n = n
    self.a = a
    # Сброс предыдущих ключей
    self.K_A = self.K_B = self.Y_A = self.Y_B = None
    self.shared_K_A = self.shared_K_B = None
    print(f"\n  Публичные параметры установлены:")
    print(f"    n = {n}")
    print(f"    a = {a}  (основание степени)")
    return True
def auto_generate_params(self, bits=24):
    """Генерирует учебные параметры; подбор корня ограничен 32 битами."""
    if not 4 <= bits <= 32:
        print("  Ошибка: учебная автогенерация поддерживает битность от 4 до 32.")
        return False
    print(f"\n  Генерация простого числа ({bits} бит)...")
    n = generate_prime(bits)
    print(f"  Найдено простое n = {n}")
    print(f"  Поиск примитивного корня по модулю n...")
    a = find_primitive_root(n)
    if a == -1:
        print("  Ошибка: не удалось найти примитивный корень.")
        return False
    print(f"  Найден примитивный корень a = {a}")
    self.n = n
    self.a = a
    self.K_A = self.K_B = self.Y_A = self.Y_B = None
    self.shared_K_A = self.shared_K_B = None
    return True
# ── Шаг 1: Выбор секретных ключей ──────────────────────
def set_secret_key_A(self, K_A):
    """Задаёт секретный ключ пользователя A."""
    if self.n is None:
        print("  Ошибка: сначала установите публичные параметры!")
```

```
            return False
        if not (2 <= K_A <= self.n - 1):
            print(f"  Ошибка: K_A должно быть в диапазоне [2, n-1 = {self.n - 1}]!")
            return False
        self.K_A = K_A
        self.Y_A = self.Y_B = None
        self.shared_K_A = self.shared_K_B = None
        print(f"  Секретный ключ A: K_A = {K_A}")
        return True

    def auto_secret_key_B(self):
        """Виртуальный пользователь B сам выбирает случайный секретный ключ."""
        if self.n is None:
            print("  Ошибка: сначала установите публичные параметры!")
            return False
        self.K_B = random.randint(2, self.n - 1)
        self.Y_A = self.Y_B = None
        self.shared_K_A = self.shared_K_B = None
        print(f"  Виртуальный пользователь B выбрал: K_B = {self.K_B}  (СЕКРЕТ — известен только B)")
        return True

    def set_secret_key_B(self, K_B):
        """Задаёт секретный ключ пользователя B, введённый вручную."""
        if self.n is None:
            print("  Ошибка: сначала установите публичные параметры!")
            return False
        if not (2 <= K_B <= self.n - 1):
            print(f"  Ошибка: K_B должно быть в диапазоне [2, n-1 = {self.n - 1}]!")
            return False
        self.K_B = K_B
        self.Y_A = self.Y_B = None
        self.shared_K_A = self.shared_K_B = None
        print(f"  Секретный ключ B: K_B = {K_B}")
        return True

    # ── Шаг 2: Вычисление открытых ключей ──────────────────

    def compute_public_keys(self):
        """
        Шаг 2 алгоритма:
          Y_A = a^K_A mod n
          Y_B = a^K_B mod n
        """
        if None in (self.n, self.a, self.K_A, self.K_B):
            print("  Ошибка: не все параметры установлены!")
            return False
        self.Y_A = mod_pow(self.a, self.K_A, self.n)
        self.Y_B = mod_pow(self.a, self.K_B, self.n)
        print(f"\n  Открытые ключи (передаются по открытому каналу):")
        print(f"    Y_A = a^K_A mod n = {self.a}^{self.K_A} mod {self.n} = {self.Y_A}")
        print(f"    Y_B = a^K_B mod n = {self.a}^{self.K_B} mod {self.n} = {self.Y_B}")
        return True

    # ── Шаг 3: Вычисление общего секрета ───────────────────

    def compute_shared_secret(self):
        """
        Шаг 3 алгоритма:
          K_A: K = Y_B^K_A mod n
          K_B: K = Y_A^K_B mod n
          Проверка: вычисленные сторонами значения K совпадают и не равны 1
        """
        if None in (self.Y_A, self.Y_B, self.K_A, self.K_B):
            print("  Ошибка: сначала вычислите открытые ключи!")
```

```
            return False
        self.shared_K_A = mod_pow(self.Y_B, self.K_A, self.n)
        self.shared_K_B = mod_pow(self.Y_A, self.K_B, self.n)
        print(f"\n  Вычисление общего секрета:")
        print(f"    A: K = Y_B^K_A mod n = {self.Y_B}^{self.K_A} mod {self.n} = {self.shared_K_A}")
        print(f"    B: K = Y_A^K_B mod n = {self.Y_A}^{self.K_B} mod {self.n} = {self.shared_K_B}")
        if self.shared_K_A == self.shared_K_B == 1:
            print("\n  ✗ ОШИБКА: общий секрет равен 1 и считается недопустимым.")
            print("    Выберите другие секретные ключи A и B.")
            self.shared_K_A = self.shared_K_B = None
            return False
        if self.shared_K_A == self.shared_K_B:
            print(f"\n  ✓ ПРОВЕРКА ПРОЙДЕНА: K у A = K у B = {self.shared_K_A}")
            print(f"    Общий секретный ключ успешно выработан!")
        else:
            print(f"\n  ✗ ОШИБКА: K у A ≠ K у B  ({self.shared_K_A} ≠ {self.shared_K_B})")
        return self.shared_K_A == self.shared_K_B

    # ── Полная демонстрация протокола ──────────────────────────
    def full_demo(self):
        """Полная пошаговая демонстрация протокола Диффи-Хеллмана."""
        sep = "─" * 60
        print(f"\n{sep}")
        print("  ПРОТОКОЛ ОБМЕНА КЛЮЧАМИ ДИФФИ-ХЕЛЛМАНА")
        print(f"  Источник: ЛАБКРИПТ_2022, Блок K, Задание 28")
        print(sep)
        print("\n  Шаг 1. Публичные параметры (известны всем):")
        print(f"    n = {self.n}  (простое число)")
        print(f"    a = {self.a}  (1 < a < n)")
        print("\n  Шаг 2. Выбор секретных ключей:")
        print(f"    Пользователь A выбирает K_A = {self.K_A}  (из интервала [2, n-1])")
        print(f"    Пользователь B выбирает K_B = {self.K_B}  (из интервала [2, n-1])")
        print("\n  Шаг 3. Вычисление открытых ключей:")
        self.compute_public_keys()
        print("\n  Шаг 4. Обмен открытыми ключами по открытому каналу:")
        print(f"    A → B: Y_A = {self.Y_A}")
        print(f"    B → A: Y_B = {self.Y_B}")
        print("\n  Шаг 5. Независимое вычисление общего секрета:")
        ok = self.compute_shared_secret()
        print(f"\n{sep}")
        if ok:
            print(f"  ИТОГ: Общий секретный ключ K = {self.shared_K_A}")
        else:
            print("  ИТОГ: Ошибка в протоколе!")
        print(sep)
        return ok

# ═══════════════════════════════════════════════
#  Меню
# ═══════════════════════════════════════════════
def print_menu(dh):
    """Выводит текущее меню с состоянием."""
    print("\n" + "=" * 60)
    print("  БЛОК K: ОБМЕН КЛЮЧАМИ (Задание 28 — Диффи-Хеллман)")
    print("=" * 60)
    # Текущее состояние параметров
    if dh.n:
        print(f"  n = {dh.n}  |  a = {dh.a}")
```

```
    else:
        print("  Параметры не установлены")
    if dh.K_A:
        print(f"  K_A = {dh.K_A}  |  K_B = {'установлен' if dh.K_B else 'не установлен'}")
    if dh.shared_K_A:
        print(f"  Общий ключ K = {dh.shared_K_A}")
    print()
    print("  1 — Ввести публичные параметры (n, a) вручную")
    print("  2 — Авто-генерация параметров (простое n и генератор a)")
    print("  3 — Ввести секретный ключ A (K_A) вручную")
    print("  4 — Ввести секретный ключ B (K_B) вручную")
    print("  5 — Вычислить открытые ключи Y_A и Y_B")
    print("  6 — Вычислить общий секрет и проверить протокол")
    print("  7 — Пошаговый обмен (параметры, ключи, проверка)")
    print("  8 — Пример из методички (малые числа)")
    print("  0 — Выход")
    print()

def demo_textbook(dh):
    """
    Демонстрация с малыми числами по образцу методички.
    Используем простое n=23, генератор a=5 — классический учебный пример.
    """
    print("\n" + "—" * 60)
    print("  ПРИМЕР ПО МЕТОДИЧКЕ (малые числа)")
    print("—" * 60)
    print("  Публичные параметры: n = 23 (простое), a = 5")
    print("  Пользователь A выбирает K_A = 6")
    print("  Пользователь B выбирает K_B = 15")
    dh.n = 23
    dh.a = 5
    dh.K_A = 6
    dh.K_B = 15
    dh.Y_A = None
    dh.Y_B = None
    dh.shared_K_A = None
    dh.shared_K_B = None
    print("\n  Открытые ключи:")
    dh.compute_public_keys()
    print("\n  Общий секрет:")
    dh.compute_shared_secret()

def manual_protocol(dh):
    """Последовательно запрашивает параметры и ключи, затем проверяет обмен."""
    print("\n" + "—" * 60)
    print("  ПОШАГОВЫЙ ОБМЕН ДИФФИ-ХЕЛЛМАНА")
    print("—" * 60)
    print("\n1. Введите публичные параметры")
    try:
        n = int(input("  Простое число n: "))
        a = int(input("  Основание a (1 < a < n): "))
    except ValueError:
        print("  Ошибка: введите целые числа.")
        return False
    if not dh.set_params(n, a):
        return False
    while True:
        print("\n2. Введите секретные ключи A и B")
        try:
            K_A = int(input(f"  Секретный ключ A (2..{n - 1}): "))
```

```
            K_B = int(input(f"  Секретный ключ B (2..{n - 1}): "))
        except ValueError:
            print("  Ошибка: введите целые числа; повторите ввод ключей.")
            continue
        if not dh.set_secret_key_A(K_A) or not dh.set_secret_key_B(K_B):
            print("  Повторите ввод секретных ключей.")
            continue
        print("\n3. Вычислите открытые ключи")
        if not dh.compute_public_keys():
            return False
        print("\n4. Вычислите общий секрет и проверьте протокол")
        if dh.compute_shared_secret():
            print("\n  Протокол успешно завершён.")
            return True
        print("\n  Обмен не принят. Введите другие секретные ключи A и B.")

def main():
    dh = DiffieHellman()
    while True:
        print_menu(dh)
        choice = input("  Ваш выбор: ").strip()
        if choice == "1":
            try:
                n = int(input("  Введите простое число n: "))
                a = int(input("  Введите основание a (1 < a < n): "))
                dh.set_params(n, a)
            except ValueError:
                print("  Ошибка: введите целые числа.")
        elif choice == "2":
            try:
                bits = int(input("  Битность n для учебной автогенерации (4..32) [24]: ").strip() or "24")
                dh.auto_generate_params(bits)
            except ValueError:
                print("  Ошибка: введите целое число.")
        elif choice == "3":
            if dh.n is None:
                print("  Ошибка: сначала установите параметры (п.1 или п.2).")
            else:
                try:
                    K_A = int(input(f"  Введите секретный ключ K_A (из [2, {dh.n - 1}]): "))
                    dh.set_secret_key_A(K_A)
                except ValueError:
                    print("  Ошибка: введите целое число.")
        elif choice == "4":
            if dh.n is None:
                print("  Ошибка: сначала установите параметры (п.1 или п.2).")
            else:
                try:
                    K_B = int(input(f"  Введите секретный ключ K_B (из [2, {dh.n - 1}]): "))
                    dh.set_secret_key_B(K_B)
                except ValueError:
                    print("  Ошибка: введите целое число.")
        elif choice == "5":
            dh.compute_public_keys()
        elif choice == "6":
            dh.compute_shared_secret()
        elif choice == "7":
```

```
            manual_protocol(dh)
        elif choice == "8":
            demo_textbook(dh)
        elif choice == "0":
            print("  Выход.")
            break
        else:
            print("  Неверный выбор.")
        input("\n  Нажмите Enter для продолжения...")

if __name__ == "__main__":
    main()
```

Результат работы программы:

```
ОБМЕН КЛЮЧАМИ ПО АЛГОРИТМУ ДИФФИ-ХЕЛЛМАНА
1. Ввести параметры и ключи (p, g, a, b)
2. Показать текущие данные
3. Выход
Ваш выбор: 1

Введите общие параметры и секретные ключи:
p (простое число, например 23): 23
g (основание, 1 < g < p): 5
Подсказка: первообразный корень по модулю 23 — 5
Секретный ключ пользователя A (0 < a < p-1): 7
Секретный ключ пользователя B (0 < b < p-1): 9

=== ВЫЧИСЛЕНИЯ ===
Открытый ключ A: A = g^a mod p = 5^7 mod 23 = 17
Открытый ключ B: B = g^b mod p = 5^9 mod 23 = 11
Общий секретный ключ (сторона A): K = B^a mod p = 11^7 mod 23 = 7
Общий секретный ключ (сторона B): K = A^b mod p = 17^9 mod 23 = 7
```

Рисунок 46. Результат обмена ключами.

Работа в тетради:

Обмен ключами с виртуальными пользователями по алгоритму Диффи-Хеллмана

Пусть $p = 23$, $g = 5$ $(1 < g < p)$

Пользователь A: $k_A = 7$ $k_A \in [2, 22]$

$y_A = g^{k_A} \bmod p$, $y_A = 5^7 \bmod 23 = 17$

Далее A отправляет открытый ключ

Пользователь B $k_B = 9$, $k_B \in [2, 22]$

$y_B = 5^9 \bmod 23 = 11$

A: $K = y_B^{k_A} \bmod p$ $11^7 \bmod 23 = 7$

$K = 7$

B: $y_A^{k_B} \bmod p$ $17^9 \bmod 23 = 7$

$K = 7$

Оба пользователя пришли к одному ключу, все верно

Рисунок 47. Обмен ключами в тетради.

Заключение

В работе рассмотрены криптографические алгоритмы из блоков А–К: от классических шифров до цифровых подписей и обмена ключами. Общий интерфейс позволяет запускать реализации из одного меню, а приведённые схемы, исходный код и результаты выполнения помогают проследить основные этапы каждого алгоритма.

Проверка обратных преобразований, контрольных примеров и результатов протоколов позволяет оценить корректность работы реализованных программ на приведённых входных данных. Актуальные версии алгоритмов МАГМА, Кузнечик и ГОСТ Р 34.10-2012 вынесены в приложение А.

Приложение А. Дополнительные реализации алгоритмов

В этом приложении приведены дополнительные версии программ МАГМА и Кузнечик. Основные программы, включённые в общий интерфейс, размещены в соответствующих разделах отчёта.

17. Шифр МАГМА

Файл программы: 17.MAGMA.py

```
# -*- coding: utf-8 -*-
"""
17. Шифр МАГМА (ГОСТ Р 34.12-2015)
Реализация блочного шифра «Магма» с режимом ECB.
Блок: 64 бита, ключ: 256 бит, 32 раунда (сеть Фейстеля).
Тестирование по ГОСТ Р 34.12-2018 (Приложение А).
Источник: ЛАБКРИПТ_2022, Блок G, задание 17.
ВВОД (как в методичке):
  - Открытый текст: произвольная строка (кодируется в UTF-8)
  - Ключ: 64 шестнадцатеричных символа (256 бит)
АЛГОРИТМ (по ГОСТ Р 34.12-2018 раздел 5):
  Блок a = a1|a0 (a1 — старшие 32 бит, a0 — младшие 32 бит).
  Раунд: G[k](a1,a0) = (a0, g[k](a0) XOR a1)
  Последний раунд: G*[k](a1,a0) = (g[k](a0) XOR a1, a0)
  Ключ K делится на K1..K8 (big-endian, 32 бит каждый).
  Расписание: K1..K8 × 3, затем K8..K1.
"""
# ──────────────────────────────────────────────
#  S-блоки ГОСТ Р 34.12-2015 (Приложение А)
# ──────────────────────────────────────────────
S_BOX = (
    (12, 4, 6, 2, 10, 5, 11, 9, 14, 8, 13, 7, 0, 3, 15, 1),
    (6, 8, 2, 3, 9, 10, 5, 12, 1, 14, 4, 7, 11, 13, 0, 15),
    (11, 3, 5, 8, 2, 15, 10, 13, 14, 1, 7, 4, 12, 9, 6, 0),
    (12, 8, 2, 1, 13, 4, 15, 6, 7, 0, 10, 5, 3, 14, 9, 11),
    (7, 15, 5, 10, 8, 1, 6, 13, 0, 9, 3, 14, 11, 4, 2, 12),
    (5, 13, 15, 6, 9, 2, 12, 10, 11, 7, 8, 1, 4, 3, 14, 0),
    (8, 14, 2, 5, 6, 9, 1, 12, 15, 4, 11, 0, 13, 10, 3, 7),
    (1, 7, 14, 13, 0, 5, 8, 3, 4, 15, 10, 6, 9, 12, 11, 2),
)
# ──────────────────────────────────────────────
#  Нелинейное преобразование T (8 × S-блоков)
# ──────────────────────────────────────────────
def t_direct(a: int) -> int:
    a &= 0xFFFFFFFF
    out = 0
    for i in range(8):
        nibble = (a >> (4 * i)) & 0xF
        out |= S_BOX[i][nibble] << (4 * i)
    return out & 0xFFFFFFFF
# ──────────────────────────────────────────────
#  Циклический сдвиг влево на 11 бит
# ──────────────────────────────────────────────
def rotl32(x: int, n: int) -> int:
    x &= 0xFFFFFFFF
    return ((x << n) | (x >> (32 - n))) & 0xFFFFFFFF
# ──────────────────────────────────────────────
#  Раундовая функция g[k](a) = rotl11(T(a + k mod 2^32))
# ──────────────────────────────────────────────
def g_round(k: int, a: int) -> int:
```

```
    x = (a + k) & 0xFFFFFFFF
    x = t_direct(x)
    return rotl32(x, 11)

# ──────────────────────────────────────────────
#  Расписание ключей
#  K = K1|K2|...|K8 (big-endian 32-бит слова)
#  Раунды 1-24: K1,K2,...,K8 повторить 3 раза
#  Раунды 25-32: K8,K7,...,K1
# ──────────────────────────────────────────────
def round_keys_from_key(key32: bytes) -> list:
    if len(key32) != 32:
        raise ValueError("Ключ должен быть 32 байта (256 бит)")
    K = [int.from_bytes(key32[4 * i: 4 * (i + 1)], "big") for i in range(8)]
    rk = []
    for i in range(32):
        if i < 24:
            rk.append(K[i % 8])
        else:
            rk.append(K[7 - (i - 24)])
    return rk

# ──────────────────────────────────────────────
#  Шифрование одного 64-битного блока (big-endian)
#  a = a1|a0  (a1 — старшие 32 бит, a0 — младшие 32 бит)
#  G[k](a1,a0)  = (a0,  g[k](a0) XOR a1)   — обычный раунд
#  G*[k](a1,a0) = (g[k](a0) XOR a1, a0)    — последний раунд
# ──────────────────────────────────────────────
def encrypt_block_magma(block: bytes, rk: list) -> bytes:
    a1 = int.from_bytes(block[0:4], "big")
    a0 = int.from_bytes(block[4:8], "big")
    for i in range(31):
        # G[rk[i]](a1, a0)
        a1, a0 = a0, (g_round(rk[i], a0) ^ a1) & 0xFFFFFFFF
    # G*[rk[31]](a1, a0) — без перестановки полублоков
    b1 = (g_round(rk[31], a0) ^ a1) & 0xFFFFFFFF
    b0 = a0
    return b1.to_bytes(4, "big") + b0.to_bytes(4, "big")

def decrypt_block_magma(block: bytes, rk: list) -> bytes:
    b1 = int.from_bytes(block[0:4], "big")
    b0 = int.from_bytes(block[4:8], "big")
    # Обратное G*[rk[31]]: a1 = g[k](b0) XOR b1, a0 = b0
    a1 = (g_round(rk[31], b0) ^ b1) & 0xFFFFFFFF
    a0 = b0
    # Обратные раунды 30..0
    for i in range(30, -1, -1):
        # Обратное G[rk[i]](a1, a0) = (old_a0, ...) = (a0 прежде свопа)
        # G дало: new_a1=a0_old, new_a0 = g[k](a0_old) XOR a1_old
        # Значит: a0_old = a1, a1_old = g[k](a1) XOR a0
        a1, a0 = (g_round(rk[i], a1) ^ a0) & 0xFFFFFFFF, a1
    return a1.to_bytes(4, "big") + a0.to_bytes(4, "big")

# ──────────────────────────────────────────────
#  ECB-режим с PKCS#7-дополнением (блок 8 байт)
# ──────────────────────────────────────────────
def pad_pkcs7(data: bytes, block_size: int) -> bytes:
    pad_len = block_size - (len(data) % block_size)
    return data + bytes([pad_len] * pad_len)

def unpad_pkcs7(data: bytes) -> bytes:
```

```
    if not data:
        raise ValueError("Нельзя снять дополнение с пустых данных.")
    pad_len = data[-1]
    if pad_len < 1 or pad_len > 8 or data[-pad_len:] != bytes([pad_len]) * pad_len:
        raise ValueError("Неверный ключ или повреждённый шифртекст: дополнение некорректно.")
    return data[:-pad_len]

def magma_ecb_encrypt(plaintext: bytes, key: bytes) -> bytes:
    rk = round_keys_from_key(key)
    padded = pad_pkcs7(plaintext, 8)
    ciphertext = b''
    for i in range(0, len(padded), 8):
        ciphertext += encrypt_block_magma(padded[i:i + 8], rk)
    return ciphertext

def magma_ecb_decrypt(ciphertext: bytes, key: bytes) -> bytes:
    rk = round_keys_from_key(key)
    plaintext = b''
    for i in range(0, len(ciphertext), 8):
        plaintext += decrypt_block_magma(ciphertext[i:i + 8], rk)
    return unpad_pkcs7(plaintext)

# ──────────────────────────────────────────────
#  Тест по ГОСТ Р 34.12-2018 (Приложение А)
# ──────────────────────────────────────────────

def test_gost_vectors():
    print("=" * 70)
    print("ТЕСТ МАГМА по ГОСТ Р 34.12-2018 (Приложение А)")
    print("=" * 70)
    key = bytes.fromhex(
        "ffeeddccbbaa99887766554433221100"
        "f0f1f2f3f4f5f6f7f8f9fafbfcfdfeff"
    )
    plaintext          = bytes.fromhex("fedcba9876543210")
    expected_ciphertext= bytes.fromhex("4ee901e5c2d8ca3d")
    rk         = round_keys_from_key(key)
    ciphertext = encrypt_block_magma(plaintext, rk)
    decrypted  = decrypt_block_magma(ciphertext, rk)
    print(f"Ключ (hex)        : {key.hex()}")
    print(f"Открытый текст    : {plaintext.hex()}")
    print(f"Ожидаемый шифр    : {expected_ciphertext.hex()}")
    print(f"Полученный шифр   : {ciphertext.hex()}")
    print(f"Тест шифрования   : {'ДА ✓' if ciphertext == expected_ciphertext else 'НЕТ ✗'}")
    print(f"Расшифрование     : {decrypted.hex()}")
    print(f"Тест расшифрования: {'ДА ✓' if decrypted == plaintext else 'НЕТ ✗'}")
    print("=" * 70)
    return ciphertext == expected_ciphertext and decrypted == plaintext

# ──────────────────────────────────────────────
#  Ввод ключа от пользователя
# ──────────────────────────────────────────────

def read_key_from_user() -> bytes:
    """Запрашивает 256-битный ключ в виде 64 hex-символов."""
    print("Ключ вводится как 64 шестнадцатеричных символа (256 бит).")
    print("Пример: ffeeddccbbaa99887766554433221100f0f1f2f3f4f5f6f7f8f9fafbfcfdfeff")
    key_hex = input("Введите ключ: ").strip().replace(" ", "")
    if len(key_hex) != 64:
        raise ValueError("Ключ должен содержать ровно 64 hex-символа (32 байта).")
    try:
        return bytes.fromhex(key_hex)
    except ValueError as exc:
```

```
        raise ValueError("Ключ должен состоять только из символов 0-9 и A-F.") from exc

# ──────────────────────────────────────────────
#  Интерактивные функции
# ──────────────────────────────────────────────
def interactive_encrypt_hex():
    """
    Зашифровать hex-блок (как в контрольном примере ГОСТ Р 34.12-2018).
    Ввод : hex-строка, кратная 16 символам (= 8 байт на блок).
    Пример: PT = fedcba9876543210  →  CT = 4ee901e5c2d8ca3d
    """
    pt_hex = input("Введите открытый текст в hex (кратно 16 символам): ").strip().replace(" ", "")
    if not pt_hex:
        raise ValueError("Блок не должен быть пустым.")
    if len(pt_hex) % 16 != 0:
        raise ValueError("Длина hex должна быть кратна 16 (8 байт на блок).")
    try:
        pt_bytes = bytes.fromhex(pt_hex)
    except ValueError as exc:
        raise ValueError("Допустимы только символы 0-9 и A-F.") from exc
    key = read_key_from_user()
    rk = round_keys_from_key(key)
    ciphertext = b''
    for i in range(0, len(pt_bytes), 8):
        ciphertext += encrypt_block_magma(pt_bytes[i:i + 8], rk)
    print(f"Шифртекст (hex): {ciphertext.hex()}")


def interactive_decrypt():
    """Расшифровать hex-шифртекст."""
    ciphertext_hex = input("Введите шифртекст в hex: ").strip().replace(" ", "")
    if not ciphertext_hex:
        raise ValueError("Шифртекст не должен быть пустым.")
    if len(ciphertext_hex) % 16 != 0:
        raise ValueError("Длина hex-шифртекста должна быть кратна 16 символам (8 байт на блок).")
    try:
        ciphertext = bytes.fromhex(ciphertext_hex)
    except ValueError as exc:
        raise ValueError("Шифртекст должен состоять только из символов 0-9 и A-F.") from exc
    key = read_key_from_user()
    plaintext = magma_ecb_decrypt(ciphertext, key)
    try:
        print(f"Открытый текст: {plaintext.decode('utf-8')}")
    except UnicodeDecodeError:
        print(f"Результат (hex): {plaintext.hex()}")

# ──────────────────────────────────────────────
#  Главное меню
# ──────────────────────────────────────────────
def interactive_menu():
    print("\n" + "=" * 70)
    print("ШИФР МАГМА (ГОСТ Р 34.12-2015)  —  Задание 17")
    print("Сеть Фейстеля, 32 раунда, блок 64 бит, ключ 256 бит")
    print("Источник: ЛАБКРИПТ_2022, Блок G, стр. 29")
    print("=" * 70)
    print()
    print("ВВОД:")
    print("  Открытый текст : hex-строка, кратная 16 символам (8 байт на блок)")
    print("  Ключ           : 64 шестнадцатеричных символа (256 бит)")
    print("  Пример ГОСТ    : PT=fedcba9876543210 → CT=4ee901e5c2d8ca3d")
    print()
```

```
    while True:
        print("1 — Зашифровать hex-блок")
        print("2 — Расшифровать hex-шифртекст")
        print("0 — Выход")
        choice = input("Выберите действие: ").strip()
        if choice == "0":
            break
        elif choice == "1":
            try:
                interactive_encrypt_hex()
            except (ValueError, UnicodeError) as exc:
                print(f"Ошибка ввода: {exc}")
        elif choice == "2":
            try:
                interactive_decrypt()
            except (ValueError, UnicodeError) as exc:
                print(f"Ошибка ввода: {exc}")
        else:
            print("Выберите 0, 1 или 2.")
        print()

if __name__ == "__main__":
    interactive_menu()
```

20. Шифр Кузнечик

Файл программы: 20.KUZNECHIK.py

```
# -*- coding: utf-8 -*-
"""
20. Шифр КУЗНЕЧИК (ГОСТ Р 34.12-2015)
Реализация блочного шифра «Кузнечик».
Блок: 128 бит, ключ: 256 бит, 10 раундов.
Тестирование по ГОСТ Р 34.12-2015 (Приложение А).
"""
PI = [
    252, 238, 221, 17, 207, 110, 49, 22, 251, 196, 250, 218, 35, 197, 4, 77,
    233, 119, 240, 219, 147, 46, 153, 186, 23, 54, 241, 187, 20, 205, 95, 193,
    249, 24, 101, 90, 226, 92, 239, 33, 129, 28, 60, 66, 139, 1, 142, 79,
    5, 132, 2, 174, 227, 106, 143, 160, 6, 11, 237, 152, 127, 212, 211, 31,
    235, 52, 44, 81, 234, 200, 72, 171, 242, 42, 104, 162, 253, 58, 206, 204,
    181, 112, 14, 86, 8, 12, 118, 18, 191, 114, 19, 71, 156, 183, 93, 135,
    21, 161, 150, 41, 16, 123, 154, 199, 243, 145, 120, 111, 157, 158, 178, 177,
    50, 117, 25, 61, 255, 53, 138, 126, 109, 84, 198, 128, 195, 189, 13, 87,
    223, 245, 36, 169, 62, 168, 67, 201, 215, 121, 214, 246, 124, 34, 185, 3,
    224, 15, 236, 222, 122, 148, 176, 188, 220, 232, 40, 80, 78, 51, 10, 74,
    167, 151, 96, 115, 30, 0, 98, 68, 26, 184, 56, 130, 100, 159, 38, 65,
    173, 69, 70, 146, 39, 94, 85, 47, 140, 163, 165, 125, 105, 213, 149, 59,
    7, 88, 179, 64, 134, 172, 29, 247, 48, 55, 107, 228, 136, 217, 231, 137,
    225, 27, 131, 73, 76, 63, 248, 254, 141, 83, 170, 144, 202, 216, 133, 97,
    32, 113, 103, 164, 45, 43, 9, 91, 203, 155, 37, 208, 190, 229, 108, 82,
    89, 166, 116, 210, 230, 244, 180, 192, 209, 102, 175, 194, 57, 75, 99, 182,
]
PI_INV = [0] * 256
for i in range(256):
    PI_INV[PI[i]] = i
L_VEC = [148, 32, 133, 16, 194, 192, 1, 251, 1, 192, 194, 16, 133, 32, 148, 1]
def gf_mul(a, b):
    p = 0
    hi_bit_set = 0
    for _ in range(8):
        if b & 1:
            p ^= a
        hi_bit_set = a & 0x80
        a <<= 1
        if hi_bit_set:
            a ^= 0x1c3
        a &= 0xff
        b >>= 1
    return p
def S(block):
    return bytes(PI[b] for b in block)
def S_inv(block):
    return bytes(PI_INV[b] for b in block)
def R(block):
    a = 0
    for i in range(16):
        a ^= gf_mul(block[i], L_VEC[i])
    return bytes([a]) + block[:15]
```

```
def R_inv(block):
    a = block[0]
    block = block[1:] + bytes([0])
    result = block[:-1] + bytes([0])
    new_last = 0
    for i in range(15):
        new_last ^= gf_mul(result[i], L_VEC[i])
    new_last ^= gf_mul(a, L_VEC[15])
    return result[:15] + bytes([new_last])

def L(block):
    for _ in range(16):
        block = R(block)
    return block

def L_inv(block):
    for _ in range(16):
        block = R_inv(block)
    return block

def F(k, a1, a0):
    tmp = bytes(a ^ b for a, b in zip(k, a1))
    tmp = L(S(tmp))
    return tmp, bytes(a ^ b for a, b in zip(tmp, a0))

def C(i):
    c = bytes([0] * 15 + [i])
    return L(c)

def key_expansion(key):
    k1 = key[:16]
    k2 = key[16:]
    round_keys = [k1, k2]
    a1, a0 = k1, k2
    for i in range(1, 5):
        for j in range(8):
            c = C(8 * (i - 1) + j + 1)
            a1, a0 = F(c, a1, a0)
        round_keys.extend([a1, a0])
    return round_keys

def encrypt_block(block, round_keys):
    state = block
    for i in range(9):
        state = bytes(a ^ b for a, b in zip(state, round_keys[i]))
        state = S(state)
        state = L(state)
    state = bytes(a ^ b for a, b in zip(state, round_keys[9]))
    return state

def decrypt_block(block, round_keys):
    state = block
    state = bytes(a ^ b for a, b in zip(state, round_keys[9]))
    for i in range(8, -1, -1):
        state = L_inv(state)
        state = S_inv(state)
        state = bytes(a ^ b for a, b in zip(state, round_keys[i]))
    return state

def pad_pkcs7(data: bytes, block_size: int) -> bytes:
    pad_len = block_size - (len(data) % block_size)
    return data + bytes([pad_len] * pad_len)
```

```
def unpad_pkcs7(data: bytes) -> bytes:
    if not data:
        raise ValueError("Нельзя снять дополнение с пустых данных.")
    pad_len = data[-1]
    if pad_len < 1 or pad_len > 16 or data[-pad_len:] != bytes([pad_len]) * pad_len:
        raise ValueError("Неверный ключ или повреждённый шифртекст: дополнение некорректно.")
    return data[:-pad_len]

def kuznechik_ecb_encrypt(plaintext: bytes, key: bytes) -> bytes:
    round_keys = key_expansion(key)
    padded = pad_pkcs7(plaintext, 16)
    ciphertext = b''
    for i in range(0, len(padded), 16):
        block = padded[i:i+16]
        ciphertext += encrypt_block(block, round_keys)
    return ciphertext

def kuznechik_ecb_decrypt(ciphertext: bytes, key: bytes) -> bytes:
    round_keys = key_expansion(key)
    plaintext = b''
    for i in range(0, len(ciphertext), 16):
        block = ciphertext[i:i+16]
        plaintext += decrypt_block(block, round_keys)
    return unpad_pkcs7(plaintext)

def test_gost_vectors():
    print("=" * 70)
    print("ТЕСТ КУЗНЕЧИК по ГОСТ Р 34.12-2015 (Приложение А)")
    print("=" * 70)
    key = bytes.fromhex(
        "8899aabbccddeeff0011223344556677"
        "fedcba9876543210123456789abcdef"
    )
    plaintext = bytes.fromhex("1122334455667700ffeeddccbbaa9988")
    expected_ciphertext = bytes.fromhex("7f679d90bebc24305a468d42b9d4edcd")
    round_keys = key_expansion(key)
    ciphertext = encrypt_block(plaintext, round_keys)
    print(f"Ключ (hex): {key.hex()}")
    print(f"Открытый текст: {plaintext.hex()}")
    print(f"Ожидаемый шифртекст: {expected_ciphertext.hex()}")
    print(f"Полученный шифртекст: {ciphertext.hex()}")
    enc_ok = ciphertext == expected_ciphertext
    print(f"Тест шифрования пройден: {'ДА' if enc_ok else 'НЕТ'}")
    decrypted = decrypt_block(ciphertext, round_keys)
    print(f"Расшифрованный текст: {decrypted.hex()}")
    dec_ok = decrypted == plaintext
    print(f"Расшифрование верно: {'ДА' if dec_ok else 'НЕТ'}")
    print("\nПроверка S-преобразования (A.1.1):")
    s_input = bytes.fromhex("ffeeddccbbaa99881122334455667700")
    s_expected = bytes.fromhex("b66cd8887d38e8d77765aeea0c9a7efc")
    s_result = S(s_input)
    print(f"S({s_input.hex()}) = {s_result.hex()}")
    print(f"Ожидается: {s_expected.hex()}")
    s_ok = s_result == s_expected
    print(f"Тест S пройден: {'ДА' if s_ok else 'НЕТ'}")
    print("\nПроверка R-преобразования (A.1.2):")
    r_input = bytes.fromhex("00000000000000000000000000000100")
    r_expected = bytes.fromhex("94000000000000000000000000000001")
    r_result = R(r_input)
```

```
    print(f"R({r_input.hex()}) = {r_result.hex()}")
    print(f"Ожидается: {r_expected.hex()}")
    r_ok = r_result == r_expected
    print(f"Тест R пройден: {'ДА' if r_ok else 'НЕТ'}")

    print("\nПроверка L-преобразования (A.1.3):")
    l_input = bytes.fromhex("64a59400000000000000000000000000")
    l_expected = bytes.fromhex("d456584dd0e3e84cc3166e4b7fa2890d")
    l_result = L(l_input)
    print(f"L({l_input.hex()}) = {l_result.hex()}")
    print(f"Ожидается: {l_expected.hex()}")
    l_ok = l_result == l_expected
    print(f"Тест L пройден: {'ДА' if l_ok else 'НЕТ'}")

    print("=" * 70)
    return enc_ok and dec_ok

def test_long_text():
    print("\n" + "=" * 70)
    print("ТЕСТ НА ТЕКСТЕ 1000+ СИМВОЛОВ")
    print("=" * 70)

    long_text = """Криптография — наука о методах обеспечения конфиденциальности,
целостности данных, аутентификации и невозможности отказа от авторства.
Шифр «Кузнечик» (ГОСТ Р 34.12-2015) является одним из стандартных
блочных шифров Российской Федерации. Он использует SP-сеть с 10 раундами,
размером блока 128 бит и ключом 256 бит.

Шифр «Кузнечик» был разработан для замены устаревшего шифра «Магма»
(ГОСТ 28147-89) и обеспечивает более высокий уровень безопасности
благодаря увеличенному размеру блока.

Основные преобразования шифра:
1. S-преобразование (нелинейная подстановка)
2. R-преобразование (линейный сдвиговый регистр)
3. L-преобразование (16-кратное применение R)
4. Сложение с раундовым ключом по модулю 2 (XOR)

Ключевое расписание шифра «Кузнечик» основано на сети Фейстеля
и использует константы C_i, вычисляемые как L([i]).

Данный алгоритм широко применяется для защиты конфиденциальной информации
в государственных и коммерческих системах Российской Федерации.

The Kuznyechik cipher is a symmetric block cipher standardized in
GOST R 34.12-2015. It operates on 128-bit blocks with a 256-bit key
and uses a substitution-permutation network structure with 10 rounds.
""" * 2

    print(f"Длина текста: {len(long_text)} символов")

    key = bytes.fromhex(
        "8899aabbccddeeff0011223344556677"
        "fedcba98765432100123456789abcdef"
    )

    plaintext_bytes = long_text.encode('utf-8')
    print(f"Длина в байтах: {len(plaintext_bytes)}")

    ciphertext = kuznechik_ecb_encrypt(plaintext_bytes, key)
    print(f"Длина шифртекста: {len(ciphertext)} байт")
    print(f"Первые 64 байта шифртекста (hex): {ciphertext[:64].hex()}")

    decrypted = kuznechik_ecb_decrypt(ciphertext, key)
    decrypted_text = decrypted.decode('utf-8')

    print(f"Расшифрование успешно: {'ДА' if decrypted_text == long_text else 'НЕТ'}")
    print("=" * 70)
    return decrypted_text == long_text

def demo_phrase():
```

```
    print("\n" + "=" * 70)
    print("ДЕМОНСТРАЦИЯ НА ФРАЗЕ ВАРИАНТА")
    print("=" * 70)
    phrase = "Леопард не может изменить своих пятен"
    print(f"Фраза: {phrase}")
    key = bytes.fromhex(
        "8899aabbccddeeff0011223344556677"
        "fedcba98765432100123456789abcdef"
    )
    plaintext_bytes = phrase.encode('utf-8')
    ciphertext = kuznechik_ecb_encrypt(plaintext_bytes, key)
    print(f"Шифртекст (hex): {ciphertext.hex()}")
    decrypted = kuznechik_ecb_decrypt(ciphertext, key)
    decrypted_text = decrypted.decode('utf-8')
    print(f"Расшифрованный текст: {decrypted_text}")
    print(f"Совпадение: {'ДА' if decrypted_text == phrase else 'НЕТ'}")
    print("=" * 70)

def read_key_from_user() -> bytes:
    """Запрашивает ключ Кузнечика длиной 256 бит в hex-формате."""
    key_hex = input(
        "Введите ключ (64 hex-символа, пример "
        "8899aabbccddeeff0011223344556677fedcba98765432100123456789abcdef): "
    ).strip().replace(" ", "")
    if len(key_hex) != 64:
        raise ValueError("Ключ должен содержать ровно 64 hex-символа (32 байта).")
    try:
        return bytes.fromhex(key_hex)
    except ValueError as exc:
        raise ValueError("Ключ должен состоять только из символов 0-9 и A-F.") from exc

def interactive_menu():
    print("Кузнечик использует ключ длиной 256 бит (32 байта).")
    print("Ключ вводится вручную как 64 шестнадцатеричных символа.")
    while True:
        print("\n1 - Зашифровать текст")
        print("2 - Расшифровать hex-шифртекст")
        print("3 - Показать тестовый пример ГОСТ")
        print("0 - Выход")
        choice = input("Выберите действие: ").strip()
        if choice == "0":
            break
        if choice == "3":
            test_gost_vectors()
            continue
        if choice not in ("1", "2"):
            print("Выберите 0, 1, 2 или 3.")
            continue
        try:
            if choice == "1":
                plaintext = input("Введите открытый текст: ")
                key = read_key_from_user()
                ciphertext = kuznechik_ecb_encrypt(plaintext.encode("utf-8"), key)
                print(f"Шифртекст (hex): {ciphertext.hex()}")
            else:
                ciphertext_hex = input("Введите шифртекст в hex: ").strip().replace(" ", "")
                if not ciphertext_hex:
                    raise ValueError("Шифртекст не должен быть пустым.")
                if len(ciphertext_hex) % 32 != 0:
```

```
                    raise ValueError("Длина hex-шифртекста должна быть кратна 32 символам (16 байт).")
                try:
                    ciphertext = bytes.fromhex(ciphertext_hex)
                except ValueError as exc:
                    raise ValueError("Шифртекст должен состоять только из символов 0-9 и A-F.") from exc
                key = read_key_from_user()
                plaintext = kuznechik_ecb_decrypt(ciphertext, key)
                try:
                    print(f"Открытый текст: {plaintext.decode('utf-8')}")
                except UnicodeDecodeError:
                    print(f"Результат (hex): {plaintext.hex()}")
        except ValueError as exc:
            print(f"Ошибка ввода: {exc}")

if __name__ == "__main__":
    print("\n" + "=" * 70)
    print("ШИФР КУЗНЕЧИК (ГОСТ Р 34.12-2015)")
    print("=" * 70)
    interactive_menu()
```

Приложение Б. Полная таблица расчёта A5/1 для слова ПЕРЕСДАЧА

Ниже приведена полная трассировка учебной реализации A5/1. Для каждого такта показаны состояния трёх регистров после шага, управляющие биты, большинство, обратная связь и решение о сдвиге. На этапах генерации также приведены бит гаммы, бит открытого текста и результат их XOR.

Параметр	Значение
Ключ в программе	АВГУСТ
Ключ UTF-8, первые 8 байт (hex)	D090D092D093D0A3
Номер кадра	155 (22 бита: 0000000000000010011011)
Открытый текст	ПЕРЕСДАЧА (UTF-8: 18 байт, 144 бит)
Гамма (hex)	D1842338C5696D732372388B7D231FE74044
Шифртекст (hex)	011BF3AD15C9BDE6F3D3E81FADB3CF4090D4

Этапы: 64 такта загрузки ключа, 22 такта загрузки кадра, 100 тактов холостого хода и 144 такта генерации. R1 содержит 19 бит, R2 — 22 бита, R3 — 23 бита. Индексы управляющих и обратной связи в таблице совпадают с индексами Python и начинаются с нуля.

Таблица 4. Полная трассировка загрузки, холостого хода и генерации A5/1.

№	Этап		x	R1 19b	R2 22b	R3 23b	1	2	3		ОС 1/2/3	Сдвиг 1/2/3			
0	Начальное состояние			0000000000000000000	0000000000000000000000	00000000000000000000000					//	0/0/0			
1	Ключ			0100000000000000000	0100000000000000000000	01000000000000000000000					0/0/0	1/1/1			
2	Ключ			0110000000000000000	0110000000000000000000	01100000000000000000000					0/0/0	1/1/1			
3	Ключ			0011000000000000000	0011000000000000000000	00110000000000000000000					0/0/0	1/1/1			
4	Ключ			0101100000000000000	0101100000000000000000	01011000000000000000000					0/0/0	1/1/1			
5	Ключ			0010110000000000000	0010110000000000000000	00101100000000000000000					0/0/0	1/1/1			
6	Ключ			0001011000000000000	0001011000000000000000	00010110000000000000000					0/0/0	1/1/1			
7	Ключ			0000101100000000000	0000101100000000000000	00001011000000000000000					0/0/0	1/1/1			
8	Ключ			0000010110000000000	0000010110000000000000	10000101100000000000000					0/0/1	1/1/1			
9	Ключ			0100001011000000000	0100001011000000000000	10000010110000000000000					0/0/1	1/1/1			
10	Ключ	0		0010000101100000000	0010000101100000000000	01000001011000000000000					0/0/0	1/1/1			
11	Ключ	1		0001000010110000000	0001000010110000000000	10100000101100000000000					0/0/1	1/1/1			
12	Ключ	2		0100100001011000000	0100100001011000000000	00010000010110000000000					0/0/0	1/1/1			
13	Ключ	3		0010010000101100000	0010010000101100000000	00001000001011000000000					0/0/0	1/1/1			

Продолжение таблицы 4. Такты 14–27.

№	Этап		x	R1 19b	R2 22b	R3 23b	1	2	3		ОС 1/2/3	Сдвиг 1/2/3			
14	Ключ	4		1001001000010110000	0001001000010110000000	0000010000010110000000					1/0/0	1/1/1			
15	Ключ	5		1100100100001011000	0000100100001011000000	0000001000001011000000					1/0/0	1/1/1			
16	Ключ	6		0110010010000101100	0000010010000101100000	0000000100000101100000					0/0/0	1/1/1			
17	Ключ	7		0111001001000010110	0100001001000010110000	1100000010000010110000					0/0/1	1/1/1			
18	Ключ	8		0111100100100001011	0110000100100001011000	0010000001000001011000					0/0/0	1/1/1			
19	Ключ	9		0011110010010000101	0011000010010000101100	0001000000100000101100					0/0/0	1/1/1			
20	Ключ	0		0101111001001000010	0101100001001000010110	0100100000010000010110					0/0/0	1/1/1			
21	Ключ	1		1010111100100100001	1010110000100100001011	1010010000001000010110					1/1/1	1/1/1			
22	Ключ	2		0101011110010010000	0101011000010010000101	0101001000000100000101					0/0/0	1/1/1			
23	Ключ	3		0010101111001001000	1010101100001001000010	0010010000000100000101					0/1/0	1/1/1			
24	Ключ	4		0001010111100100100	1101010110000100100001	1001010010000001000010					0/1/1	1/1/1			
25	Ключ	5		0100101011110010010	1010101011000010010000	1000101001000001000001					0/1/1	1/1/1			
26	Ключ	6		1010010101111001001	0101010101100001001000	1100010100100000100000					1/0/1	1/1/1			
27	Ключ	7		1101001010111100100	0010101010110000100100	1110001010010000010000					1/0/1	1/1/1			

Продолжение таблицы 4. Такты 28–41.

№	Этап		x	R1 19b	R2 22b	R3 23b	1	2	3		ОС 1/2/3	Сдвиг 1/2/3			
28	Ключ	8		0010100101011110010	0101010101011000010010	0011000101001000001000					0/0/0	1/1/1			
29	Ключ	9		0001010010101111001	1010101010101100001001	1001100010100100000100					0/1/1	1/1/1			
30	Ключ	0		0000101001010111100	1101010101010110000100	1100110001010010000010					0/1/1	1/1/1			
31	Ключ	1		0100010100101011110	0010101010101011000010	1010011000101001000001					0/0/1	1/1/1			
32	Ключ	2		0010001010010101111	1001010101010101100001	1101001100010100100000					0/1/1	1/1/1			
33	Ключ	3		0101000101001010111	1000101010101010110000	1010100110001010010000					0/1/1	1/1/1			
34	Ключ	4		1110100010100101011	0000010101010101011000	1001010011000101001000					1/0/1	1/1/1			
35	Ключ	5		1111010001010010101	0000001010101010101100	0100101001100010100100					1/0/0	1/1/1			
36	Ключ	6		0011101000101001010	0100000101010101010110	0110010100110001010010					0/0/0	1/1/1			
37	Ключ	7		1001110100010100101	1010000010101010101011	0011001010011000101001					1/1/0	1/1/1			
38	Ключ	8		1100111010001010010	0101000001010101010101	1001100101001100010100					1/0/1	1/1/1			
39	Ключ	9		1110011101000101001	1010100000101010101010	0100110010100110001010					1/1/0	1/1/1			
40	Ключ	0		0111001110100010100	1101010000010101010101	1010011001010011001010					0/1/1	1/1/1			
41	Ключ	1		1111100111010001010	1010101000001010101010	1001001100101001100010					1/1/1	1/1/1			

Продолжение таблицы 4. Такты 42–55.

№	Этап		x	R1 19b	R2 22b	R3 23b	1	2	3		ОС 1/2/3	Сдвиг 1/2/3			
42	Ключ	2		1111110011101000101	1101010100000101010101	1100100110010100110010					1/1/1	1/1/1			
43	Ключ	3		0111111001110100010	1110101010000010101010	0110010011001010011000 1					0/1/0	1/1/1			
44	Ключ	4		0111111100111010001	1011010101000001010101	1111001001100101001100 0					0/1/1	1/1/1			
45	Ключ	5		1011111110011101000	1101101010100000101010	0111100100110010100110 0					1/1/0	1/1/1			
46	Ключ	6		1101111111001110100	1110110101010000010101	0011110010011001010011 0					1/1/0	1/1/1			
47	Ключ	7		0010111111100111010	1011011010101000001010	0101111001001100101001 1					0/1/0	1/1/1			
48	Ключ	8		0101011111110011101	1001101101010100000101	0110111100100110010100 1					0/1/0	1/1/1			
49	Ключ	9		0110101111111001110	1000110110101010000010	0111011110010011001010 0					0/1/0	1/1/1			
50	Ключ	0		0111010111111100111	1000011011010101000001	0111101111001001100101 0					0/1/0	1/1/1			
51	Ключ	1		0011101011111110011	1100001101101010100000	0011110111100100110010 1					0/1/0	1/1/1			
52	Ключ	2		1101110101111111001	0010000110110101010000	1101111011110010011001 0					1/0/1	1/1/1			
53	Ключ	3		0110111010111111100	0001000011011010101000	1110111101111001001100 1					0/0/1	1/1/1			
54	Ключ	4		0011011101011111110	0000100001101101010100	0111011110111001001100 0					0/0/0	1/1/1			
55	Ключ	5		1001101110101111111	0000010000110110101010	0011101111011100100110 0					1/0/0	1/1/1			

Продолжение таблицы 4. Такты 56–69.

№	Этап		x	R1 19b	R2 22b	R3 23b	1	2	3		ОС 1/2/3	Сдвиг 1/2/3			
56	Ключ	6		0100110111010111111	1000001000011011010101	1001110111011110010011					0/1/1	1/1/1			
57	Ключ	7		0110011011101011111	1000000100001101101010	1000111011111011001001					0/1/1	1/1/1			
58	Ключ	8		1011001101110101111	1100000010000110110101	1100011101111011111100100					1/1/1	1/1/1			
59	Ключ	9		0001100110111010111	1010000001000011011010	0010001110111110111110010					0/1/0	1/1/1			
60	Ключ	0		1000110011011101011	1101000000100001101101	0001000111011110111111001					1/1/0	1/1/1			
61	Ключ	1		1100011001101110101	1110100000010000110110	0000100011101111011111100					1/1/0	1/1/1			
62	Ключ	2		1110001100110111010	1111010000001000011011	1000010001110111101111110					1/1/1	1/1/1			
63	Ключ	3		0011000110011011101	0011101000000100001101	0000001000111011111101111					0/0/0	1/1/1			
64	Ключ	4		0101100011001101110	1101110100000010000110	1100000100011101111011111					0/1/1	1/1/1			
65	Кадр			1010110001100110111	1110111010000001000011	0110000100011101111011					1/1/0	1/1/1			
66	Кадр			0101011000110011011	0111011101000000100001	0011000001000111011111101					0/0/0	1/1/1			
67	Кадр			0010101100011001101	1011101110100000010000	0001100000100011101111 0					0/1/0	1/1/1			
68	Кадр			0001010110001100110	0101110111010000001000	0000110000010001110111 1					0/0/0	1/1/1			
69	Кадр			1000101011000110011	0010111011101000000100	1000011000001000111011 1					1/0/1	1/1/1			

Продолжение таблицы 4. Такты 70–83.

№	Этап		x	R1 19b	R2 22b	R3 23b	1	2	3		ОС 1/2/3	Сдвиг 1/2/3			
70	Кадр			1100010101100011001	0001011101110100000010	1100001100000100011101 1					1/0/1	1/1/1			
71	Кадр			1110001010110001100	1000101110111010000001	1110000110000100011101					1/1/1	1/1/1			
72	Кадр			1111000101011000110	1100010111011101000000	1111000011000001000111 0					1/1/1	1/1/1			
73	Кадр			0111100010101100011	0110001011101110100000	0111100001100000100011 1					0/0/0	1/1/1			
74	Кадр	0		1011110001010110001	0011000101110111010000	1011110000110000010001 1					1/0/1	1/1/1			
75	Кадр	1		0101111000101011000	0001100010111011101000	0101111000011000001000 1					0/0/0	1/1/1			
76	Кадр	2		0010111100010101100	0000110001011101110100	1010111100001100000100 0					0/0/1	1/1/1			
77	Кадр	3		0001011110001010110	0000011000101110111010	1101011110000110000010 0					0/0/1	1/1/1			
78	Кадр	4		0000101111000101011	1000001100010111011101	0110101111000011000001 0					0/1/0	1/1/1			
79	Кадр	5		1100010111100010101	1000000110001011101110	0111010111100001100000 1					1/1/0	1/1/1			
80	Кадр	6		0110001011110001010	1100000011000101110111	0011101011110000110000 0					0/1/0	1/1/1			
81	Кадр	7		1011000101111000101	0110000001100010111011	0001110101111000011000 0					1/0/0	1/1/1			
82	Кадр	8		0001100010111100010	0111000000110001011101	1100111010111100001100 0					0/0/1	1/1/1			
83	Кадр	9		0100110001011110001	1111100000011000101110	0010011101011110000110 0					0/1/0	1/1/1			

Продолжение таблицы 4. Такты 84–97.

№	Этап		x	R1 19b	R2 22b	R3 23b	1	2	3		ОС 1/2/3	Сдвиг 1/2/3			
84	Кадр	0		0010011000101111000	11111100000011000010111	00010011101011110000110					0/1/0	1/1/1			
85	Кадр	1		1101001100010111100	00111110000001100010111	11001001110101111000011					1/0/1	1/1/1			
86	Кадр	2		0010100110001011110	01011111000000110001010	10100100111010111100001					0/0/1	1/1/1			
87	Холостой ход			0001010011000101111	01011111000000110001010	11010010011101011110000					0/1/1	1/0/1			
88	Холостой ход			0000101001100010111	01011111000000110001010	01101001001110101111000					0/1/0	1/0/1			
89	Холостой ход			1000010100110001011	10101111100000011000010	01101001001110101111000					1/1/1	1/1/0			
90	Холостой ход			0100001010011000101	11010111110000000110001	01101001001110101111000					0/1/1	1/1/0			
91	Холостой ход			0010000101001100010	11010111110000000110001	10110100100111010111100					0/1/1	1/0/1			
92	Холостой ход			0001000010100110001	11101011111000000011000	11011010010011101011110					0/1/1	1/1/1			
93	Холостой ход			0000100001010011000	01110101111100000011000	11011010010011101011110					0/0/0	1/1/0			
94	Холостой ход			0000010000101001100	01110101111100000011000	01101101001001110101111					0/0/0	1/0/1			
95	Холостой ход			0000010000101001100	00111010111110000000110	00110110100100111010111					1/0/0	0/1/1			
96	Холостой ход	0		1000001000010100110	00111010111110000000110	10011011010010011101011					1/1/1	1/0/1			
97	Холостой ход	1		1100000100001010011	00111010111110000000110	11001101101001001110101					1/1/1	1/0/1			

Продолжение таблицы 4. Такты 98–111.

№	Этап		x	R1 19b	R2 22b	R3 23b	1	2	3		ОС 1/2/3	Сдвиг 1/2/3			
98	Холостой ход	2		1100000100001010011	1001110101111100000011	1110011011010010011010					0/1/1	0/1/1			
99	Холостой ход	3		0110000010000101001	1001110101111100000011	1111001101101001001101					0/0/1	1/0/1			
100	Холостой ход	4		0011000001000010100	0100111010111110000001	1111100110110100100110					0/0/1	1/1/1			
101	Холостой ход	5		0011000001000010100	1010011101011111000000	1111110011011010010011					1/1/1	0/1/1			
102	Холостой ход	6		1001100000100001010	0101001110101111100000	1111111001101101001001					1/0/1	1/1/1			
103	Холостой ход	7		1001100000100001010	0010100111010111110000	0111111100110110100100					1/0/0	0/1/1			
104	Холостой ход	8		1100110000010000101	0001010011101011111000	0111111100110110100100					1/0/0	1/1/0			
105	Холостой ход	9		1100110000010000101	0000101001110101111100	0011111110011011010010					0/0/0	0/1/1			
106	Холостой ход	0		0110011000001000010	0000101001110101111100	0001111111001101101001					0/0/0	1/0/1			
107	Холостой ход	1		1011001100000100001	0000101001110101111100	0000111111100110110100					1/0/0	1/0/1			
108	Холостой ход	2		1011001100000100001	0000010100111010111110	0000011111110011011010					0/0/0	0/1/1			
109	Холостой ход	3		1011001100000100001	1000001010011101011111	0000001111111001101101					0/1/0	0/1/1			
110	Холостой ход	4		0101100110000010000	0100000101001110101111	0000001111111001101101					0/0/0	1/1/0			
111	Холостой ход	5		0010110011000001000	0100000101001110101111	0000000111111100110110					0/0/0	1/0/1			

Продолжение таблицы 4. Такты 112–125.

№	Этап		x	R1 19b	R2 22b	R3 23b	1	2	3		ОС 1/2/3	Сдвиг 1/2/3			
112	Холостой ход	6		0001011001100000100	0100000101001110101111	1000000011111100110110					0/0/1	1/0/1			
113	Холостой ход	7		1000101100110000010	0010000010100111010111	1000000011111100110110					1/0/0	1/1/0			
114	Холостой ход	8		1000101100110000010	0001000001010011101011	0100000001111110011011					1/0/0	0/1/1			
115	Холостой ход	9		1100010110011000001	0000100000101001110101	0100000001111111100110					1/0/0	1/1/0			
116	Холостой ход	0		1110001011001100000	1000010000010100111010	0010000000111111110011					1/1/0	1/1/1			
117	Холостой ход	1		1111000101100110000	1000010000010100111010	0001000000011111100110					1/1/0	1/0/1			
118	Холостой ход	2		1111100010110011000	1100001000001010011101	0000100000001111110011					1/1/0	1/1/1			
119	Холостой ход	3		1111100010110011000	1110000100000101001110	0000010000000111111001					0/1/0	0/1/1			
120	Холостой ход	4		1111100010110011000	1111000010000010100111	1000001000000111111100					0/1/1	0/1/1			
121	Холостой ход	5		1111100010110011000	0111100001000001010011	1100000100000011111110					0/0/1	0/1/1			
122	Холостой ход	6		1111100010110011000	0011110000100000101001	1110000010000000111111					0/0/1	0/1/1			
123	Холостой ход	7		0111110001011001100	1001111000010000010100	1110000010000000111111					0/1/1	1/1/0			
124	Холостой ход	8		1011111000101100110	0100111100001000001010	1111000001000000111111					1/0/1	1/1/1			
125	Холостой ход	9		1101111100010110011	1010011110000100000101	1111100000100000011111					1/1/1	1/1/1			

Продолжение таблицы 4. Такты 126–139.

№	Этап		x	R1 19b	R2 22b	R3 23b	1	2	3		ОС 1/2/3	Сдвиг 1/2/3			
126	Холостой ход	0		1110111110001011001	11010011110000010000010	11111000001000000011111					1/1/1	1/1/0			
127	Холостой ход	1		1111011111000101100	11010011110000010000010	11111100000100000001111					1/1/1	1/0/1			
128	Холостой ход	2		1111011111000101100	11101001111000010000001	11111110000010000000111					0/1/1	0/1/1			
129	Холостой ход	3		0111101111100010110	11110100111100001000000	11111110000010000000111					0/1/1	1/1/0			
130	Холостой ход	4		0011110111110001011	01111010011110000010000	11111110000010000000111					0/0/1	1/1/0			
131	Холостой ход	5		0001111011111000101	00111101001111000010000	11111110000010000000111					0/0/1	1/1/0			
132	Холостой ход	6		0000111101111100010	00011110100111100000100	11111110000010000000111					0/0/1	1/1/0			
133	Холостой ход	7		0000011110111110001	00001111010011110000010	11111111000001000000011					0/0/1	1/1/1			
134	Холостой ход	8		0000011110111110001	10000111101001111000001	11111111100000100000001					0/1/1	0/1/1			
135	Холостой ход	9		0000001111011111000	11000011110100111100000	11111111100000100000001					0/1/0	1/1/0			
136	Холостой ход	0		0000001111011111000	01100001111010011110000	01111111110000010000000					1/0/0	0/1/1			
137	Холостой ход	1		1000000111101111100	00110000111101001111000	01111111110000010000000					1/0/1	1/1/0			
138	Холостой ход	2		0100000011110111110	00011000011110100111100	01111111110000010000000					0/0/1	1/1/0			
139	Холостой ход	3		1010000001111011111	10001100001111010011111	01111111110000010000000					1/1/1	1/1/0			

Продолжение таблицы 4. Такты 140–153.

№	Этап		x	R1 19b	R2 22b	R3 23b	1	2	3		ОС 1/2/3	Сдвиг 1/2/3			
140	Холостой ход	4		1101000000111101111	1000110000111101001111	10111111111000001000000					1/0/1	1/0/1			
141	Холостой ход	5		1101000000111101111	0100011000011110100111	11011111111100000100000					0/0/1	0/1/1			
142	Холостой ход	6		0110100000011110111	0010001100001111010011	11011111111100000100000					0/0/1	1/1/0			
143	Холостой ход	7		0011010000001111011	0001000110000111101001	11011111111100000100000					0/0/1	1/1/0			
144	Холостой ход	8		1001101000000111101	1000100011000011110100	11011111111100000100000					1/1/1	1/1/0			
145	Холостой ход	9		1100110100000011110	0100010001100001111010	11011111111100000100000					1/0/1	1/1/0			
146	Холостой ход	0		1100110100000011110	1010001000110000111101	11101111111110000010000					0/1/1	0/1/1			
147	Холостой ход	1		1100110100000011110	1101000100011000011110	11110111111111000001000					0/1/1	0/1/1			
148	Холостой ход	2		0110011010000001111	1110100010001100001111	11110111111111000001000					0/1/1	1/1/0			
149	Холостой ход	3		1011001101000000111	1110100010001100001111	11111011111111100000100					1/0/1	1/0/1			
150	Холостой ход	4		1101100110100000011	0111010001000110000111	11111011111111100000100					1/0/0	1/1/0			
151	Холостой ход	5		0110110011010000001	0111010001000110000111	01111101111111110000010					0/0/0	1/0/1			
152	Холостой ход	6		1011011001101000000	0111010001000110000111	00111110111111111000001					1/0/0	1/0/1			
153	Холостой ход	7		0101101100110100000	0011101000100011000011	00111110111111111000001					0/0/1	1/1/0			

Продолжение таблицы 4. Такты 154–167.

№	Этап		x	R1 19b	R2 22b	R3 23b	1	2	3		ОС 1/2/3	Сдвиг 1/2/3			
154	Холостой ход	8		0101101100110100000	0001110100010001100001	10011111011111111100000					1/0/1	0/1/1			
155	Холостой ход	9		1010110110011010000	1000111010001000110000	10011111011111111100000					1/1/1	1/1/0			
156	Холостой ход	0		0101011011001101000	1000111010001000110000	11001111101111111110000					0/0/1	1/0/1			
157	Холостой ход	1		1010101101100110100	1000111010001000110000	11100111110111111111000					1/0/1	1/0/1			
158	Холостой ход	2		0101010110110011010	0100011101000100011000	11110011111011111111100					0/0/1	1/1/1			
159	Холостой ход	3		1010101011011001101	0100011101000100011000	01111001111101111111110					1/0/0	1/0/1			
160	Холостой ход	4		0101010101101100110	0100011101000100011000	10111100111101111111111					0/0/1	1/0/1			
161	Холостой ход	5		1010101010110110011	0010001110100010001100	10111100111101111111111					1/0/1	1/1/0			
162	Холостой ход	6		1101010101011011001	0001000111010001000110	11011110011111011111111					1/0/1	1/1/1			
163	Холостой ход	7		1110101010101101100	1000100011101000100011	11011110011111011111111					1/1/1	1/1/0			
164	Холостой ход	8		0111010101010110110	0100010001110100010001	11101111001111101111111					0/0/1	1/1/1			
165	Холостой ход	9		0111010101010110110	1010001000111010001000	01110111100111110111111					1/1/0	0/1/1			
166	Холостой ход	0		1011101010101011011	1010001000111010001000	00111011111001111011111					1/0/0	1/0/1			
167	Холостой ход	1		0101110101010101101	0101000100011101000100	00111011111001111011111					0/0/0	1/1/0			

Продолжение таблицы 4. Такты 168–181.

№	Этап		x	R1 19b	R2 22b	R3 23b	1	2	3		ОС 1/2/3	Сдвиг 1/2/3			
168	Холостой ход	2		1010111010101010110	0010100010001110100010	0001110111100111101111					1/0/0	1/1/1			
169	Холостой ход	3		0101011101010101011	0010100010001110100010	0000111011110011110111					0/1/0	1/0/1			
170	Холостой ход	4		1010101110101010101	1001010001000111010001	0000111011110011110111					1/1/1	1/1/0			
171	Холостой ход	5		0101010111010101010	1001010001000111010001	1000011101111001111011					0/1/1	1/0/1			
172	Холостой ход	6		0010101011101010101	1001010001000111010001	1100001110111100111101					0/1/1	1/0/1			
173	Холостой ход	7		0001010101110101010	1001010001000111010001	1110000111011110011110					0/1/1	1/0/1			
174	Холостой ход	8		0000101010111010101	1100101000100011101000	1111000011101111001111					0/1/1	1/1/1			
175	Холостой ход	9		0000010101011101010	0110010100010001110100	1111100001110111001111					0/0/1	1/1/1			
176	Холостой ход	0		0000001010101110101	0011001010001000111010	1111100001110111001111					0/0/1	1/1/0			
177	Холостой ход	1		1000000101010111010	0011001010001000111010	1111110000111011110011					1/1/1	1/0/1			
178	Холостой ход	2		0100000010101011101	1001100101000100011101	1111110000111011110011					0/1/1	1/1/0			
179	Холостой ход	3		0010000001010101110	1001100101000100011101	1111111000011101111001					0/1/1	1/0/1			
180	Холостой ход	4		1001000000101010111	1100110010100010001110	0111111100001101111001					1/1/0	1/1/1			
181	Холостой ход	5		1100100000010101011	1100110010100010001110	0011111110000111011110					1/1/0	1/0/1			

Продолжение таблицы 4. Такты 182–195.

№	Этап		x	R1 19b	R2 22b	R3 23b	1	2	3		ОС 1/2/3	Сдвиг 1/2/3			
182	Холостой ход	6		1110010000001010101	1100110010100010001110	00011111110000111011110					1/1/0	1/0/1			
183	Холостой ход	7		0111001000000101010	1100110010100010001110	10001111111000011101111					0/1/1	1/0/1			
184	Холостой ход	8		0111001000000101010	1110011001010001000111	01000111111100001110111					0/1/0	0/1/1			
185	Холостой ход	9		0011100100000010101	0111001100101000100011	01000111111100001110111					0/0/0	1/1/0			
186	Холостой ход	00		0011100100000010101	0011100110010100010001	00100011111110000111011					0/0/0	0/1/1			
187	Генерация			0001110010000001010	1001110011001010001000	00100011111110000111011					0/1/1	1/1/0			
188	Генерация			1000111001000000101	1001110011001010001000	10010001111111000011101					1/0/1	1/0/1			
189	Генерация			0100011100100000010	0100111001100101000100	10010001111111000011101					0/0/1	1/1/0			
190	Генерация			0100011100100000010	0010011100110010100010	11001000111111100001110					1/0/1	0/1/1			
191	Генерация			0100011100100000010	1001001110011001010001	01100100011111110000111					1/1/0	0/1/1			
192	Генерация			1010001110010000001	1100100111001100101000	01100100011111110000111					1/1/1	1/1/0			
193	Генерация			1101000111001000000	1100100111001100101000	10110010001111111000011					1/0/1	1/0/1			
194	Генерация			0110100011100100000	1100100111001100101000	01011001000111111100001					0/0/0	1/0/1			
195	Генерация			0110100011100100000	0110010011100110010100	00101100100011111110000					1/0/0	0/1/1			

Продолжение таблицы 4. Такты 196–209.

№	Этап		x	R1 19b	R2 22b	R3 23b	1	2	3		ОС 1/2/3	Сдвиг 1/2/3			
196	Генерация	0		1011010001110010000	0011001001110011001010	0010110010001111110000					1/0/0	1/1/0			
197	Генерация	1		0101101000111001000	0011001001110011001010	0001011001000111111000					0/1/0	1/0/1			
198	Генерация	2		0010110100011100100	0011001001110011001010	0000101100100011111100					0/1/0	1/0/1			
199	Генерация	3		0010110100011100100	1001100100111001100101	0000010110010001111110					0/1/0	0/1/1			
200	Генерация	4		0001011010001110010	1001100100111001100101	1000001011001000111111					0/1/1	1/0/1			
201	Генерация	5		0000101101000111001	1100110010011100110010	1000001011001000111111					0/1/1	1/1/0			
202	Генерация	6		0000010110100011100	1110011001001110011001	1100000101100100011111					0/1/1	1/1/1			
203	Генерация	7		1000001011010001110	1110011001001110011001	0110000010110010011111					1/1/0	1/0/1			
204	Генерация	8		0100000101101000111	1110011001001110011001	1011000001011001000111					0/1/1	1/0/1			
205	Генерация	9		1010000010110100011	1111001100100111001100	1101100000101100100011					1/1/1	1/1/1			
206	Генерация	0		1101000001011010001	0111100110010011100110	1110110000010110010011					1/0/1	1/1/1			
207	Генерация	1		1110100000101101000	1011110011001001110011	0111011000001011001000					1/1/0	1/1/1			
208	Генерация	2		1111010000010110100	0101111001100100111001	1011101100000101100100					1/0/1	1/1/1			
209	Генерация	3		0111101000001011010	0101111001100100111001	1101110110000010110010					0/1/1	1/0/1			

Продолжение таблицы 4. Такты 210–223.

№	Этап		x	R1 19b	R2 22b	R3 23b	1	2	3		ОС 1/2/3	Сдвиг 1/2/3			
210	Генерация	4		1011110100000101101	0101111001100100111001	0110111011000001011001 0					1/1/0	1/0/1			
211	Генерация	5		1101111010000010110	0101111001100100111001	1011011101100000101100 1					1/1/1	1/0/1			
212	Генерация	6		0110111101000001011	1010111100110010011100	0101101110110000010110 0					0/1/0	1/1/1			
213	Генерация	7		0110111101000001011	0101011110011001001110	0010110111011000001011 0					0/0/0	0/1/1			
214	Генерация	8		0011011101000001 01	1010101111001100100111	1001011011101100000101 1					0/1/1	1/1/1			
215	Генерация	9		0001101111010000010	1010101111001100100111	0100101101110110000010 1					0/0/0	1/0/1			
216	Генерация	0		1000110111101000001	1010101111001100100111	1010010110111011000001 0					1/0/1	1/0/1			
217	Генерация	1		1100011011110100000	1010101111001100100111	0101001011011101100000 1					1/0/0	1/0/1			
218	Генерация	2		1100011011110100000	0101010111100110010011	1010100101101110110000 0					1/0/1	0/1/1			
219	Генерация	3		1110001101111010000	0010101011110011001001	1101010010110111011000 0					1/0/1	1/1/1			
220	Генерация	4		1110011011111010000	1001010101111001100100	0110101001011011101100 0					0/1/0	0/1/1			
221	Генерация	5		0111000110111101000	1001010101111001100100	0011010100101101110110 0					0/0/0	1/0/1			
222	Генерация	6		1011100011011110100	0100101010111100110010	0001101010010110111011 0					1/0/0	1/1/1			
223	Генерация	7		0101110001101111010	1010010101011110011001	0001101010010110111011 0					0/1/0	1/1/0			

Продолжение таблицы 4. Такты 224–237.

№	Этап		x	R1 19b	R2 22b	R3 23b	1	2	3		ОС 1/2/3	Сдвиг 1/2/3			
224	Генерация	8		0010111000110111101	1101001010101111001100	0000110101000101101111011					0/1/0	1/1/1			
225	Генерация	9		1001011100011011110	1101001010101111001100	1000011010100101101111101					1/0/1	1/0/1			
226	Генерация	0		1001011100011011110	0110100101010111100110	0100001101010010110111110					0/0/0	0/1/1			
227	Генерация	1		0100101110001101111	1011010010101011110011	1010000110101001011011111					0/1/1	1/1/1			
228	Генерация	2		0010010111000110111	0101101001010101111001	0101000011010100101101111					0/0/0	1/1/1			
229	Генерация	3		0010010111000110111	1010110100101010111100	0010100001101010010110111					0/1/0	0/1/1			
230	Генерация	4		0001001011100011011	0101011010010101011110	0001010000110101001011110					0/0/0	1/1/1			
231	Генерация	5		0000100101110001101	0101011010010101011110	0000101000011010100101111					0/1/0	1/0/1			
232	Генерация	6		0000010010111000110	1010101101001010101111	0000010100001101010010111					0/1/0	1/1/1			
233	Генерация	7		0000010010111000110	0101010110100101010111	1000001010000110101001011					0/0/1	0/1/1			
234	Генерация	8		0000001001011100011	0010101011010010101011	1000001010000110101001011					0/0/1	1/1/0			
235	Генерация	9		1000000100101110001	0001010101101001010101	1100000101000011010100111					1/0/1	1/1/1			
236	Генерация	0		0100000010010111000	0001010101101001010101	0110000010100001101010011					0/1/0	1/0/1			
237	Генерация	1		1010000001001011100	1000101010110100101010	1011000001010000110101011					1/1/1	1/1/1			

Продолжение таблицы 4. Такты 238–251.

№	Этап		x	R1 19b	R2 22b	R3 23b	1	2	3		ОС 1/2/3	Сдвиг 1/2/3			
238	Генерация	2		1101000000100101110	1000101010110100101010	1101100000101000011010 1					1/1/1	1/0/1			
239	Генерация	3		1101000000100101110	1100010101011010010101	0110110000010100001101 0					1/1/0	0/1/1			
240	Генерация	4		1110100000010010111	1110001010101101001010	1011011000001010000110 1					1/1/1	1/1/1			
241	Генерация	5		1111010000001001011	1110001010101101001010	0101101100000101000011 0					1/1/0	1/0/1			
242	Генерация	6		0111101000000100101	1110001010101101001010	1010110110000010100001 1					0/1/1	1/0/1			
243	Генерация	7		1011110100000010010	1110001010101101001010	1101011011000001010000 1					1/1/1	1/0/1			
244	Генерация	8		1101111010000001001	1110001010101101001010	1110101101100000101000 0					1/1/1	1/0/1			
245	Генерация	9		1110111101000000100	1111000101010110100101	1111010110110000010100 0					1/1/1	1/1/1			
246	Генерация	0		1111011110100000010	1111100010101011010010	1111010110110000010100 0					1/1/1	1/1/0			
247	Генерация	1		1111101111010000001	1111110001010101101001	1111101011011000001010 0					1/1/1	1/1/1			
248	Генерация	2		1111101111010000001	1111111000101010110100	1111110101101100000101 0					1/1/1	0/1/1			
249	Генерация	3		1111110111101000000	0111111100010101011010	0111111010110110000010 1					1/0/0	1/1/1			
250	Генерация	4		0111111011110100000	0111111100010101011010	0011111101011011000001 0					0/1/0	1/0/1			
251	Генерация	5		0111111011110100000	1011111110001010101101	0001111110101101100000 1					1/1/0	0/1/1			

Продолжение таблицы 4. Такты 252–265.

№	Этап		x	R1 19b	R2 22b	R3 23b	1	2	3		ОС 1/2/3	Сдвиг 1/2/3			
252	Генерация	6		1011111101111010000	10111111100010101101101	00001111101010110110000 0					1/1/0	1/0/1			
253	Генерация	7		0101111101111101000	11011111110001010110	10000111111010110110000					0/1/1	1/1/1			
254	Генерация	8		1010111111011110100	11011111110001010110	11000011111101011011000					1/1/1	1/0/1			
255	Генерация	9		0101011111101111010	11011111110001010110	11100001111110101101100					0/1/1	1/0/1			
256	Генерация	0		0010101111110111101	11011111110001010110	01110000111111010110110					0/1/0	1/0/1			
257	Генерация	1		1001010111111011110	11011111110001010110	00111000011111101011011					1/1/0	1/0/1			
258	Генерация	2		0100101011111101111	11011111110001010110	00011100001111110101101					0/1/0	1/0/1			
259	Генерация	3		0010010101111110111	11011111110001010110	00001110000111111010110					0/1/0	1/0/1			
260	Генерация	4		0001001010111111011	11101111111000101011	00000111000011111101011					0/1/0	1/1/1			
261	Генерация	5		1000100101011111101	01110111111100010101	00000111000011111101011					1/0/1	1/1/0			
262	Генерация	6		1100010010101111110	01110111111100010101	10000011100001111110101					1/1/1	1/0/1			
263	Генерация	7		1110001001010111111	10111011111110001010	10000011100001111110101					1/1/1	1/1/0			
264	Генерация	8		0111000100101011111	10111011111110001010	11000001110000111111010					0/1/1	1/0/1			
265	Генерация	9		1011100010010101111	10111011111110001010	01100000111000011111101					1/1/0	1/0/1			

Продолжение таблицы 4. Такты 266–279.

№	Этап		x	R1 19b	R2 22b	R3 23b	1	2	3		ОС 1/2/3	Сдвиг 1/2/3			
266	Генерация	0		0101110001001010111	11011101111111100010101	00110000011100001111110					0/1/0	1/1/1			
267	Генерация	1		0101110001001010111	11101110111111100010101	00011000001110000111111					1/1/0	0/1/1			
268	Генерация	2		0101110001001010111	11110111011111111000101	10001100000111000011111					1/1/1	0/1/1			
269	Генерация	3		1010111000100101011	11110111011111111000101	11000110000011100001111					1/1/1	1/0/1			
270	Генерация	4		1101011100010010101	11110111011111111000101	11100011000001110000111					1/1/1	1/0/1			
271	Генерация	5		0110101110001001010	11110111011111111000101	01110001100000111000011					0/1/0	1/0/1			
272	Генерация	6		1011010111000100101	11111011101111111000010	01110001100000111000011					1/1/1	1/1/0			
273	Генерация	7		1101101011100010010	11111101110111111110001	01110001100000111000011					1/1/1	1/1/0			
274	Генерация	8		1101101011100010010	11111110111011111111000	10111000110000011100001					1/1/1	0/1/1			
275	Генерация	9		1110110101110001001	01111111011101111111100	10111000110000011100001					1/0/1	1/1/0			
276	Генерация	0		1111011010111000100	01111111011101111111100	11011100011000001110000					1/0/1	1/0/1			
277	Генерация	1		1111101101011100010	00111111101110111111110	01101110001100000111000					1/0/0	1/1/1			
278	Генерация	2		1111101101011100010	10011111110111011111111	00110111000110000011100					0/1/0	0/1/1			
279	Генерация	3		0111110110101110001	01001111111011101111111	00011011100011000001110					0/0/0	1/1/1			

Продолжение таблицы 4. Такты 280–293.

№	Этап		x	R1 19b	R2 22b	R3 23b	1	2	3		ОС 1/2/3	Сдвиг 1/2/3			
280	Генерация	4		0011111011010111000	0010011111110111011111	0001101110001100000111 0					0/0/1	1/1/0			
281	Генерация	5		1001111101101011100	0001001111111011101111	0001101110001100000111 0					1/0/1	1/1/0			
282	Генерация	6		1100111110110101110	0001001111111011101111	1000110111000110000011 1					1/0/1	1/0/1			
283	Генерация	7		1110011111011010111	0000100111111101110111	1000110111000110000011 1					1/0/0	1/1/0			
284	Генерация	8		1111001111101101011	0000010011111110111011	1000110111000110000011 1					1/0/0	1/1/0			
285	Генерация	9		1111100111110110101	0000001001111111011101	1000110111000110000011 1					1/0/0	1/1/0			
286	Генерация	00		1111110011111011010	1000000100111111101110	1000110111000110000011 1					1/1/0	1/1/0			
287	Генерация	01		1111111001111101101	1100000010011111110111	1000110111000110000011 1					1/1/0	1/1/0			
288	Генерация	02		1111111100111110110	0110000001001111111011	0100011011100011000001 1					1/0/0	1/1/1			
289	Генерация	03		1111111110011111011	0011000000100111111101	0100011011100011000001 1					1/0/0	1/1/0			
290	Генерация	04		1111111111001111101	1001100000010011111110	0010001101110001100000 1					1/1/0	1/1/1			
291	Генерация	05		1111111111100111110	1001100000010011111110	0001000110111000110000 0					1/1/0	1/0/1			
292	Генерация	06		1111111111110011111	1001100000010011111110	1000100011011100011000 0					1/1/1	1/0/1			
293	Генерация	07		1111111111110011111	1100110000001001111111	0100010001101110001100 0					1/1/0	0/1/1			

Продолжение таблицы 4. Такты 294–307.

№	Этап		x	R1 19b	R2 22b	R3 23b	1	2	3		ОС 1/2/3	Сдвиг 1/2/3			
294	Генерация	08		1111111111111001111	11001100000010011111111	00100010001101110001100					1/0/0	1/0/1			
295	Генерация	09		1111111111111100111	11001100000010011111111	10010001000110111000110					1/0/1	1/0/1			
296	Генерация	10		1111111111111100111	01100110000001001111111	11001000100011011100011					0/0/1	0/1/1			
297	Генерация	11		1111111111111100111	00110011000000100111111	01100100010001101110001					0/0/0	0/1/1			
298	Генерация	12		1111111111111100111	00011001100000010011111	10110010001000110111000					0/0/1	0/1/1			
299	Генерация	13		0111111111111110011	00011001100000010011111	01011001000100011011100					0/0/0	1/0/1			
300	Генерация	14		0111111111111110011	00001100110000000100111	00101100100010001101110					1/0/0	0/1/1			
301	Генерация	15		0111111111111110011	00000110011000000100111	00010110010001000110111					1/0/0	0/1/1			
302	Генерация	16		1011111111111111001	00000011001100000010011	00010110010001000110111					1/0/1	1/1/0			
303	Генерация	17		0101111111111111100	10000001100110000001000	00010110010001000110111					0/1/1	1/1/0			
304	Генерация	18		0101111111111111100	01000000110011000000100	10001011001000100011011					0/0/1	0/1/1			
305	Генерация	19		0010111111111111110	01000000110011000000100	11000101100100010001101					0/1/1	1/0/1			
306	Генерация	20		0010111111111111110	10100000011001100000010	11100010110010001000110					1/1/1	0/1/1			
307	Генерация	21		1001011111111111111	11010000001100110000000	11100010110010001000110					1/1/0	1/1/0			

Продолжение таблицы 4. Такты 308–321.

№	Этап		x	R1 19b	R2 22b	R3 23b	1	2	3		ОС 1/2/3	Сдвиг 1/2/3			
308	Генерация	22		0100101111111111111	0110100000011001100000	11100010110010001000110					0/0/0	1/1/0			
309	Генерация	23		0100101111111111111	0011010000001100110000	01110001011001000100011					0/0/0	0/1/1			
310	Генерация	24		0010010111111111111	0011010000001100110000	10111000101100100010001					0/0/1	1/0/1			
311	Генерация	25		0001001011111111111	0011010000001100110000	11011100010110010001000					0/0/1	1/0/1			
312	Генерация	26		0001001011111111111	0001101000000110011000	01101110001011001000100					0/0/0	0/1/1			
313	Генерация	27		0000100101111111111	0001101000000110011000	10110111000101100100010					0/0/1	1/0/1			
314	Генерация	28		0000010010111111111	0000110100000011001100	01011011100010110010001					0/0/0	1/1/1			
315	Генерация	29		0000010010111111111	0000011010000001100110	00101101110001011001000					0/0/0	0/1/1			
316	Генерация	30		0000010010111111111	1000001101000000110011	10010110111000101100100					0/1/1	0/1/1			
317	Генерация	31		0000001001011111111	1000001101000000110011	11001011011100010110010					0/0/1	1/0/1			
318	Генерация	32		0000000100101111111	0100000110100000011001	11001011011100010110010					0/0/0	1/1/0			
319	Генерация	33		0000000100101111111	1010000011010000001100	01100101101110001011001					0/1/0	0/1/1			
320	Генерация	34		0000000010010111111	0101000001101000000110	01100101101110001011001					0/0/0	1/1/0			
321	Генерация	35		0000000001001011111	1010100000110100000011	00110010110111000101100					0/1/0	1/1/1			

Продолжение таблицы 4. Такты 322–330.

№	Этап		x	R1 19b	R2 22b	R3 23b	1	2	3		ОС 1/2/3	Сдвиг 1/2/3			
322	Генерация	36		1000000000100101111	1010100000110100000011	10011001011011100010110					1/0/1	1/0/1			
323	Генерация	37		1000000000100101111	0101010000011010000001	11001100101101110001011					0/0/1	0/1/1			
324	Генерация	38		0100000000010010111	1010101000001101000000	11001100101101110001011					0/1/0	1/1/0			
325	Генерация	39		1010000000001001011	0101010100000110100000	11001100101101110001011					1/0/0	1/1/0			
326	Генерация	40		0101000000000100101	0010101010000011010000	11001100101101110001011					0/0/0	1/1/0			
327	Генерация	41		1010100000000010010	0001010101000001101000	11001100101101110001011					1/0/0	1/1/0			
328	Генерация	42		1101010000000001001	0000101010100000110100	11001100101101110001011					1/0/0	1/1/0			
329	Генерация	43		1101010000000001001	0000010101010000011010	01100110010110111000101					1/0/0	0/1/1			
330	Генерация	44		1110101000000000100	10000010101010000001101	00110011001011011100010					1/1/0	1/1/1			